Introduction to Database Systems

1. What Is a Database

A database stores organized data so it can be turned into useful information, and it does this more efficiently than simple files. This lesson explains the difference between data and information, why databases are important, and how they differ from traditional file systems.

What you will be able to do

- Define data and information
- Explain why organized storage matters
- Compare file systems and databases
- Identify situations where a database is preferable

What Is Data?

Data is simply the raw facts that we collect from the world around us. Think of data as the individual pieces of a puzzle that have not yet been put together. Each piece might be a number, a word, a date, a picture, or any other kind of observation. At this stage the pieces have no meaning by themselves - they are just *raw* and *unprocessed*.

> **Definition:** *Data* are raw facts, numbers, symbols, or text that have been recorded but not yet interpreted.

Why do we call these "raw"? Imagine you are baking a cake. Before you mix anything you have flour, sugar, eggs, and butter sitting in separate bowls. Each ingredient on its own is useful, but you cannot call it a cake until you combine them according to a recipe. In the same way, a single number like **42** or a word like **"apple"** is just data. It only becomes useful information after we organize, compare, or calculate with it.

Simple examples of data

| Example | What it looks like as data | |---------|---------------------------| | Temperature reading at 9 am | 23 degreesC | | Student's birth year | 2005 | | Product name in a store | "Blue T■shirt" | | A photo taken with a phone | a file of binary bits |

Notice that each entry is a single fact. It does not tell us anything about trends, relationships, or conclusions. It is simply a recorded observation.

From data to meaning

To turn data into something we can understand, we need to *process* it. Processing might involve sorting the numbers, counting how many times a word appears, or linking a birth year to an age. When we do this, the data start to form *information* - a higher■level description that can help us make decisions.

> **Note:** Data becomes information only after we apply some kind of analysis or context. Without that step, it stays just a collection of facts.

How computers store data

Computers store data in tiny units called *bits* (binary digits). A bit can be either **0** or **1**. By grouping bits together we can represent numbers, letters, and even images. For example, the

letter **A** might be stored as the binary pattern **01000001**. Even though the computer sees only 0s and 1s, we interpret those patterns as meaningful symbols.

Why we care about raw data

1. **Foundation for decisions** - Managers need sales numbers before they can decide whether to order more stock. 2. **Evidence for research** - Scientists record measurements (temperature, pressure) before they can test a hypothesis. 3. **Automation** - Software programs need data to trigger actions, such as sending a reminder when a deadline approaches.

> **In daily life:** When you check the weather app, the temperature, humidity, and wind speed you see are raw data gathered from sensors. The app then formats them into a readable forecast, turning data into useful information for you.

Common pitfalls

A frequent mistake is to treat a single data point as if it tells the whole story. Seeing that one day the temperature was **30 degreesC** does not tell you whether the week was hot or cool. You need multiple data points and some analysis to draw a reliable conclusion.

> **Common mistake:** Assuming a single piece of data represents the entire situation.

Remember the basics

> **Remember:** Data are the building blocks - raw, unprocessed facts. They become valuable only after we organize, relate, and interpret them. Keeping this distinction clear helps us ask the right questions and avoid drawing conclusions from incomplete evidence.

What Is Information?

Data on its own is just a collection of symbols - numbers, letters, dates, or codes. When we arrange those symbols in a way that tells us something about the world, we call the result **information**.

> **Definition:** Information is data that has been organized, interpreted, or combined so that it conveys meaning to a person or a system.

Why does organization matter? Imagine you have a bag of loose LEGO bricks. Each brick is a piece of data: a red 2x4, a blue 1x2, a yellow 2x2, and so on. By themselves, the bricks don't tell you anything about a building. If you sort them by color and size, then follow a set of instructions to snap them together, you create a model house. The model house is the information - it represents a real■world object in a form you can understand.

From raw numbers to a story

Take a simple list of numbers:

`23, 45, 12, 67, 34`

If you see only the list, you might think "just a random set of values." Now add a label to each number:

| Student | Score | |---------|-------| | Alice | 23 | | Bob | 45 | | Carol | 12 | | Dave | 67 | | Eva | 34 |

The table gives the numbers context - they are test scores for specific students. This context is the first step toward information.

Next, we can interpret the scores: the average score is

$$\text{Average} = \frac{23+45+12+67+34}{5}=36.2$$

Now we know how the class performed overall. The average is a *derived* piece of information because we combined the raw data using a simple formula.

> **Note:** Interpretation is the act of assigning meaning to data, often by adding labels, grouping, or performing calculations.

Combining data from different sources

Information becomes richer when we bring together data that were stored separately. Suppose we have another table with each student's attendance:

| Student | Days Present | |---------|--------------| | Alice | 18 | | Bob | 20 | | Carol | 15 | | Dave | 22 | | Eva | 19 |

If we join the two tables on the *Student* column, we get a combined view:

| Student | Score | Days Present | |---------|-------|--------------| | Alice | 23 | 18 | | Bob | 45 | 20 | | Carol | 12 | 15 | | Dave | 67 | 22 | | Eva | 34 | 19 |

Now we can ask new questions: *Do students who attend more days tend to score higher?* By looking at both columns together, we can spot a pattern - perhaps a positive correlation. This pattern is information that was hidden when the data lived in separate lists.

> **In daily life:** A nutrition label on a food package turns raw numbers (calories, grams of fat, sodium) into useful information by grouping them under headings, adding daily■value percentages, and letting you compare products quickly.

Why information matters

When data become information, we can make decisions. A teacher can identify students who need extra help, a manager can spot a product that sells poorly, and a doctor can notice a trend in patient symptoms. Without the step of turning data into information, we would be left with endless piles of numbers that are hard to act upon.

> **Common mistake:** Thinking that collecting more data automatically gives better insight. Without organizing or interpreting the data, the extra numbers remain just data, not information.

Summing up

1. **Label** the data so you know what each piece represents. 2. **Organize** it - sort, group, or place it in tables. 3. **Interpret** by calculating summaries (averages, totals) or by comparing with other data. 4. **Combine** with related data to reveal deeper patterns.

> **Remember:** Information is the bridge between raw data and useful knowledge; it appears only after we give data context, order, and meaning.

Why Store Data?

When we talk about **storing data**, we are really asking a simple question: *Where will the facts we collect live so that we can use them later?* Imagine you write down a phone number on a scrap of paper. The moment the paper gets lost, torn, or smudged, that number is gone. In a computer, data can be saved in a way that makes it much harder to lose, and much easier to find

again when we need it.

> **Note:** *Data* means raw facts such as numbers, words, dates, or pictures. It has no meaning by itself until we interpret it.

Reliability - keeping data safe

A reliable storage system protects data from accidental loss, damage, or corruption. - **Accidental loss**: a hard■drive crash, a power outage, or a user accidentally deleting a file. - **Damage**: physical wear of storage media, magnetic interference, or natural disasters. - **Corruption**: bits flipping from 0 to 1, making the stored value wrong.

To guard against these problems, computers use techniques like **backups** (making a second copy) and **redundancy** (storing the same piece of data in multiple places). Think of a library that keeps two copies of a rare book: if one copy gets damaged, the other is still available.

> **In daily life:** A family might keep a paper copy of an important contract *and* a digital scan on a cloud service. If the paper burns, the digital copy saves the day.

Searchability - finding the right piece quickly

If you keep all your facts in a single, long list, locating a single item can take a long time. Databases add **indexes**, which are like the alphabetical tabs at the back of a textbook. An index tells the system exactly where to look for a particular value, turning a search that could take minutes into one that finishes in milliseconds.

For example, consider a list of 10,000 student records. To find the record for *Alice Johnson* without an index, the system might have to check each record one by one - a process called a **linear scan**. With an index on the "name" field, the system can jump directly to the part of the storage where *Alice Johnson* is stored, using a method called **binary search**. The difference is similar to flipping through a phone book versus scanning every page of a newspaper.

$$\text{Time without index} \approx O(n)$$ $$\text{Time with index} \approx O(\log n)$$

where n is the number of records. The notation O (big■O) describes how the time grows as the amount of data grows.

Shareability - letting many people use the same data

When several people need the same information, a single, well■organized storage place makes sharing easy and safe. A **file system** can store a document that anyone can copy, but each copy becomes a separate version that can drift apart. A **database** allows many users to read and write to the same set of facts, while the system ensures that everyone sees a consistent view.

Imagine a classroom where each student writes the same homework answer on their own notebook. If the teacher changes the answer, every notebook must be updated manually - a lot of work and easy to miss someone. In a database, the answer is stored once; every student reads it from the same place, so any change is instantly visible to all.

> **Common mistake:** Thinking that saving a file on a shared drive is the same as using a database. A shared file does not automatically handle simultaneous edits, prevent duplicate entries, or provide fast search.

Putting it together

Reliable, searchable, and shareable storage solves three everyday problems:

1. **Loss prevention** - data stays safe even when hardware fails. 2. **Quick retrieval** - you can locate exactly what you need without scrolling through endless lists. 3. **Collaboration** - many users can work with the same data without creating conflicting copies.

When these needs grow larger - more records, more users, more frequent updates - the simple file■based approach becomes cumbersome, and a database becomes the natural solution. By storing data in a structured, managed way, we turn raw facts into a resource that can be trusted, accessed fast, and used together by many people.

File Systems vs. Databases

Imagine you have a drawer full of loose papers. Each paper is a *file* that you can open, read, and maybe write on. This is how most computers store data when you simply create folders and files. Now picture a well■organized filing cabinet where every paper is placed in a specific *folder* (called a *table* in a database) and each folder follows the same layout: every paper has the same sections, such as "Name", "Date", and "Amount". This cabinet is a *database system*.

When you keep data only in separate files, you must remember exactly where each piece of information lives and you have to open many files to answer a question like "How many students scored above 90 in the last exam?". With a database, you can ask the system a single *query* - a short command that tells the computer to find all rows that match a condition - and it will search the appropriate tables automatically.

> **Definition:** A *file system* is the part of an operating system that manages how data is stored in files and directories (folders). A *database system* is software that stores data in structured tables and provides tools (queries) to retrieve, update, and manage that data efficiently.

How data is organized

- **Files**: Each file is independent. If you have a list of students, you might keep one file called `students.txt`. If you also need their exam scores, you might create another file `scores.txt`. To know which score belongs to which student, you must write the student's ID in both files and manually match them later. - **Tables**: In a database, the same information is stored in *tables*. A table is like a spreadsheet: rows represent individual records (e.g., one student) and columns represent attributes (e.g., `StudentID`, `Name`, `Score`). Because every row follows the same column layout, the computer can quickly locate any piece of data.

Searching and retrieving

Suppose you want to find all students whose score is higher than 90.

- With files, you would open `scores.txt`, read each line, remember the student IDs that meet the condition, then open `students.txt` and look up each ID one by one. This is slow and error■prone.

- With a database, you write a single query such as

$$\text{SELECT Name FROM Students WHERE Score > 90;}$$

The database engine reads the `Students` table, checks the `Score` column for each row, and returns the matching names instantly.

> **Note:** A *query* is a request written in a special language (most often SQL) that tells the database what data you want. The database does the searching for you.

Updating data

If a student changes their address, in a file system you might have to edit several files (the student list, any mailing lists, etc.) and risk forgetting one. In a database, the address is stored in one column of the `Students` table; you issue an update command and the change is applied everywhere the address is used.

Reliability and concurrency

When many people need to read or write data at the same time, a plain file system can cause conflicts: two users might overwrite each other's changes, leading to lost information. Database systems include *transaction* mechanisms that ensure either all parts of a change happen together or none do, preserving data integrity.

> **In daily life:** Think of a library catalog versus a pile of books on a table. The catalog (database) lets you quickly find any book by title, author, or subject, while the pile (files) forces you to scan each book one by one.

Performance

Databases use indexes - special data structures that act like the alphabetical tabs at the front of a phone book. An index on the `Score` column lets the system jump directly to the rows with high scores, rather than scanning every row. Files have no built■in indexes, so every search may require reading the whole file.

When the simple approach is enough

If you only have a handful of tiny text files that never change and you never need to search across them, a file system is perfectly fine. It requires no extra software and is easy to understand. However, as soon as the amount of data grows, relationships between data become important, or multiple users need reliable access, a database becomes the smarter choice.

> **Remember:** Use a file system for very small, static collections of data. Switch to a database when you need structured storage, fast searching, safe updates, or multiple users to work together.

When to Use a Database

Imagine you have a small shop that sells toys. Every day you receive new shipments, sell some items, and sometimes a customer returns a product. You could write all this down on a piece of paper, but as the shop grows the paper becomes hard to read, easy to lose, and almost impossible to answer questions like "Which toy sold the most last month?" or "How many blue trucks are still in stock?"

A **database** is a tool that helps you keep large amounts of data organized so you can find answers quickly and reliably. Below are three everyday situations where using a database is not just helpful - it is essential.

> **In daily life:** *Inventory tracking* - A warehouse that stores thousands of products needs to know exactly how many items are on hand, where they are located, and when to reorder. A database records each product (its name, SKU, quantity, location) in a table, and software can instantly tell the manager "we have only 12 units of item #452 left, reorder today."

Why is this better than a simple spreadsheet? A spreadsheet can hold the data, but it does not enforce rules automatically. In a database you can define a **constraint** that the quantity column must never be negative. If someone tries to record a sale that would make the stock go below zero, the database rejects the entry and alerts the user. This prevents mistakes that could otherwise cause stockouts or over■ordering.

> **Note:** *User accounts* - Websites, mobile apps, and online games all need to remember who you are, what you are allowed to do, and what you have done. Each user's profile (username, password hash, email, preferences) is stored as a row in a **users** table. When you log in, the system checks the table for a matching username and verifies the password. Because the data lives in a database, the system can handle millions of users, enforce security rules (e.g., passwords must be at least 8 characters), and quickly retrieve your personal settings each time you visit.

If you tried to keep this information in separate text files, you would have to write custom code to search, update, and protect each file. A database does all of that automatically and safely, letting developers focus on building the features you enjoy.

> **Definition:** *Analytics* - Companies collect data about sales, website visits, sensor readings, or social■media interactions. By storing this raw data in a database, they can run **queries** - structured questions written in a language called SQL - to calculate totals, averages, trends, and patterns. For example, a query might compute the average purchase amount per customer per month:

$$ \text{AveragePurchase} = \frac{\sum \text{PurchaseAmount}}{\text{NumberOfCustomers}} $$

Because the data is indexed (organized for fast lookup), the query returns results in seconds even when the table contains billions of rows. This speed enables real■time dashboards, personalized recommendations, and data■driven decisions.

> **Common mistake:** *Thinking a database is only for big companies.* Many small businesses, schools, and hobby projects benefit from a database. Even a personal budget app that tracks expenses, categories, and dates works better with a tiny database than with scattered notes. The key is the need for **structured** data - data that follows a clear pattern - and for reliable ways to add, change, and retrieve that data.

When deciding whether to use a database, ask yourself three questions:

1. **Volume:** Do I have many records, or will I soon? 2. **Complexity:** Do I need to relate different kinds of data (e.g., customers ↔ orders ↔ products)? 3. **Speed:** Do I need answers quickly, especially while many users are accessing the system at the same time?

If the answer to any of these is "yes," a database is the right choice. It gives you consistency (the data stays correct), scalability (it can grow with you), and accessibility (many people or programs can read and write safely).

> **Remember:** A database is not a magic box; it is a well■organized collection of tables that lets you store, protect, and query data efficiently. When you see a problem that involves many pieces of information that must stay linked and up■to■date, think "database."

Figure (described): Individual puzzle pieces representing raw data items.

Figure (described): LEGO bricks assembled into a model to illustrate organized information.

Figure (described): A filing cabinet with labeled drawers compared to a simple file drawer of loose papers.

Key terms

Data: Raw facts, numbers, symbols, or text that have not been interpreted.

Information: Data that has been organized or processed to convey meaning.

Database: A structured collection of data stored in tables that can be queried efficiently.

Table: A set of rows and columns in a database, similar to a spreadsheet.

Query: A request to retrieve or manipulate data from a database.

File System: A way computers store files in folders without the relational structure of a database.

Reliability: The ability of a storage system to keep data safe from loss, damage, or corruption.

Inventory Tracking: Using a database to record product quantities, locations, and movements.

2. Data Models

This lesson introduces the core concepts of data modeling, focusing on entities, attributes, relationships, and how to represent them using an Entity■Relationship (ER) diagram. Students will learn to identify these components in a real■world scenario and draw a simple ER diagram.

What you will be able to do

- Identify entities, attributes, and relationships in a given scenario
- Distinguish between different types of relationships (one■to■one, one■to■many, many■to■many)
- Create a basic Entity■Relationship diagram using standard symbols

What Is an Entity?

An **entity** is simply "something we care about" in a database. Think of a story you have read or a situation you know well. Whenever the story mentions a person, a place, a piece of equipment, or even an event that can be talked about separately, that thing is a candidate for an entity.

> **Definition:** An entity is a thing or object of interest that can be uniquely identified within the domain we are modeling.

Why do we need this idea? When we store data, we must decide what pieces of information belong together. By grouping all facts about the same "thing" into one entity, we avoid duplication and make it easy to retrieve, update, or delete that thing later. For example, if we keep a list of students, each student is an entity. All the data that describe a particular student - name, birthdate, address - stay together, and we can find that student again by using a unique identifier such as a student ID.

Recognising entities in a story

Imagine a short story about a library:

> *Emma borrows a book from the Central Library. The book's title is "Data Adventures". Emma returns the book two weeks later.*

Read the story carefully and ask yourself: **What objects does the story talk about that we might want to keep information about?**

1. **Emma** - a person who interacts with the library. 2. **Book** - the item that can be borrowed and returned. 3. **Library** - the place that holds the books.

Each of these can be treated as an entity because we can imagine assigning a unique label to each one (e.g., a member number for Emma, an ISBN for the book, a branch code for the library).

> **In daily life:** When you fill out a form at a doctor's office, the "patient", the "doctor", and the "appointment" are all entities you would see on the paper.

What makes an entity "uniquely identified"?

To retrieve a specific entity later, we need a **key** - a piece of data that is different for every instance of that entity. In the library story, the key for the *Book* entity could be its ISBN number, because no two different books share the same ISBN. The key for the *Emma* entity could be her

library card number. The key for the *Library* entity could be its branch code.

If we tried to use a non■unique attribute, such as the book's title, we could run into trouble: many books might share the title "Data Adventures". That is why the key must be **unique**.

> **Note:** Uniqueness does not mean the attribute has to be a number; it can be any combination of characters that never repeats for the same entity type.

From story to database sketch

When you start turning a story into a database design, list all the entities first. Write them as nouns (person, place, thing, event). Then, for each noun, think about what makes it distinct. This step is called **entity identification** and it is the foundation of the whole ER (Entity■Relationship) model.

For the library example, a quick list looks like:

| Entity | Possible Key | |--------|--------------| | Person (Emma) | Library card number | | Book | ISBN | | Library | Branch code |

Once you have this list, you can move on to describing the **attributes** (the details) of each entity, and later the **relationships** that connect them (e.g., "borrows" links Person to Book). But everything starts with a clear, correct identification of entities.

> **Common mistake:** Treating an attribute (like a book's title) as an entity. Remember, an entity must be something you can single out with a unique identifier; a title alone cannot guarantee that.

Attributes of an Entity

Imagine you have a **thing** you want to keep track of - for example, a student in a school. In a database we call that thing an **entity**. An entity is just a real■world object or concept that is important for our application.

Now think about what makes that student recognizable. We can write down a list of facts such as the student's name, birth date, and email address. Each of those facts is an **attribute**.

> **Definition:** An *attribute* is a property or characteristic that describes an entity.

Attributes are the pieces of information that together give a complete picture of the entity. They are like the labels on a photo album: each label tells you something about the picture (who is in it, when it was taken, where). Without the labels you would have the picture, but you would not know the details that make it useful.

Why list attributes clearly?

When we design a database we need to decide **exactly** which attributes we will store for each entity. If we miss an important attribute, later we might have to add a new column to a table, which can be messy and may break existing queries. If we store unnecessary attributes, the database becomes larger and slower, and users may become confused about which fields are relevant.

How to write down attributes

1. **Identify the entity** - start with a clear name, e.g., *Student*. 2. **Brainstorm all facts** you might need about that entity. Write them in plain language. 3. **Turn each fact into a single word or short phrase** that can serve as a column name. 4. **Check for uniqueness** - each attribute

should hold one kind of data for the entity.

For the *Student* entity, a good list might look like:

- **StudentID** - a unique number that identifies the student (often called a *primary key*). - **FirstName** - the student's given name. - **LastName** - the family name. - **DateOfBirth** - the birth date, stored as a date value. - **Email** - the electronic mail address. - **EnrollmentDate** - the date the student started at the school.

Notice how each attribute has a clear, single meaning. We avoid putting two ideas into one attribute, such as "Name" that tries to hold both first and last names. Splitting them makes searching, sorting, and updating easier.

Types of attributes

Attributes can be classified by the kind of data they hold:

- **Simple (atomic)** - cannot be divided further, e.g., *StudentID* or *Email*. - **Composite** - can be broken into smaller parts, e.g., *Address* could be split into *Street*, *City*, *PostalCode*. - **Derived** - not stored directly but calculated from other attributes, e.g., *Age* can be derived from *DateOfBirth* using the formula

$$\text{Age} = \text{CurrentYear} - \text{Year(DateOfBirth)}$$

- **Multi■valued** - can hold more than one value for the same entity, e.g., a student might have several *PhoneNumbers*. In relational design we usually move multi■valued attributes to a separate related entity.

> **Note:** Keep attributes *atomic* whenever possible; this makes the database easier to query and maintain.

Everyday comparison

Think of a library card. The card itself is the **entity** (the patron). The printed information on the card - name, card number, expiration date - are the **attributes**. When the library wants to send a reminder, it looks at the *ExpirationDate* attribute. If the card missed that attribute, the reminder could not be sent automatically.

> **In daily life:** When you fill out an online form, each box you type into (name, address, phone) is an attribute of the person you are registering.

Common pitfalls

A frequent error is to treat a whole sentence as one attribute, like "Student lives in New York". That mixes a fact (*City = New York*) with a relationship (the student *lives in* a city). Instead, store *City* as an attribute and model the *lives■in* connection as a separate relationship between the *Student* entity and a *City* entity.

> **Common mistake:** Using vague or compound attribute names makes data ambiguous and harder to query later.

Quick checklist

- Does each attribute describe only one piece of information? - Is the attribute name clear and concise? - Have you decided whether the attribute is simple, composite, derived, or multi■valued? - Is there a primary■key attribute that uniquely identifies each entity instance?

> **Remember:** A well■defined set of attributes is the foundation of a clean, useful database. It lets you store exactly what you need, retrieve it quickly, and keep the data consistent over time.

Relationships Between Entities

When we talk about **relationships** in a database, we are describing how two kinds of things - called *entities* - are linked together. Think of an entity as a real■world object that we want to store information about, such as a **Student**, a **Course**, or a **Library Card**. A relationship tells us *how* one entity can be associated with another.

> **Definition:** A **relationship** is a connection between two (or more) entity types that describes how instances of those entities are related to each other.

Why do we need relationships? If we stored every piece of information in a single giant table, we would repeat the same data many times and waste space. By separating data into logical groups (entities) and then linking them with relationships, we keep the database tidy, avoid duplication, and make it easier to ask questions like "Which students are enrolled in which courses?"

Direction of a Relationship

A relationship has a *direction* that points from one entity to another. In most diagrams the direction is shown with an arrow, but the direction does not mean that data flows only one way; it simply indicates which side we are describing first. For example, in the relationship **enrolled in** we might draw an arrow from **Student** to **Course** to read "A student *is enrolled in* a course." The same relationship can be read backwards: "A course has many students enrolled in it."

Cardinality - How Many?

The most important property of a relationship is its **cardinality**. Cardinality tells us how many instances of one entity can be linked to how many instances of the other entity. There are three basic patterns:

1. **One■to■One (1:1)** - Each instance of Entity A is linked to at most one instance of Entity B, and vice■versa. *Example*: Each **Person** has exactly one **Passport**, and each passport belongs to exactly one person.

2. **One■to■Many (1:N)** - One instance of Entity A can be linked to many instances of Entity B, but each instance of B links back to only one A. *Example*: One **Teacher** teaches many **Classes**, but each class has only one main teacher.

3 **Many■to■Many (M:N)** - Instances of both entities can be linked to many of each other. *Example*: Students can enroll in many courses, and each course can have many students.

These patterns can be written with a simple formula:

$$ \text{Cardinality} = \frac{\text{max instances of B per A}}{\text{max instances of A per B}} $$

- For 1:1, the fraction is $1/1$. - For 1:N, it is $1/N$ (where N can be any number, often written as "many"). - For M:N, it is M/N (both sides "many").

> **In daily life:** Think of a library. A **Library Card** (entity A) is issued to one **Member** (entity B) - that's 1:1. A **Member** can borrow many **Books** (1:N). But a **Book** can be borrowed by many members over time, and a member can borrow many books at once - that's

M:N when we consider the whole borrowing history.

How to Choose the Right Cardinality

When you design an ER diagram, ask yourself two questions for each pair of entities:

1. **Can one instance of the first entity be linked to more than one instance of the second?** 2. **Can one instance of the second entity be linked to more than one instance of the first?**

If the answer to both is "yes," you need a many■to■many relationship. If only one side answers "yes," you have a one■to■many relationship. If both answer "no," you have a one■to■one relationship.

> **Common mistake:** Forgetting that a many■to■many relationship usually requires a *junction* (or associative) entity. In the student■course example, we create an **Enrollment** entity that stores each pairing of a student and a course, turning the M:N link into two 1:N links (Student -> Enrollment, Course -> Enrollment).

Why Cardinality Matters

Correct cardinality ensures that the database enforces the real■world rules we expect. If we incorrectly model a 1:N relationship as M:N, the system might allow impossible situations, such as a single teacher being assigned to multiple primary classes when the rule says each class has only one main teacher. Conversely, modeling a many■to■many situation as 1:N would prevent legitimate multiple links, like a student taking several electives.

> **Remember:** Cardinality is the "counting rule" of a relationship. Getting it right keeps your data accurate and your queries meaningful.

Entity■Relationship Diagram Basics

When we want to show how data items are related, we draw a picture called an **Entity■Relationship (ER) diagram**. Think of it as a simple map: the places on the map are the things we store (the *entities*), the labels on the places are the details about each thing (the *attributes*), and the roads that connect the places tell us how the things interact (the *relationships*).

The three basic symbols

| Symbol | What it stands for | How you draw it | |--------|--------------------|-----------------| | **Rectangle** | An **entity** - a thing we care about, such as *Student* or *Book* | A plain box with the entity name inside, written in **singular** form | | **Oval (or ellipse)** | An **attribute** - a property of an entity, like *Name* or *ISBN* | A small oval attached to its entity by a straight line; the oval contains the attribute name | | **Diamond** | A **relationship** - a verb that links two or more entities, such as *Enrolls* or *Writes* | A diamond shape placed between the entities it connects; the verb is written inside the diamond |

> **Definition:** *Entity* - a distinct object or concept that can be identified uniquely in the domain you are modelling (e.g., a single student, a single library book).

> **Note:** Every attribute belongs to exactly one entity (unless you are modelling a *weak* entity, which we will meet later).

Connecting the symbols

1. **Link an attribute to its entity** - Draw a straight line from the rectangle to the oval. If the attribute must always have a value, you can underline the attribute name (this is called a *mandatory* attribute).

2. **Link entities through a relationship** - * Place a diamond between the two (or more) rectangles. * Draw a line from each rectangle to the diamond. * Near each line, write a *cardinality* symbol that tells how many times the entity can participate in the relationship. The most common notations are:

$$1\text{ (one)}\quad\text{or}\quad N\text{ (many)}$$

For example, a line labeled "1" from *Student* to *Enrolls* means each student can be linked to **one** enrollment record at a time, while an "N" from *Course* to *Enrolls* means a course can have **many** enrollment records.

3. **Direction is not required** - The lines are usually undirected; the meaning comes from the verb inside the diamond and the cardinalities.

A tiny example on paper

Imagine a school that tracks which students take which classes.

* **Entities**: `Student`, `Class` * **Attributes**: * `Student` has *StudentID* (underlined, mandatory), *Name*, *BirthDate* * `Class` has *ClassID* (underlined), *Title*, *Room* * **Relationship**: `Registers` (the verb "registers for")

On a sheet of paper you would:

1. Draw a rectangle labeled **Student** and another labeled **Class**. 2. Attach ovals for each attribute to the appropriate rectangle. 3. Place a diamond labeled **Registers** between the two rectangles. 4. Connect the rectangles to the diamond with straight lines. 5. Write "1" on the line touching **Student** (a student can register for many classes, but each registration record belongs to one student) and "N" on the line touching **Class** (a class can have many registrations).

> **In daily life:** Think of a family photo album. The *people* are rectangles, the *details* like birthdate are ovals, and the *relationships* like "parent■of" are diamonds that connect the people. The album layout is an ER diagram for a family tree.

Using a tool

Many software packages (draw.io, Lucidchart, Microsoft Visio) already contain the three shapes. The steps are the same:

1. Drag a rectangle, type the entity name. 2. Drag ovals, type attribute names, and connect them with the line tool. 3. Drag a diamond, type the relationship verb, and connect it to the relevant rectangles. 4. Add text boxes near the connectors to indicate cardinalities (1, N, 0..1, etc.).

> **Common mistake:** Forgetting to label the cardinalities. Without "1" or "N" the diagram does not tell how many instances are allowed, and the model can be misinterpreted.

When you finish, the diagram should read like a simple story: *A **Student** (identified by StudentID) **Registers** for a **Class** (identified by ClassID). Each registration links one student to one class, but a student may have many registrations and a class may have many registrations.*

> **Remember:** An ER diagram is a visual shorthand. Keep it clear, use the standard symbols, and always add the cardinalities - then anyone can understand the structure of your data at a glance.

Putting It All Together: A Simple ER Diagram

Imagine you have a small library that lends books to students. We want to draw one picture that shows **everything** we have talked about so far: the things (entities), what we know about them (attributes), and how they are linked (relationships). Creating this picture - an **Entity■Relationship (ER) diagram** - is like making a map of a town: the towns are the entities, the street names are the attributes, and the roads are the relationships.

Step 1: List the entities

From our story we can see three main things:

1. **Student** - a person who can borrow books. 2. **Book** - an item that can be borrowed. 3. **Loan** - the event that connects a student with a book at a particular time.

Each of these will become a **rectangle** in the diagram, because rectangles are the standard shape for entities.

> **Definition:** An *entity* is a thing we want to store information about, such as a person, place, object, or event.

Step 2: Add attributes to each entity

Inside each rectangle we write the **attributes** - the pieces of data that describe the entity. Use **ovals** (or simply list them under the rectangle) for each attribute.

- **Student**: `StudentID` (primary key), `Name`, `Email`, `Major`. - **Book**: `ISBN` (primary key), `Title`, `Author`, `YearPublished`. - **Loan**: `LoanID` (primary key), `LoanDate`, `DueDate`.

The primary key is the attribute that uniquely identifies each instance of the entity, just like a house number identifies a single house on a street.

> **Note:** The primary key must be unique for every record; never use an attribute that can repeat, such as `Name`, as a primary key.

Step 3: Define the relationships

Now we connect the rectangles with **diamonds** (the standard shape for relationships) and label the connections.

1. **Borrows** - connects **Student** to **Loan**. 2. **Contains** - connects **Loan** to **Book**.

Each line between an entity and a relationship gets a **cardinality** (how many of one side can be linked to the other). We use the "crow's foot" notation:

- A student can have **many** loans, but each loan belongs to **exactly one** student. - A loan involves **exactly one** book, but a book can appear in **many** loans over time.

So we write:

- Student - (1) --- (0..*) Borrows --- (1) - Loan - Loan - (1) --- (0..*) Contains --- (1) - Book

The numbers mean: "1" = one, "0..*" = zero or many.

> **In daily life:** Think of a library card (Student) that can be used for many check■outs (Loans), while each check■out records exactly one book.

Step 4: Put it all together

Arrange the shapes on a sheet of paper (or a drawing tool) so the diagram is easy to read:

``` [Student]
■■(1)■■■<Borrows>■■■(1)■■■[Loan]■■■(1)■■■<Contains>■■■(1)■■■[Book] | | | | | v v v StudentID LoanID ISBN Name LoanDate Title Email DueDate Author Major YearPublished ```

(Here brackets `[]` represent rectangles, angle brackets `<>` represent diamonds, and the arrows show the direction of the cardinality.)

### Step 5: Check the diagram

Ask yourself:

- Does every entity have a primary key? - Are all attributes attached to the correct entity? - Do the cardinalities reflect the real■world rules?

If the answer is "yes", the ER diagram is complete.

> **Remember:** An ER diagram is a visual checklist. If you can read it and instantly know "who can borrow what, and how many times", you have built it correctly.

### Why this matters

Having a clear ER diagram is the first step to creating a real database. The diagram tells the database designer exactly what tables to create, which columns belong in each table, and how the tables will be linked with **foreign keys** (the database version of our relationships).

In formula form, a foreign key relationship can be written as

$$ \text{Loan.StudentID} \rightarrow \text{Student.StudentID} $$

meaning the `StudentID` stored in the **Loan** table must match a `StudentID` that already exists in the **Student** table.

When you later turn this diagram into SQL commands, the diagram becomes the blueprint that ensures data is stored consistently and without duplication.

By practicing with this simple library example, you have used every concept introduced earlier: entities, attributes, primary keys, relationships, and cardinalities. Now you can try drawing your own ER diagram for a different scenario - perhaps a school cafeteria, a sports league, or a music collection - and you will see how the same steps apply every time.

Figure (described): Shows entities, attributes, and relationships for a library system.

Figure (described): Illustrates the three basic ER diagram symbols.

Figure (described): Visual example of a student entity with its attributes.

## Key terms

**Entity**: A thing or object of interest that can be uniquely identified in the domain being modeled.
```

Attribute: A property or characteristic that describes an entity.

Relationship: A connection between two or more entity types that shows how their instances are related.

Primary Key: An attribute (or set of attributes) that uniquely identifies each instance of an entity.

Foreign Key: An attribute in one entity that references the primary key of another entity to establish a relationship.

ER Diagram: A visual representation of entities, attributes, and relationships using standard symbols.

Cardinality: Specifies how many instances of one entity can be associated with instances of another (e.g., one■to■many).

Entity Set: A collection of all instances of a particular entity type.

3. The Relational Model

This lesson introduces the relational model, explaining how data is organized into tables with rows, columns, keys, and integrity constraints. Students will learn the basic building blocks that make relational databases work.

What you will be able to do

- Identify tables, rows, and columns in a relational database
- Explain the purpose of primary and foreign keys
- Describe common integrity constraints such as NOT NULL and UNIQUE
- Illustrate how keys enforce relationships between tables

What Is a Table?

A table is simply a way to store a group of related facts in a tidy, rectangular shape. Imagine you have a notebook where each page lists information about a single student: their name, age, and grade. If you line up all the pages side by side, you get a grid where every column holds the same kind of detail (all names, all ages, all grades) and each row represents one student. That grid is exactly what a **table** is in a database.

> **Definition:** A table is a collection of related data organized in rows and columns.

Rows and columns explained

- **Column** - Think of a column as a vertical strip that holds one specific kind of information for every item in the table. In our student example, one column is "Name", another is "Age", and a third is "Grade". All entries under the "Age" column are numbers that tell how old each student is.

- **Row** - A row is a horizontal line that brings together all the pieces of information about a single item. One row might read: *Alice, 14, 9th*. That row tells us everything we know about Alice in this table.

Why do we separate data this way? By putting the same type of data in a column, the database can quickly find, sort, or compare that data. By keeping all facts about one item in a row, we can retrieve the whole record with a single request.

The shape of a table

If a table has *r* rows and *c* columns, the total number of cells (places where data can sit) is simply

$$ \text{cells} = r \times c $$

Each cell holds a single **value** - the smallest piece of information, such as the number 14 or the word "Alice". A value cannot be split further inside the table; if you need more detail, you create another column or another related table.

> **Note:** Every column has a **data type** that tells the system what kind of values it can store, such as *text*, *integer*, or *date*. This prevents putting a word where a number belongs.

Why tables are useful

Tables let us answer questions like:

- "What is the age of every student?" - we look down the **Age** column. - "Who is in the 9th grade?" - we scan the **Grade** column for the value 9. - "Show me all information about Alice." - we find the row where the **Name** column equals "Alice".

Because the data is organized, a computer can perform these searches much faster than if the information were written in a paragraph of text.

> **In daily life:** A spreadsheet program (like Microsoft Excel or Google Sheets) is a visual version of a database table. When you type numbers into cells arranged in rows and columns, you are creating a table that can be sorted, filtered, and calculated.

Building a table step by step

1. **Identify the entity** - Decide what you are describing (students, books, products). 2. **List the attributes** - Write down the pieces of information you need for each entity (name, price, author). Each attribute becomes a column. 3. **Create rows** - For every individual entity, fill in a row with the appropriate values.

If you skip any of these steps, the table may become confusing or incomplete.

> **Common mistake:** Adding the same piece of information in two different columns (e.g., having both "First Name" and "Full Name" when only one is needed). This creates redundancy and can lead to inconsistent data.

Putting it together

When you look at a table, you should be able to answer two simple questions instantly:

1. **What does each column represent?** - The column header tells you the attribute. 2. **What does each row represent?** - The row groups together all attributes for one specific item.

If you can answer both, you have a well■structured table ready for the next steps in relational database design, such as defining keys and relationships.

> **Remember:** A table is the fundamental building block of a relational database; everything else (keys, relationships, constraints) is built on top of this simple row■and■column structure.

Rows and Columns

A table in a relational database is a simple way to store many pieces of information that belong together. Think of the table as a **grid** made of horizontal lines (rows) and vertical lines (columns).

Each **row** (sometimes called a *tuple*) holds the data for **one single record** - that is, one complete item you want to keep track of. Each **column** (sometimes called an *attribute*) describes one property of that item.

> **Definition:** A **record** is a collection of values that together describe one entity, such as a single student, a single book, or a single sale.

Why this layout works so well: - It lets us see at a glance all the information about one entity (read across a row). - It lets us compare the same kind of information for many entities (read down a column).

Step■by■step picture

1. **Identify the entity** you want to store. Example: a school wants to keep a list of its students. 2. **List the attributes** that are important for each student. Typical attributes might be *StudentID*, *FirstName*, *LastName*, *BirthDate*, *GradeLevel*. 3. **Create a column for each attribute**. The column header is the attribute name. 4. **Add a row for each student** and fill in the values under the appropriate columns.

If we write the first three students, the table looks like this:

| StudentID | FirstName | LastName | BirthDate | GradeLevel |
|---|---|---|---|---| | 1001 | Alice | Chen | 2005■03■12 | 10 | | 1002 | Ben | Patel | 2004■11■23 | 11 | | 1003 | Carla | Nguyen | 2005■07■05 | 10 |

Reading across the first row tells us everything the database knows about Alice Chen: her ID is 1001, she was born on March 12 2005, and she is in grade 10. Reading down the *FirstName* column tells us the first names of all students.

Everyday comparison

Imagine a **spreadsheet** you might use to track your weekly allowance. - Each **row** could be a different week (Week 1, Week 2, ...). - Each **column** could be a type of expense (Food, Transport, Entertainment).

Just as you would fill in the amount you spent on food for Week 1 in the cell where the *Week 1* row meets the *Food* column, a database table stores a single value at the intersection of a row and a column.

> **In daily life:** A **menu** at a restaurant works like a tiny table. Each dish (row) has a name, a price, and a calorie count (columns).

Why rows and columns matter for queries

When you ask the database a question, you usually specify **which rows** you want and **which columns** to see. For example, the query

$$\text{SELECT FirstName, GradeLevel FROM Students WHERE GradeLevel = 10}$$

asks the system to return only the *FirstName* and *GradeLevel* columns, but only for rows where *GradeLevel* equals 10. The clear separation of rows and columns makes such filtering straightforward.

> **Note:** A column always holds values of the same *data type* (e.g., all numbers, all dates, all text). Mixing types in one column would break the relational model and cause errors.

Building intuition

- **Rows are like individual cards** in a deck, each card showing all the details of one person or object. - **Columns are like the categories** printed on the back of every card**, telling you what each position on the card means.

When you add a new student, you simply **append a new row** with the appropriate values. When you discover a new attribute, such as *EmailAddress*, you **add a new column** and then fill in the email for each existing row.

> **Common mistake:** Thinking that the order of columns matters for the data itself. In a relational table, the column names define the meaning; moving a column left or right does not change the data, only the visual layout.

Quick recap

- A **row** = one complete record (one entity). - A **column** = one attribute (one property) of that entity. - All rows share the same set of columns, so the table has a uniform structure. - This uniform grid lets the database store, retrieve, and compare data efficiently.

> **Remember:** When you can picture a table as a collection of rows (records) and columns (attributes), you have the core mental model needed to understand every later concept in the relational model.

Primary Keys

A **primary key** is a special column (or a small set of columns) that the database uses to tell every row apart from every other row in the same table. Think of it as the table's "name tag" - just as a name tag on a conference badge lets you identify one person among hundreds, a primary key lets the database identify one record among thousands.

Why do we need this? When we store data, we often want to look up, change, or delete a specific piece of information. If two rows looked exactly the same, the system would not know which one you meant. The primary key guarantees **uniqueness**: no two rows can have the same value in the primary■key column(s). This makes operations fast and reliable because the database can jump straight to the right row without scanning the whole table.

> **Definition:** A primary key is a column or combination of columns whose values are unique for every row and never null (empty).

How it works, step by step

1. **Choose a column that is naturally unique.** Some tables already have a column that is different for each record. For example, a *Students* table might have a *StudentID* that the school assigns to each student. Because every student gets a different number, *StudentID* is a good candidate.

2. **If no natural column exists, create an artificial one.** Sometimes the data we collect does not contain a naturally unique field. In a *Books* table, the title and author might repeat (different editions). We can add a new column called *BookID* and let the database automatically generate a unique number for each row. This is called a **surrogate key**.

3. **Mark the column as the primary key.** In SQL, the command looks like:

$$ CREATE TABLE Students (StudentID INT PRIMARY KEY, Name VARCHAR(50), BirthDate DATE); $$

The word **PRIMARY KEY** tells the database system to enforce the uniqueness rule and to disallow null values in that column.

4. **Use the primary key in other tables.** When another table needs to refer to a row, it does so by storing the primary■key value. For example, an *Enrollments* table might have a column *StudentID* that points back to the *Students* table. This creates a **relationship** between the two tables.

Everyday comparison

Imagine a library where each book has a unique barcode. The barcode is printed on the cover and never changes. Librarians scan the barcode to find the exact book they need, even if many books have the same title or author. The barcode is the primary key for the library's inventory system. Without it, a librarian would have to compare every detail of every book to be sure they have the right one, which would be slow and error■prone.

> **In daily life:** A primary key is like a social security number for a person - it uniquely identifies that person even if many people share the same name.

Things to watch out for

- **Never allow duplicate values.** The database will reject any attempt to insert a row whose primary■key value already exists. - **Never allow nulls.** A null means "unknown", and the database cannot use "unknown" to identify a row uniquely. - **Keep the key stable.** Changing a primary■key value is possible but can break relationships in other tables, so choose a key that is unlikely to change (e.g., an auto■generated number rather than a mutable email address).

> **Common mistake:** Using a column that can change (like an email address) as the primary key. When the email changes, every related table must be updated, which is risky and inefficient.

Quick recap

- The primary key uniquely identifies each row. - It must contain **no duplicate** values and **no nulls**. - It can be a single column or a combination of columns (called a **composite key**). - It is often used by other tables as a reference point, forming relationships.

> **Remember:** The primary key is the anchor that keeps the whole relational structure stable and searchable. Without it, the database would lose its ability to pinpoint exact records quickly and safely.

Foreign Keys and Relationships

A **foreign key** is a column (or a group of columns) in one table that refers to the **primary key** of another table. Think of it as a "pointer" that tells you where to find more information about a particular item.

When we store data in separate tables, we usually do it because each table represents a different kind of thing. For example, a *Students* table holds data about each student, while a *Courses* table holds data about each course. A student can enroll in many courses, and each course can have many students. To capture this real■world relationship we need a way to link rows from the *Students* table to rows in the *Enrollments* table, and from *Enrollments* to rows in the *Courses* table. The foreign key does exactly that.

> **Definition:** A foreign key is a column (or set of columns) in one table whose values must match values of the primary key column(s) in another table.

Why do we need this link? Without it, the database would have no way to enforce that the data we enter makes sense together. Imagine you typed a course code that does not exist in the *Courses* table; the database would happily store it unless we tell it that the column must reference a valid course. The foreign■key rule prevents such "orphan" records, keeping the data **consistent**.

Step■by■step example

1. **Create the tables** - *Students* table: `StudentID` (primary key), `Name`, `Email`. - *Courses* table: `CourseID` (primary key), `Title`, `Credits`. - *Enrollments* table: `EnrollID` (primary key), `StudentID`, `CourseID`, `Grade`.

2. **Declare foreign keys** In *Enrollments*, `StudentID` is declared as a foreign key that references `Students.StudentID`. Likewise, `CourseID` is a foreign key that references `Courses.CourseID`.

$$\text{Enrollments.StudentID} \rightarrow \text{Students.StudentID}$$
$$\text{Enrollments.CourseID} \rightarrow \text{Courses.CourseID}$$

3. **Insert data** - Add a student with `StudentID = 101`. - Add a course with `CourseID = CS101`. - When you insert an enrollment row, you must use `StudentID = 101` and `CourseID = CS101`. If you tried `CourseID = MATH999` (which does not exist), the database would reject the insert.

Everyday comparison

Think of a library. Each book has a unique barcode (the primary key). When you write a checkout record, you don't write the whole book description again; you just write the barcode on the checkout slip. That barcode is the foreign key linking the slip to the book's full entry in the catalog. If someone writes a barcode that isn't on any book, the librarian will notice the mistake immediately. The same idea works in databases: the foreign key points to the "real" record elsewhere.

> **In daily life:** A foreign key works like a reference number on a ticket that tells you which flight, train, or concert the ticket belongs to.

How relationships are expressed

- **One■to■many**: One row in the parent table (e.g., a single student) can be linked to many rows in the child table (e.g., many enrollment records). The foreign key lives in the "many" side. - **Many■to■many**: Neither table can hold a foreign key to the other directly. Instead we create a **junction table** (like *Enrollments*) that has two foreign keys, each pointing to one of the original tables. This table's rows represent each pairing of student and course.

Mathematically, a foreign■key constraint can be written as:

$$\forall r \in \text{ChildTable},\; r.\text{FK} \in \text{ParentTable}.\text{PK}$$

which reads "for every row *r* in the child table, the foreign■key value must be found among the primary■key values of the parent table."

> **Note:** When a parent row is deleted, the database can either (a) forbid the deletion, (b) automatically delete all child rows (**cascade delete**), or (c) set the foreign■key column to NULL (**set null**). Choosing the right option depends on the meaning of the data.

Why foreign keys matter

- **Data integrity**: Prevents mismatched or missing references. - **Query simplicity**: Allows you to join tables confidently, knowing the link is valid. - **Business rules**: Enforces real■world constraints automatically, reducing programming errors.

> **Remember:** A foreign key is not just a column with the same name as another table's primary key; it is a *constraint* that tells the database to enforce the link.

By using foreign keys, we turn a collection of isolated tables into a network that mirrors how objects relate to each other in the real world. This network is the foundation for powerful queries, reports, and applications built on top of a relational database.

Integrity Constraints

Imagine a table as a tidy notebook where every entry must follow the same rules. **Integrity constraints** are those rules. They tell the database *what is allowed* and *what is not* so that the data stays correct, reliable, and easy to use.

> **Definition:** *Integrity constraints* are conditions defined on a table's columns that the database system checks automatically whenever data is inserted, updated, or deleted. If a change would break a rule, the operation is rejected.

Why do we need them? Without constraints, anyone could write nonsense into the table: a student could be recorded with a negative age, two employees could share the same employee number, or a sale could refer to a product that does not exist. Such mistakes spread quickly, make reports wrong, and cause programs to crash. By enforcing constraints, the database itself becomes a guard that catches errors at the moment they happen.

NOT NULL - "must have a value"

A column declared **NOT NULL** cannot be left empty. Think of a library card: you must write your name on it; a blank card is useless. In a table of customers, the column `email` might be `NOT NULL` if every customer must have an email address for contact.

```sql CREATE TABLE Customers (CustomerID INT PRIMARY KEY, Name VARCHAR(50) NOT NULL, Email VARCHAR(100) NOT NULL); ```

If someone tries to insert a row without an email, the database says *"Error: Email cannot be null."* This prevents missing information that later queries would have to handle specially.

UNIQUE - "no duplicates"

The **UNIQUE** constraint guarantees that every value in a column (or a set of columns) appears only once. It is like a locker number in a school: two students cannot be assigned the same locker. A common use is to make sure each user name appears only once.

```sql CREATE TABLE Users (UserID INT PRIMARY KEY, Username VARCHAR(30) UNIQUE, PasswordHash CHAR(64)); ```

If a second row tries to use the same `Username`, the database rejects it. This keeps identifiers reliable for looking up records.

> **Note:** A table can have many UNIQUE columns, but only one PRIMARY KEY. The primary key is automatically UNIQUE, so you do not need to add both.

CHECK - "must satisfy a condition"

A **CHECK** constraint lets you write a logical expression that each row must satisfy. It is similar to a rule a teacher might set: "All homework scores must be between 0 and 100." In SQL you write the condition inside parentheses.

```sql CREATE TABLE Exams (ExamID INT PRIMARY KEY, StudentID INT, Score INT CHECK (Score BETWEEN 0 AND 100)); ```

Here the database evaluates the expression `Score BETWEEN 0 AND 100` for every insert or update. If someone tries to store a score of 150, the operation fails. CHECK can involve multiple columns, for example ensuring that `StartDate` is earlier than `EndDate`.

Combining constraints

Constraints can be stacked on the same column:

```sql CREATE TABLE Products (ProductID INT PRIMARY KEY, SKU VARCHAR(20) NOT NULL UNIQUE, Price DECIMAL(8,2) CHECK (Price >= 0)); ```

- `SKU` must be present (`NOT NULL`) and cannot repeat (`UNIQUE`). - `Price` must be a non■negative number (`CHECK`).

When all these rules are in place, the database itself guarantees that the data respects the business rules you care about.

> **In daily life:** Think of a school registration form that requires a student's birthdate, forbids two students from having the same ID number, and insists the grade entered must be between 1 and 12. Those are exactly the same ideas as NOT NULL, UNIQUE, and CHECK, only enforced automatically by the computer.

What happens when a constraint is violated?

If an operation breaks a constraint, the database returns an error and rolls back the change. This protects the existing good data from being corrupted. Applications can catch the error and show a friendly message to the user, guiding them to correct the input.

> **Common mistake:** Forgetting to declare a column as NOT NULL because you think "nulls are harmless." In reality, nulls can hide missing data and cause calculations to return unexpected results.

Quick recap

- **NOT NULL**: every row must supply a value. - **UNIQUE**: no two rows can share the same value (or combination of values). - **CHECK**: values must satisfy a logical condition you specify. - Constraints are checked automatically on INSERT, UPDATE, and DELETE. - They keep the database clean, reliable, and safe for analysis.

By defining these constraints once, you let the database do the heavy lifting of data validation, freeing you to focus on higher■level tasks like building queries and reports.

Putting It All Together

Imagine you have a small notebook where you write down information about the books you own and the people who borrow them. One page lists the books, another page lists the borrowers, and a third page tells you which borrower took which book. In a relational database the same idea is expressed with **tables** (the pages), **rows** (the entries on each page), and **keys** (the special columns that let the pages talk to each other).

Step 1 - Create the "Books" table

| **book_id** | title | author | year | |------------|----------------------|----------------|------| | 1 | The Little Prince | Antoine de Saint■Exupéry | 1943 | | 2 | 1984 | George Orwell | 1949 | | 3 | Pride and Prejudice | Jane Austen | 1813 |

* **book_id** is a **primary key** - a column whose value is unique for every row and never empty. It works like a barcode on a library book: no two books share the same barcode, so you can always find the exact book you need.

Step 2 - Create the "Borrowers" table

| **borrower_id** | name | email | |----------------|---------------|----------------------| | 10 | Alice Green | alice@example.com | | 11 | Bob Smith | bob@example.com | | 12 | Carol Jones | carol@example.com |

Again, **borrower_id** is the primary key for this table.

Step 3 - Create the "Loans" table that links the two

| **loan_id** | **book_id** | **borrower_id** | loan_date | |------------|------------|----------------|-----------| | 100 | 2 | 10 | 2026■10■01 | | 101 | 1 | 12 | 2026■10■03 | | 102 | 3 | 11 | 2026■10■04 |

In this table we have two **foreign keys**:

* **book_id** references **Books.book_id** * **borrower_id** references **Borrowers.borrower_id**

A foreign key is a column (or set of columns) whose values must already exist in the primary■key column of another table. It is the database's way of saying "this row is related to a row in another table".

> **Definition:** A **foreign key** enforces a *referential integrity constraint* - the rule that a value in a foreign■key column must match an existing primary■key value in the referenced table, or be NULL if the relationship is optional.

Step 4 - Adding integrity constraints

1. **Primary■key constraint** on each table guarantees uniqueness: $$\forall\;r_1, r_2 \in \text{Table},\; (r_1.\text{PK}=r_2.\text{PK}) \Rightarrow (r_1 = r_2)$$

2. **Foreign■key constraint** on *Loans.book_id* ensures you cannot record a loan for a book that is not in the *Books* table. Similarly for *Loans.borrower_id*.

If you try to insert a row like `(103, 5, 10, 2026■10■05)` into *Loans*, the database will reject it because there is no book with **book_id = 5**.

> **Note:** Constraints are checked automatically by the database engine; you do not have to write extra code to enforce them.

Step 5 - Why this matters

Think of the three tables as three spreadsheets that are linked by common columns. If you change a book's title in the *Books* table, you do **not** need to edit the *Loans* table - the link is made through the immutable **book_id**. This separation makes data easier to maintain, prevents accidental duplication, and lets you ask powerful questions, such as:

* "Which books are currently loaned out?" * "How many times has each borrower taken a book?"

Both queries can be answered by joining the tables on their keys.

> **In daily life:** A primary key is like a person's passport number - unique and permanent. A foreign key is like writing that passport number on a visa application; the visa must refer to a real passport.

Step 6 - Putting it all together in a query

To list every loan with the book title and borrower's name:

```sql SELECT L.loan_id, B.title, Br.name, L.loan_date FROM Loans AS L JOIN Books AS B ON L.book_id = B.book_id JOIN Borrowers AS Br ON L.borrower_id = Br.borrower_id; ```

The **JOIN** operation uses the foreign■key relationships to combine rows from the three tables into a single, meaningful view.

> **Remember:** The power of the relational model comes from defining clear primary keys, linking them with foreign keys, and letting the database enforce integrity automatically. When you follow these steps, your data stays consistent and your queries stay simple.

Summary and Quick Check

> **Definition:** A **relational database** stores data in **tables** (also called **relations**). Each table is a grid of **rows** (records) and **columns** (attributes). One column is chosen as the **primary key** to uniquely identify each row, and other tables can refer to that key with a **foreign key** to create **relationships**. **Integrity constraints** are rules - such as "every foreign key must match a primary key" or "no two rows may have the same primary key" - that keep the data correct.

> **In daily life:** Think of a school's student roster. The roster is a table where each row is a student and each column holds a piece of information (name, ID, grade). The student ID is the primary key because no two students share it. A separate table that lists which classes each student takes uses the student ID as a foreign key, linking the two tables just like a name tag connects a student to their locker.

> **Remember:** A primary key must be **unique** (no duplicates) and **not null** (every row must have a value). A foreign key does not have to be unique, but every non■null value must already exist as a primary key in the referenced table. These rules are the core of **referential integrity**, one of the main integrity constraints.

Quick Review

- **Tables** organize data in rows and columns, making it easy to read and query. - **Rows** represent individual records; **columns** describe the attributes of those records. - The **primary key** is the unique identifier for each row; it prevents duplicate records. - A **foreign key** points to a primary key in another table, establishing a link between tables. - **Integrity constraints** (primary■key, foreign■key, NOT NULL, UNIQUE, CHECK) protect the database from inconsistent or invalid data. - By combining tables with keys, we can model real■world relationships such as "students enroll in courses" or "employees belong to departments".

```

Conceptual Check

1. **Identify the key** You have a table **Books** with columns `ISBN`, `Title`, `Author`, `Year`. Which column should be the primary key and why?

2. **Link two tables** Suppose there is a **Orders** table with `OrderID`, `CustomerID`, `Date`, and a **Customers** table with `CustomerID`, `Name`, `Email`. Explain how you would use a foreign key to connect an order to the customer who placed it.

3. **Spot the mistake** A table **Employees** uses `PhoneNumber` as its primary key. What problem could arise from this choice?

4. **Apply an integrity rule** In a **Enrollments** table, the column `StudentID` is a foreign key referencing the **Students** table. What will happen if someone tries to insert a row with a `StudentID` that does not exist in **Students**?

5. **Combine concepts** Draw a simple mental picture of how three tables - **Students**, **Courses**, and **Enrollments** - fit together using primary and foreign keys. Which columns act as keys in each table?

Think through each question, write a short answer, and then compare your responses with the explanations above. If any answer feels unclear, revisit the definitions of primary key, foreign key, and integrity constraints - they are the building blocks that keep a relational database reliable and useful.

Figure (described): A rectangular grid showing rows as students and columns as attributes like name, age, grade.

Figure (described): Shows how a primary key in one table links to a foreign key in another table.

Figure (described): A visual checklist of common integrity rules applied to tables.

Key terms

Table: A collection of data organized in rows (records) and columns (attributes).

Row (Record): A horizontal entry that holds all attribute values for a single entity.

Column (Attribute): A vertical field that stores one type of information for every row.

Primary Key: A column or set of columns whose values are unique and non■null for each row.

Foreign Key: A column that references the primary key of another table to create a relationship.

Integrity Constraint: A rule that the database enforces to keep data valid, such as uniqueness or referential integrity.

Relation: Another name for a table in the relational model.

Tuple: Another term for a row or record in a table.

4. Designing Tables

In this lesson students learn why normalisation is essential and how to apply the first, second, and third normal forms to design clean tables. They will practice converting an unstructured table into a well■structured relational design.

What you will be able to do

- Identify repeating groups and partial dependencies in a table
- Transform a table to First Normal Form
- Transform a table to Second Normal Form
- Transform a table to Third Normal Form
- Explain how normalisation improves data integrity

What is Normalisation?

When we store information in a computer we usually put it into tables, just like a spreadsheet. If we are not careful, the same piece of data can appear many times in different rows or columns. That repetition is called **redundancy**. Redundancy is not a problem by itself, but it can cause two big headaches:

1. **Wasting space** - the same fact is stored over and over, so the file gets larger for no reason.
2. **Losing consistency** - if the repeated fact needs to change, we have to update every copy. If we miss one copy, the database now contains two different values for the same real■world item, and the data is no longer trustworthy.

> **Definition:** *Normalisation* is the systematic process of organising a database's tables so that each fact is stored only once, eliminating unnecessary redundancy and protecting data integrity.

Why does this work?

Think of a library catalogue. Instead of writing the full address of the publisher on every book record, the catalogue stores the publisher's name in a separate list and lets each book point to that list. If the publisher moves, we change the address in just one place, and every book automatically shows the new address. The same idea applies to any set of related facts: keep each distinct piece of information in its own place and let other tables refer to it.

In database language, a **table** is a collection of **rows** (also called records) and **columns** (also called fields). A **field** holds a single attribute, such as *StudentName* or *CourseCode*. When a field contains more than one piece of information - for example, a column called *PhoneNumbers* that holds "555■1234; 555■5678" - we have a **repeating group**. Repeating groups are a sign that the table is not yet normalised.

> **Note:** Reducing redundancy does not mean we throw away useful data. We simply move it to a more appropriate table and link the tables together with **keys** (special fields that identify rows uniquely).

The benefit for data integrity

Data integrity means the data stays correct and reliable over time. If the same student's address appears in ten different rows, and the student moves, we must edit ten rows. If we forget one, queries that use the address will give mixed results - some rows show the old address, others the new one. By normalising, the address lives in a single row of a *Students* table, and every other table that needs the address refers to that row. Updating the address once automatically updates every place it is used.

> **In daily life:** Imagine a contact list on your phone where you store a friend's email address in every conversation thread. If the friend changes their email, you would have to edit every thread. Instead, the phone stores the email once in the contact card and each message just points to that card. That is exactly what normalisation does for databases.

A tiny example

Suppose we start with a table that records which students are enrolled in which courses:

| StudentID | StudentName | Course1 | Course2 | |-----------|-------------|---------|---------| | 1 | Alice | Math | History | | 2 | Bob | Math | (null) |

Here the courses are stored directly in the same row as the student. If a student takes a third course, we must add another column, and the table quickly becomes messy. Moreover, the course name "Math" appears in several rows. If the department decides to rename "Math" to "Mathematics", we must change every occurrence.

A normalised design would split the data into two tables:

Students

| StudentID | StudentName | |-----------|-------------| | 1 | Alice | | 2 | Bob |

Enrollments

| StudentID | CourseName | |-----------|------------| | 1 | Math | | 1 | History | | 2 | Math |

Now the course name lives only in the *Enrollments* table, and each student appears only once in *Students*. Changing "Math" to "Mathematics" requires editing a single column in the *Enrollments* table.

> **Common mistake:** Trying to "fix" redundancy by manually deleting duplicate rows instead of redesigning the tables. Deleting rows can lose important relationships; proper normalisation keeps all relationships intact while removing unnecessary copies.

Summing up

Normalisation is not a mysterious theory - it is a practical checklist that tells us how to break a big, tangled table into smaller, well■structured pieces. By doing so we:

* Store each fact once -> less wasted space. * Keep data consistent -> easier updates, fewer errors. * Make queries clearer -> the database knows exactly where to look for each piece of

information.

> **Remember:** The goal of normalisation is to organise data so that every real■world fact has a single, unambiguous place in the database. This simple rule underpins all later concepts such as 1NF, 2NF, and 3NF, which we will explore next.

First Normal Form (1NF)

Each table in a database is meant to store facts in a tidy, predictable way. The first rule that makes this possible is called **First Normal Form (1NF)**.

> **Definition:** *A table is in 1NF when every column contains only atomic (indivisible) values and every row can be distinguished from all the others.*

What does "atomic" mean?

Think of a column as a list of boxes. Each box should hold **one single piece of information**, not a collection of items. For example, imagine a column called *PhoneNumbers* that stores the data `555■1234, 555■5678`. Here the box holds two phone numbers together, so the value is **not atomic**. To make it atomic we would store each phone number in its own row, or create a separate table that lists one phone number per row.

Why do we care about indivisible values? When a value is broken into smaller parts, the database can search, sort, and calculate much more easily. If a column mixes several pieces of data, the system cannot tell where one piece ends and the next begins, which leads to mistakes in queries and reports.

Why must each row be unique?

A row represents a single, distinct fact. If two rows look exactly the same, the database cannot know which one you are referring to when you ask for "the record for student 123". Uniqueness is usually enforced by a **primary key** - a column (or a set of columns) whose values are different for every row.

> **Note:** *A primary key can be a single column like `StudentID`, or a combination such as (`OrderID`, `ProductID`). The important thing is that the combination never repeats.*

Everyday comparison

Consider a library checkout card. - The **card number** is like a primary key: each card is unique, so the librarian can always find the right patron. - The **list of books borrowed** should be written one per line, not all jammed together in one long sentence. If the card said "Harry Potter, The Hobbit, 1984", the librarian could not quickly count how many books were borrowed or sort them alphabetically. Writing each title on its own line makes the information atomic and easy to manage.

Step■by■step check for 1NF

1. **Look at each column.** Does any cell contain more than one piece of data? - If yes, split the data into separate columns or separate rows. 2. **Identify a candidate for a primary key.** Choose a column (or group) that is never repeated. 3. **Verify uniqueness.** Scan the table (or run a query) to ensure no two rows have the same key values.

If both conditions are satisfied, the table meets 1NF.

> **Common mistake:** *Leaving a "comma■separated list" inside a column. This looks tidy on paper but breaks the atomic rule and makes later queries very hard.*

Small example

| StudentID | Name | Courses | |-----------|-----------|----------------------| | 101 | Alice Lee | Math, Science, Art | | 102 | Bob Kim | History |

The *Courses* column violates 1NF because it stores three courses in one cell for Alice. To fix it, we could create a new table:

| StudentID | Course | |-----------|----------| | 101 | Math | | 101 | Science | | 101 | Art | | 102 | History |

Now every column holds a single value, and the combination of `StudentID` and `Course` is unique, so both tables are in 1NF.

> **Remember:** *Atomic columns + a unique row identifier = First Normal Form.*

By following these simple steps, you ensure that your tables are clean, searchable, and ready for the more advanced normalisation stages that follow.

Identifying Repeating Groups

Imagine a table that stores information about a student's courses. Each row should describe **one** fact about the student, but sometimes we try to squeeze several facts into the same row. When we put more than one value into a single column, we create a **repeating group** - a collection of similar items that "repeat" inside the same cell.

> **Definition:** A *repeating group* is a set of two or more values stored together in one column, violating the rule that each column must hold a single, indivisible value.

Why is this a problem? The First Normal Form (1NF) says that every column must contain **atomic** data - data that cannot be broken down any further. If a column holds a list, the database cannot reliably search, sort, or enforce rules on each individual item.

Spotting a repeating group

Take this example of a "StudentCourses" table:

| StudentID | Name | Courses | |-----------|-----------|----------------------| | 101 | Alice | Math, History, Art | | 102 | Bob | Chemistry | | 103 | Carol | Physics, Math |

The column **Courses** contains a comma■separated list. Each entry may have a different number of courses, and the list is stored as a single text string. This is a classic repeating group.

How to recognise it: 1. Look for a column that can hold **more than one** piece of information (lists, arrays, "A;B;C", etc.). 2. Check whether the same type of data appears repeatedly inside that column. 3. Ask yourself: *If I wanted to find every student who takes "Math", can I do it with a simple query?* The answer is "no" - you would need to parse the string first.

> **In daily life:** Think of a grocery list written on a single line: "apples, bananas, carrots". If you want to count how many times "bananas" appear, you have to read the whole line and split it yourself. Storing each item on its own line (or row) makes counting automatic.

Splitting the repeating group

To bring the table into 1NF, we must **remove** the repeating group by creating separate rows for each course. The result is a new table where each row represents **one student-course pair**:

| StudentID | Name | Course | |-----------|-------|----------| | 101 | Alice | Math | | 101 | Alice | History | | 101 | Alice | Art | | 102 | Bob | Chemistry| | 103 | Carol | Physics | | 103 | Carol | Math |

Now the **Course** column holds a single, atomic value. The table satisfies 1NF because: - Every column contains only one value per row. - There are no lists or arrays hidden inside a cell.

> **Note:** You do **not** need to keep the original "Courses" column. The new structure replaces it, and the data is still the same, just organized differently.

Why the new layout is useful

1. **Easy searching** - To find all students taking "Math", the query becomes `SELECT StudentID, Name FROM StudentCourses WHERE Course = 'Math';` The database can use indexes on the **Course** column, making the search fast.

2. **Accurate counting** - `SELECT COUNT(*) FROM StudentCourses WHERE Course = 'Math';` now returns the exact number of enrollments without extra string parsing.

3. **Data integrity** - If a course name is misspelled in one row, it does not affect other rows. In the old list, a single typo could hide inside a long string and be hard to detect.

> **Common mistake:** Leaving the repeating group in place and trying to "fix" it by adding delimiters (e.g., using semicolons) instead of creating separate rows. Delimiters still keep multiple values in one cell, so the table remains not■1NF.

Step■by■step transformation

1. **Identify** the column with repeating values. 2. **Create** a new table (or modify the existing one) that has a column for the repeated item (here, **Course**). 3. **Insert** a row for each individual value, copying the other columns (StudentID, Name) unchanged. 4. **Delete** the original column that held the list.

> **Remember:** 1NF is about *atomicity* - each cell holds exactly one piece of data. Once you have removed all repeating groups, you can move on to higher normal forms (2NF, 3NF) with confidence that the foundation of your design is solid.

Second Normal Form (2NF)

When a table has a **composite primary key** - that is, a key made up of two or more columns - every non■key column should depend on the whole key, not just on a part of it.

> **Definition:** *Partial dependency* occurs when an attribute (a column) is determined by only **one** component of a composite primary key, while the primary key itself consists of **multiple** columns.

Why does this matter? If an attribute only needs part of the key to be known, then the same piece of information will be repeated in many rows. Repetition wastes space and makes it easy to introduce mistakes when the data is updated.

Spotting a partial dependency

Imagine a table **Enrollment** that records which student takes which course and the instructor for that course:

| StudentID | CourseID | Instructor | Grade | |-----------|----------|------------|-------| | 101 | CS101 | Dr. Lee | A | | 101 | MA101 | Dr. Kim | B+ | | 102 | CS101 | Dr. Lee | B |

The primary key is the pair *(StudentID, CourseID)* because a student can enroll in many courses and a course can have many students.

Look at the column **Instructor**. The instructor for a given course is the same no matter which student is enrolled. Therefore **Instructor** depends only on *CourseID*, not on *StudentID*. That is a partial dependency.

> **In daily life:** Think of a school locker system. The locker number (part of the key) tells you which hallway the locker is in, while the hallway tells you the floor. If you only need the hallway to know the floor, storing the floor for every locker is redundant - you could store the floor once per hallway instead.

Removing the partial dependency

To eliminate the problem, we split the original table into two tables:

1. **Enrollment** - keeps only the attributes that truly need the whole key.

| StudentID | CourseID | Grade | |-----------|----------|-------|

2. **CourseInfo** - stores data that depends on just one part of the key.

| CourseID | Instructor | |----------|------------|

Now the **Enrollment** table's primary key is still *(StudentID, CourseID)*, but every non■key column (here only *Grade*) depends on the full key. The **CourseInfo** table has *CourseID* as its primary key, and *Instructor* depends on that single column.

Mathematically, after the split we have:

$$ \text{Enrollment}(StudentID, CourseID, Grade) $$ $$ \text{CourseInfo}(CourseID, Instructor) $$

Both tables satisfy **Second Normal Form (2NF)** because no non■key attribute is partially dependent on a composite key.

> **Note:** The process of moving attributes that depend on only part of a key to a new table is called **decomposition**. It preserves all original information while removing redundancy.

Benefits of achieving 2NF

- **Less duplication:** The instructor's name appears once per course, not once per student■course pair. - **Easier updates:** If Dr. Lee changes to Dr. Patel, you edit a single row in **CourseInfo** instead of many rows in **Enrollment**. - **Improved integrity:** The database can enforce that each course has exactly one instructor, because *CourseID* is the primary key in **CourseInfo**.

> **Common mistake:** Some students think that simply having a primary key is enough. They forget to check whether any non■key column depends only on a subset of a composite key. This oversight leaves hidden redundancy even after the first normal form.

> **Remember:** A table is in **Second Normal Form** when **every** non■key attribute is fully dependent on the **whole** primary key, not just a piece of it. If you find an attribute that looks at only one part of a composite key, create a new table for it and link the tables with the appropriate foreign key. This small step keeps your data tidy and your future queries faster.

Third Normal Form (3NF)

When we talk about **Third Normal Form (3NF)** we are trying to make sure that every piece of information in a table tells us something *directly* about the row's primary key, and nothing else.

First, recall what a *primary key* is: it is a column (or a set of columns) that uniquely identifies each record in a table. All other columns are called *non■key attributes* because they do not identify the row on their own.

> **Definition:** A **transitive dependency** occurs when a non■key attribute depends on another non■key attribute, which in turn depends on the primary key. In symbols, if **PK -> A** and **A -> B**, then **PK -> B** is transitive through **A**.

Why does this matter? Imagine a table that stores information about university courses:

| CourseID (PK) | CourseName | DeptCode | DeptName | |---------------|------------|----------|----------| | CS101 | Intro CS | CS | Computer Science | | MA201 | Calculus II| MA | Mathematics |

Here **CourseID** is the primary key. **DeptCode** tells us the department, and **DeptName** tells us the department's full name. The dependency chain is:

- **CourseID -> DeptCode** (the course tells us its department code) - **DeptCode -> DeptName** (the department code tells us the department name)

So **DeptName** depends on **DeptCode**, not directly on **CourseID**. That is a transitive dependency, and the table is **not** in 3NF.

> **Note:** 3NF requires that every non■key attribute be *directly* dependent on the primary key, and that there be no transitive dependencies.

How to fix it

The usual remedy is to split the original table into two tables, each of which satisfies 3NF.

1. **Course table** - keeps only attributes that depend directly on the course identifier.

| CourseID (PK) | CourseName | DeptCode | |---------------|------------|----------|

2. **Department table** - stores information about each department, using the department code as its primary key.

| DeptCode (PK) | DeptName | |---------------|----------|

Now the dependencies are:

- In the **Course** table: **CourseID -> CourseName** and **CourseID -> DeptCode** (both direct). - In the **Department** table: **DeptCode -> DeptName** (direct).

There is no longer a path where a non■key attribute depends on another non■key attribute within the same table, so both tables are in 3NF.

> **In daily life:** Think of a library catalog. A book record might list the *author's name* and the *author's birth year*. The book's ISBN identifies the book (primary key). The birth year does not describe the book itself; it describes the author. To avoid a transitive dependency, you would keep a separate **Author** table (AuthorID, Name, BirthYear) and store only the AuthorID in the **Book** table. This way each table talks only about one kind of thing.

Steps to achieve 3NF

1. **Identify the primary key** of the table. 2. **List all functional dependencies** - statements of the form "X determines Y". 3. **Look for transitive dependencies**: find any non■key attribute Y that is determined by another non■key attribute Z, where the primary key determines Z. 4. **Create a new table** for the group of attributes that are together dependent on Z. Make Z the primary key of that new table. 5. **Replace Z in the original table** with a reference (foreign key) to the new table. 6. **Check each table again** to ensure no new transitive dependencies were introduced.

> **Common mistake:** Removing only the obvious repeating groups and stopping at Second Normal Form (2NF). Students often think 2NF is enough, but they forget to examine whether non■key attributes depend on each other. This oversight leaves hidden redundancy and update anomalies.

Why 3NF is useful

When a database is in 3NF:

- **Update anomalies disappear.** Changing a department's name requires updating only one row in the Department table, not every course row that mentions that department. - **Insertion anomalies are reduced.** You can add a new department without having to create a dummy course. - **Deletion anomalies are avoided.** Removing a course will not accidentally erase the department's name.

> **Remember:** 3NF = *no transitive dependencies* + *every non■key attribute depends only on the primary key*. By splitting tables so that each fact lives in the right place, we keep the data clean, consistent, and easy to maintain.

Putting It All Together - A Full Normalisation Example

Imagine you have a single spreadsheet that a small shop uses to record every sale. At first it looks convenient - one big table that holds everything. But as the shop grows, the table becomes messy: the same information is repeated, some columns depend on only part of the data, and changing a single price means editing many rows. Normalisation is the process of reorganising that big table into smaller, well■structured tables so that each piece of information lives in exactly one place.

Below is a step■by■step walk■through of normalising the same sales data from an **unnormalised** form all the way to **Third Normal Form (3NF)**.

---

1. The unnormalised table (UNF)

| SaleID | Date | CustomerName | CustomerPhone | Product1 | Qty1 | Price1 | Product2 | Qty2 | Price2 | Salesperson |

|--------|------------|--------------|---------------|----------|------|--------|----------|------|--------|-------------| | 1 | 2024■03■01 | Alice Lee | 555■0123 | Pen | 10 | 1.20 | Notebook | 5 | 2.50 | John | | 2 | 2024■03■02 | Bob Kim | 555■0456 | Pencil | 20 | 0.80 | - | - | - | Mary | | 3 | 2024■03■02 | Alice Lee | 555■0123 | Pen | 5 | 1.20 | Eraser | 2 | 0.50 | John |

Why it is a problem: - **Repeating groups** - the columns `Product1...Price2` repeat for each product in a sale. - **Partial dependencies** - `CustomerPhone` depends only on `CustomerName`, not on the whole primary key (`SaleID`). - **Transitive dependencies** - `Price1` depends on `Product1`, which in turn depends on `SaleID`.

---

2. Apply First Normal Form (1NF)

Definition: 1NF requires that each table cell holds a single atomic value and that there are no repeating groups.

We create a new row for every product sold, keeping the sale information duplicated only as needed.

| SaleID | Date | CustomerName | CustomerPhone | Product | Qty | Price | Salesperson | |--------|------------|--------------|---------------|----------|-----|-------|-------------| | 1 | 2024■03■01 | Alice Lee | 555■0123 | Pen | 10 | 1.20 | John | | 1 | 2024■03■01 | Alice Lee | 555■0123 | Notebook | 5 | 2.50 | John | | 2 | 2024■03■02 | Bob Kim | 555■0456 | Pencil | 20 | 0.80 | Mary | | 3 | 2024■03■02 | Alice Lee | 555■0123 | Pen | 5 | 1.20 | John | | 3 | 2024■03■02 | Alice Lee | 555■0123 | Eraser | 2 | 0.50 | John |

Now every column contains only one value per cell, and the table has no repeating groups.

---

3. Apply Second Normal Form (2NF)

Definition: 2NF removes *partial dependencies*: a non■key column must depend on the whole primary key, not just part of it. The current primary key is the composite `(SaleID, Product)`.

- `CustomerName` and `CustomerPhone` depend only on `SaleID`. - `Salesperson` also depends only on `SaleID`.

We split the table into two:

Sales table (one row per sale)

| SaleID | Date | CustomerID | Salesperson | |--------|------------|------------|-------------| | 1 | 2024■03■01 | C001 | John | | 2 | 2024■03■02 | C002 | Mary | | 3 | 2024■03■02 | C001 | John |

SaleLines table (one row per product in a sale)

| SaleID | Product | Qty | Price | |--------|----------|-----|-------| | 1 | Pen | 10 | 1.20 | | 1 | Notebook | 5 | 2.50 | | 2 | Pencil | 20 | 0.80 | | 3 | Pen | 5 | 1.20 | | 3 | Eraser | 2 | 0.50 |

Customers table (introduced to hold customer data once)

| CustomerID | CustomerName | CustomerPhone | |------------|--------------|---------------| | C001 | Alice Lee | 555■0123 | | C002 | Bob Kim | 555■0456 |

Now every non■key column in each table depends on the whole primary key of that table.

---

4. Apply Third Normal Form (3NF)

Definition: 3NF removes *transitive dependencies*: a non■key column must not depend on another non■key column.

In **SaleLines**, the column `Price` depends on `Product`, not on the primary key `(SaleID, Product)`. We therefore create a separate **Products** table.

Products table

| ProductID | ProductName | UnitPrice | |-----------|------------|-----------| | P001 | Pen | 1.20 | | P002 | Notebook | 2.50 | | P003 | Pencil | 0.80 | | P004 | Eraser | 0.50 |

Revised SaleLines table (now stores only references)

| SaleID | ProductID | Qty | |--------|-----------|-----| | 1 | P001 | 10 | | 1 | P002 | 5 | | 2 | P003 | 20 | | 3 | P001 | 5 | | 3 | P004 | 2 |

All tables are now in 3NF: every non■key attribute depends directly on the primary key, and there are no indirect (transitive) links.

> **Remember:** After each normalisation step you should check the **primary key** (the unique identifier) and make sure every other column tells you something about the whole key, not just a part of it.

> **In daily life:** Think of a library catalogue. Instead of writing the author's name on every book entry, you keep an **Authors** list once and link each book to the author's ID. This avoids having to correct the author's spelling in dozens of places.

> **Common mistake:** Splitting a table too early and creating many tiny tables that must be joined for every query. Normalisation is a balance - you want clean design, but you also consider performance.

The final set of tables - **Customers**, **Products**, **Sales**, **SaleLines**, and **Salespersons** (if you decide to store salesperson details separately) - are linked by foreign keys (`CustomerID`, `ProductID`, `SaleID`). This structure eliminates redundancy, makes updates safe, and keeps the data easy to understand, just like organising a messy drawer into labelled folders.

Why Normalisation Matters in Practice

When you store data in a database, the way the tables are arranged can make a big difference to how easy the system is to use, how fast it runs, and how safe the information stays. This is why **normalisation** - the process of organising tables so that each piece of data lives in the right place - matters in real■world projects.

> **Note:** *Normalisation reduces redundancy.* > Redundancy means the same fact is stored in many places. If a customer's phone number appears in ten different rows, changing that number later means you have to update all ten rows. Missing even one update leaves the database with conflicting information.

Why fewer copies help

1. **Consistency** - When a fact is stored only once, any change is made in a single spot. The database automatically keeps all related records up■to■date, so you never see two different phone numbers for the same customer.

2. **Storage efficiency** - Every extra copy of a value uses space on the disk. In large systems this adds up to gigabytes or even terabytes of unnecessary data. Normalised tables keep the data compact.

3. **Simpler queries** - Imagine you want to list every order together with the product name and the supplier's city. In a highly redundant table you would have to write a complicated query that picks the right copy of each piece of information. In a well■normalised design the query can follow clear foreign■key links, making the SQL easier to read and faster to execute.

4. **Better integrity** - Databases can enforce rules called *constraints* (for example, a foreign■key constraint that says every order must refer to a real customer). When tables are normalised, those constraints work as intended, preventing accidental entry of impossible data.

When you might keep some redundancy

Sometimes the purest normal form (usually 3NF or BCNF) is not the best choice for performance. This is called **denormalisation** - deliberately adding some redundancy back into the design.

> **In daily life:** Think of a library catalogue printed on paper. The catalogue lists each book with its title, author, and shelf location. If the library decides to add a quick■lookup sheet that repeats the shelf location next to each title, the sheet is *denormalised*: it stores the same shelf information twice, but it makes finding a book faster for the staff.

Denormalisation is useful when:

- **Read■heavy workloads** - If the application mostly shows data (e.g., a web site displaying product listings) and rarely updates it, storing a pre■joined view can avoid costly joins between many tables. - **Reporting** - Data warehouses often keep summary tables that duplicate detailed data so that reports run quickly. - **Hardware limits** - On systems with very slow disk access, fewer joins can mean a noticeable speed gain.

However, each added copy brings the risks mentioned earlier, so denormalisation should be a conscious decision, not a default.

Tips for choosing the right level of normalisation

1. **Start with 3NF** - Build your tables so that every non■key column depends only on the primary key, and no transitive dependencies remain. This gives you a solid, maintainable base.

2. **Measure performance** - Run realistic queries on a test copy of the database. If a particular query takes too long because it must join many tables, consider denormalising just the pieces involved in that query.

3. **Count updates** - Ask how often the data will change. If a field is updated many times per day, keep it in a single place. If it rarely changes (e.g., a country's name), duplicating it for speed is less risky.

4. **Document any denormalisation** - Write down why you added a redundant column, and add triggers or application logic to keep the copies in sync. This prevents the "forgotten copy" problem

later.

> **Remember:** *Normalisation is a tool, not a rule.* > Use it to create clean, reliable tables, then add controlled redundancy only when you have clear evidence that it improves the system's speed or usability. By balancing these steps, you get a database that is both trustworthy and performant.

Figure (described): A spreadsheet row showing multiple course codes in one cell, illustrating a repeating group.

Figure (described): The same data transformed from one unnormalized table into three tables in 1NF, 2NF and 3NF.

Figure (described): A diagram showing a composite primary key and a partial dependency that violates 2NF.

Key terms

Redundancy: Unnecessary repetition of the same data in multiple places.

Normalisation: Systematic process of organising tables to store each fact once.

Atomic value: A single, indivisible piece of data stored in a column.

Repeating group: A column that holds multiple values together, violating atomicity.

Primary key: One or more columns that uniquely identify each row in a table.

Partial dependency: When a non■key column depends on only part of a composite primary key.

Transitive dependency: When a non■key column depends on another non■key column that depends on the primary key.

1NF: First Normal Form; requires atomic columns and unique rows.

5. Introduction to SQL

In this lesson students learn how to create tables and manage data using SQL commands. They will practice writing statements to add, modify, and remove records in a relational database.

What you will be able to do

- Create a new table with appropriate column definitions
- Insert rows of data into a table
- Update existing records based on conditions
- Delete rows safely using criteria
- Explain the purpose of each SQL data■manipulation command

What Is a Table?

A table is the most basic way a database stores information. Think of a spreadsheet you might have used in school: it is made up of **rows** that run horizontally and **columns** that run vertically. Each row represents one complete record - for example, one student - and each column holds a specific piece of information about that record, such as the student's name or birth date. In a database, a table works exactly the same way, only the data lives on a server and can be queried by many users at the same time.

> **Definition:** A table is a collection of rows (records) and columns (fields) that together hold related data in a structured format.

Why use a table? When data is organized into rows and columns, you can quickly find, add, change, or remove a single piece of information without touching the rest. This structure also lets the database enforce rules that keep the data consistent and reliable.

Columns and Data Types

Each column has a **name** and a **data type**. The name tells you what the column stores (e.g., `first_name`). The data type tells the database what kind of values are allowed in that column, such as:

| Data type | What it stores | Example | |-----------|----------------|---------| | `INTEGER` | Whole numbers | `42` | | `REAL` | Numbers with decimals | `3.14` | | `TEXT` | Strings of characters | `'Alice'` | | `DATE` | Calendar dates | `'2023-09-01'` |

Choosing the right data type is important because it prevents mistakes like putting a word where a number should be, and it helps the database run faster.

Primary Keys

Every table should have a **primary key**. This is a column (or a combination of columns) whose values are unique for each row. The primary key lets the database identify each record without ambiguity. For a table of students, a common primary key is `student_id`, because no two students share the same ID number.

> **Note:** A primary key cannot contain `NULL` (unknown) values and must be unique across all rows.

Basic CREATE TABLE Syntax

To create a new table, you use the `CREATE TABLE` statement. The general form looks like this:

$$ \text{CREATE TABLE table_name (column1 datatype [constraints], column2 datatype [constraints], ...);} $$

Let's break it down step■by■step:

1. **`CREATE TABLE`** - tells the database you want to make a new table. 2. **`table_name`** - the name you give the table, such as `students`. Choose a name that describes the data. 3. **`( ... )`** - inside the parentheses you list each column definition, separated by commas. 4. **`column_name datatype`** - defines a column's name and what kind of data it holds. 5. **`[constraints]`** - optional rules for the column, such as `PRIMARY KEY`, `NOT NULL`, or `UNIQUE`.

Here is a concrete example that creates a simple table for storing student information:

```sql CREATE TABLE students (student_id INTEGER PRIMARY KEY, first_name TEXT NOT NULL, last_name TEXT NOT NULL, birth_date DATE, gpa REAL); ```

Explanation of the example:

- `student_id INTEGER PRIMARY KEY` creates a column named `student_id` that stores whole numbers and serves as the unique identifier for each row. - `first_name TEXT NOT NULL` and `last_name TEXT NOT NULL` store the student's names; `NOT NULL` means a name must be provided - the column cannot be left empty. - `birth_date DATE` stores the student's birth date; it can be left empty if the information is unknown. - `gpa REAL` stores the grade point average, which may have a decimal part.

> **In daily life:** Imagine a library catalog. Each book is a row, and columns hold the title, author, ISBN, and whether the book is checked out. The ISBN works like a primary key because every book has a unique ISBN.

> **Common mistake:** Forgetting to declare a primary key. Without it, the database cannot guarantee that each row is distinct, which makes updates and deletions error■prone.

> **Remember:** Choose clear column names, appropriate data types, and always define a primary key. This foundation makes all later SQL commands - `INSERT`, `UPDATE`, `DELETE`, and `SELECT` - work smoothly and safely.

Creating a Table with CREATE TABLE

When you start a new database you first need a place to store information. In SQL that place is called a **table** - think of it like a spreadsheet where each row is one record (for example, one student) and each column is a piece of data about that record (for example, the student's name or birth date).

To tell the database **what the table looks like**, you write a **CREATE TABLE** statement. This statement is the blueprint: it lists every column, decides what kind of data each column can hold, and marks which column uniquely identifies each row (the **primary key**).

Step 1 - Choose a table name
```

Pick a short, meaningful name that tells you what the table stores. If we are keeping information about students, a good name is `students`. SQL identifiers are not case■sensitive, but writing them in lower■case makes the code easier to read.

```sql CREATE TABLE students (...); ```

Step 2 - List the columns

Inside the parentheses you write one line for each column. Each line has three parts:

1. **Column name** - a single word, like `id` or `first_name`. 2. **Data type** - tells the database what kind of values are allowed (numbers, text, dates, etc.). 3. Optional **constraints** - rules such as "this column cannot be empty" or "this column must be unique".

Common data types for beginners

| Data type | What it stores | Example | |-----------|----------------|---------| | `INTEGER` | Whole numbers | `23` | | `REAL` | Numbers with decimals | `3.14` | | `TEXT` | Strings of characters | `'Alice'` | | `DATE` | Calendar dates | `'2005-09-01'` |

> **Definition:** *Data type* - a label that tells the database what kind of value a column can contain, ensuring the data is stored consistently.

Step 3 - Decide the primary key

Every row should be distinguishable from all other rows. The simplest way is to create an `id` column that contains a unique number for each student. Mark this column as the **primary key** with the keyword `PRIMARY KEY`. The primary key automatically gets two important properties:

* No two rows can have the same value. * The column cannot be `NULL` (empty).

Step 4 - Put it all together

Here is a complete, step■by■step `CREATE TABLE` statement for our `students` table:

```sql CREATE TABLE students (id INTEGER PRIMARY KEY, first_name TEXT NOT NULL, last_name TEXT NOT NULL, birth_date DATE, gpa REAL); ```

Explanation of each line:

* `id INTEGER PRIMARY KEY` - a unique whole■number identifier for each student. * `first_name TEXT NOT NULL` - the student's first name; `TEXT` means any string, `NOT NULL` means you must provide a value. * `last_name TEXT NOT NULL` - same idea for the last name. * `birth_date DATE` - optional column for the student's birth date; it can be left empty. * `gpa REAL` - optional column for the grade point average, which may have a decimal point.

> **Note:** The order of columns does not affect how the table works, but placing the primary key first is a common convention that makes the definition easier to read.

Step 5 - Run the statement

Copy the whole block into your SQL client (such as SQLite, MySQL Workbench, or psql) and execute it. If everything is correct, the database will respond with a message like "Table created". Now you have a ready■made container for student records.

> **Common mistake:** Forgetting the comma after a column definition (e.g., writing `last_name TEXT NOT NULL birth_date DATE`). The database will report a syntax error; always end each

line with a comma except the last one.

Step 6 - Verify the table

You can check that the table exists and see its structure with a command such as:

```sql PRAGMA table_info(students); ```

or, in MySQL:

```sql DESCRIBE students; ```

The output will list each column, its data type, and whether it can be `NULL`.

> **Remember:** A well■designed table starts with clear column names, appropriate data types, and a primary key. This foundation makes later tasks - adding, updating, or deleting data - much smoother.

Inserting Data with INSERT INTO

When you first learn a language, you start by saying a single word, then a short sentence, and finally a whole paragraph. In SQL the same idea applies: the first step is to **insert** one row of data, then you can add many rows at once. Inserting means "putting a new record into a table", just like putting a new card into a deck of playing cards. Each card (row) has several fields (columns) such as *name*, *age*, and *city*.

Definition: *INSERT INTO* is the SQL command that adds new rows to an existing table.

The simplest form tells the database exactly what values belong in each column, in the same order the columns were defined when the table was created. Assume we have a table called `Students` that was created like this:

```sql CREATE TABLE Students (id INT PRIMARY KEY, name VARCHAR(50), age INT, city VARCHAR(30)); ```

Here `id`, `name`, `age`, and `city` are the column names. To add a single student we write:

```sql INSERT INTO Students VALUES (1, 'Alice', 20, 'London'); ```

The word **VALUES** tells SQL that the list inside the parentheses are the actual data for the new row. The first value (`1`) goes into the `id` column, the second (`'Alice'`) into `name`, and so on. If you forget the order or give a value of the wrong type (e.g., a word where a number is expected), the database will raise an error.

> **Note:** The column list is optional **only** when you provide a value for **every** column **in the exact order** they were defined.

Often you do not want to supply a value for every column. Maybe the `id` column is set to auto■increment, meaning the database will automatically generate the next number for you. In that case you list only the columns you are providing:

```sql INSERT INTO Students (name, age, city) VALUES ('Bob', 22, 'Paris'); ```

Because we omitted `id`, the database fills it in automatically. You can also use the keyword **DEFAULT** to tell SQL to use the column's default value (if one was defined) instead of writing a literal value:
```

```sql INSERT INTO Students (id, name, age, city) VALUES (DEFAULT, 'Clara', 19, 'Berlin'); ```

Adding Multiple Rows at Once

Writing a separate `INSERT` statement for each row can be tedious and slow, especially when you have many records. SQL lets you insert several rows in a single command by separating each row's values with a comma:

```sql INSERT INTO Students (name, age, city) VALUES ('David', 21, 'Rome'), ('Eva', 20, 'Madrid'), ('Frank', 23, 'Dublin'); ```

All three rows are added together, and the database treats the operation as one transaction, which is faster and keeps the data consistent.

> **In daily life:** Think of this as buying a pack of postcards. Instead of writing each postcard one by one, you write all the addresses on a single sheet and hand the whole sheet to the post office. The post office processes them together, saving time.

When to Use a Column List vs. No List

- **Use a column list** when you: 1. Want to skip columns that have defaults or auto■generated values. 2. Are not sure of the exact column order (perhaps the table was altered later). 3. Want your code to be clear and resistant to future schema changes.

- **Omit the column list** only when: 1. You are providing a value for **every** column. 2. The order of columns in the table will never change.

> **Common mistake:** Forgetting to include the column list and then adding a new column to the table later. Existing `INSERT` statements will start failing because the order of values no longer matches the table definition.

Quick Checklist

1. Identify the table you want to add data to. 2. Decide whether you need to list the columns. 3. Write the `INSERT INTO` line, optionally followed by a column list in parentheses. 4. Use the `VALUES` keyword and supply one set of parentheses for each row. 5. Separate multiple rows with commas. 6. End the statement with a semicolon (`;`).

> **Remember:** Every value must match the data type of its column (e.g., numbers for `INT`, text in single quotes for `VARCHAR`). If a column allows `NULL` (meaning "no value"), you can write `NULL` instead of a real value.

By mastering these patterns you can quickly populate any table, whether you are entering a single record by hand or loading a batch of data from another source. The same principles apply no matter which SQL database you use - MySQL, PostgreSQL, SQLite, or others. Once the data is in the table, you will be ready to retrieve, update, or delete it in later lessons.

Updating Data with UPDATE

Updating existing rows in a table is done with the **UPDATE** command. Think of a table as a spreadsheet that already has many rows of information. Sometimes the data we entered earlier changes - a student's address moves, a product's price is reduced, or a status flag flips from "pending" to "approved". The UPDATE statement lets us change exactly those cells without having to delete the whole row and re■insert it.

The basic shape of an UPDATE statement

The simplest form looks like this:

$$ \text{UPDATE } \textit{table_name} \\ \text{SET } \textit{column_1} = \textit{new_value_1}, \; \textit{column_2} = \textit{new_value_2}, \dots \\ \text{WHERE } \textit{condition}; $$

* **UPDATE** - tells the database which table we want to change. * **SET** - introduces the list of columns that will receive new values, and the values themselves. * **WHERE** - a filter that decides which rows are affected. Only rows that satisfy the condition are updated.

If we omit the **WHERE** clause, *every* row in the table is changed. That is why the **WHERE** part is so important; it keeps our updates focused.

A concrete example

Suppose we have a table called `Students` with columns `id`, `name`, `grade`, and `city`. A student named Alice just moved from "Springfield" to "Rivertown". We can record the move with:

```sql UPDATE Students SET city = 'Rivertown' WHERE name = 'Alice' AND city = 'Springfield'; ```

The **WHERE** clause checks two things: the row's `name` must be "Alice" **and** the current `city` must be "Springfield". This double check prevents us from accidentally changing a different Alice who already lives in Rivertown.

Updating several columns at once

You can change more than one column in a single statement. Imagine the school decided to give every student who scored below 60 a "remedial" status:

```sql UPDATE Students SET grade = grade + 5, status = 'remedial' WHERE grade < 60; ```

Here the new value for `grade` is expressed as an *expression*: `grade + 5`. The database takes the existing grade, adds 5, and stores the result back into the same column.

Why limiting updates matters

> **Note:** Updating without a proper **WHERE** clause can unintentionally modify every row, leading to data loss or corruption.

Think of it like editing a word document: if you use "Replace All" without checking the search term, you might replace words you never intended to change. In a database, the consequences are often larger because many applications rely on that data.

Tips for safe updates

1. **Test with SELECT first** - Write the same **WHERE** condition in a `SELECT` query to see which rows will be affected. 2. **Back up before big changes** - Especially on production systems, make a copy of the table or the whole database. 3. **Use transactions** - Wrap the UPDATE in `BEGIN TRANSACTION` ... `COMMIT`. If something goes wrong, you can `ROLLBACK` to the original state.

Example with a transaction

```sql BEGIN TRANSACTION;

UPDATE Employees SET salary = salary * 1.10 WHERE department = 'Sales';
```

COMMIT; ```

If the `WHERE` clause accidentally matches the wrong department, you could instead issue `ROLLBACK;` before committing, and the salaries would stay unchanged.

> **Remember:** Always double■check the **WHERE** clause and, when possible, preview the rows with a `SELECT` before running the UPDATE. This habit protects your data and saves you from costly mistakes.

Deleting Data with DELETE

When you first learn SQL, you might think that once data is stored in a table it stays there forever. In reality, databases are meant to change over time - new information arrives, old information becomes irrelevant, and sometimes mistakes are made. The **DELETE** statement is the tool that lets you remove rows (records) that you no longer need.

> **Definition:** `DELETE` is an SQL command that permanently removes one or more rows from a table.

The basic idea

Imagine a notebook where each page is a row of a table, and the whole notebook is the table itself. If you write a wrong entry on a page, you can either cross it out (which leaves the mistake visible) or you can tear the page out completely. `DELETE` is like tearing the page out - the data disappears from the table and is not shown in any future queries.

Simple syntax

The most basic form of the command looks like this:

$$ \text{DELETE FROM table_name;} $$

If you run this exact statement, **every** row in `table_name` is removed. This is rarely what you want, because it erases all the data in one step.

Using `WHERE` to target specific rows

To delete only the rows that meet a certain condition, you add a **WHERE** clause. `WHERE` works like a filter: it tells the database *"only delete rows where this condition is true."*

Example: suppose we have a table called `Students` with columns `id`, `name`, and `grade`. To remove the record of a student named Alice, you would write:

```sql DELETE FROM Students WHERE name = 'Alice'; ```

Here's what each part means:

- **DELETE FROM** - tells the database you want to delete rows. - **Students** - the name of the table. - **WHERE** - introduces the condition. - **name = 'Alice'** - the condition; only rows whose `name` column equals the text *Alice* will be deleted.

You can combine multiple conditions using `AND` or `OR`:

```sql DELETE FROM Students WHERE grade < 60 AND name = 'Bob'; ```

This removes Bob's record only if his grade is below 60.

> **In daily life:** Think of `WHERE` like a grocery store's "sale aisle" sign. Only the items in that aisle get the discount; the rest stay at full price. Similarly, only rows that match the `WHERE` condition get deleted.

Safety tips - avoid accidental data loss

Because `DELETE` permanently removes data, it's easy to make a mistake that wipes out more than you intended. Follow these habits:

1. **Always preview with SELECT first.** Before running a `DELETE`, run the same `WHERE` clause with a `SELECT` to see which rows will be affected.

```sql SELECT * FROM Students WHERE grade < 60; ```

If the result set looks correct, you can proceed to delete.

2. **Use transactions when possible.** A transaction groups several statements together; you can *commit* (make permanent) or *rollback* (undo) the whole group.

```sql BEGIN TRANSACTION; DELETE FROM Students WHERE grade < 60; -- If you realize something is wrong: ROLLBACK; -- Otherwise: COMMIT; ```

3. **Limit with `TOP` or `LIMIT` (if supported).** Some databases let you delete only a certain number of rows at a time, which reduces risk.

```sql DELETE FROM Logs WHERE created_at < '2023-01-01' LIMIT 1000; ```

4. **Never omit the `WHERE` clause unless you truly want to clear the whole table.** A missing `WHERE` is the most common cause of accidental data loss.

> **Common mistake:** Running `DELETE FROM Orders;` when you meant `DELETE FROM Orders WHERE status = 'canceled';` deletes every order, not just the canceled ones.

What happens after a delete?

When a row is deleted, the space it occupied becomes available for new rows. Some database engines keep a *log* of the deletion so they can roll back the change if you use a transaction. However, once you `COMMIT` the transaction, the data is gone from the logical view of the table. It may still exist on disk for a short time, but you should treat it as permanently lost.

> **Remember:** Deleting is irreversible in normal use. Always double■check your `WHERE` clause and consider backing up the table before large deletions.

Quick checklist before you hit "run"

- [ ] Did you run a matching `SELECT` to see the rows? - [ ] Is the `WHERE` clause precise enough? - [ ] Are you inside a transaction you can roll back if needed? - [ ] Have you considered using `LIMIT` to delete in batches?

By treating `DELETE` with the same caution you would use when erasing a chalkboard, you keep your database clean without losing valuable information by accident.

Putting It All Together: A Mini Project

Imagine you are building a small digital notebook where each page is a **record** that stores information about a book you own. First we will create a table that represents the notebook, then
```

we will add some pages, change a few details, and finally remove the pages we no longer need. Doing all these steps in one place helps you see how the SQL commands fit together, just like following a recipe from start to finish.

1. Create the table

A **table** is like a spreadsheet: it has columns (the headings) and rows (the data). We will call our table **books** and give it four columns:

- `id` - a unique number for each book (like a barcode) - `title` - the name of the book - `author` - who wrote it - `year` - the year it was published

```sql CREATE TABLE books (id INTEGER PRIMARY KEY, title TEXT NOT NULL, author TEXT NOT NULL, year INTEGER); ```

> **Definition:** `PRIMARY KEY` means the column must contain a unique value for every row, so we can always find a specific book.

2. Insert some sample data

Now we add three books to our notebook. The **INSERT INTO** command puts a new row into the table.

```sql INSERT INTO books (id, title, author, year) VALUES (1, 'The Little Prince', 'Antoine de Saint-Exupéry', 1943), (2, '1984', 'George Orwell', 1949), (3, 'Pride and Prejudice', 'Jane Austen', 1813); ```

> **Note:** The list of column names `(id, title, author, year)` must match the order of the values inside each set of parentheses.

3. Update a record

Suppose you discover that the publication year for *1984* was entered incorrectly. The **UPDATE** command changes existing rows. We tell SQL *which rows* to modify with a **WHERE** clause.

```sql UPDATE books SET year = 1948 WHERE title = '1984'; ```

> **In daily life:** Think of `WHERE` as a filter that selects only the pages that match a condition, just like you would pick out all the receipts from a certain month.

4. Add another book

Maybe you bought a new novel and want to record it.

```sql INSERT INTO books (id, title, author, year) VALUES (4, 'The Hobbit', 'J.R.R. Tolkien', 1937); ```

5. Delete a record

You decide to give away *Pride and Prejudice*. The **DELETE** command removes rows that meet a condition.

```sql DELETE FROM books WHERE id = 3; ```

> **Common mistake:** Forgetting the `WHERE` clause deletes **all** rows in the table. Always double■check that the condition correctly identifies only the rows you want to remove.
```

6. Verify the final state

Run a simple **SELECT** query to see what remains:

```sql SELECT * FROM books; ```

You should see three rows: *The Little Prince*, *1984* (with the corrected year), and *The Hobbit*.

7. Wrap■up checklist

1. **Create** the table with the right column types. 2. **Insert** data using one `INSERT INTO` statement or multiple rows at once. 3. **Update** only the rows you intend to change, using a precise `WHERE`. 4. **Delete** rows carefully; always include a `WHERE`. 5. **Select** to confirm your changes.

> **Remember:** SQL works step by step, just like writing on a piece of paper: you first draw the table, then fill it, then edit, then erase. Practicing this mini project gives you a complete picture of the typical workflow you will use in real databases.

Common Errors and Best Practices

When you write SQL, the computer will tell you if something is wrong, but the messages can look like a foreign language at first. Learning how to read those messages and avoiding the most common slip■ups will save you a lot of time and frustration.

Common mistake: > **Common mistake:** Forgetting a comma between column definitions, mismatching parentheses, or using the wrong quotation marks.

SQL has a very strict grammar. Think of it like a recipe: every ingredient (column name, data type, value) must be listed in the right order, and the punctuation (commas, parentheses, quotes) tells the database where one step ends and the next begins. If you leave out a comma after a column name, the database thinks the next word is still part of the same column definition, and it will raise a syntax error.

How to read an error message When the database finds a problem, it returns a message that usually contains three parts:

1. **Error code** - a short number or word that identifies the type of error (e.g., `SQLSTATE 42601`). 2. **Location** - the line number and sometimes the column number where the parser stopped. 3. **Description** - a brief English sentence such as "syntax error near 'VARCHAR'".

Start by looking at the line number. Open your script and go to that line; the error is often right before or after the highlighted spot. If the description mentions a specific token (like `')'` or `'INSERT'`), check the surrounding punctuation. Many editors can highlight matching parentheses, which helps you see if one is missing.

Typical syntax pitfalls

| Situation | What often goes wrong | How to fix it | |---|---|---| | `CREATE TABLE` column list | Missing comma between columns, e.g. `id INT name VARCHAR(50)` | Add a comma: `id INT, name VARCHAR(50)` | | Data type spelling | Writing `VARCHR` instead of `VARCHAR` | Use the exact keyword from the documentation | | Quoting values | Using single quotes for identifiers (`'order'`) instead of double quotes or backticks |
```

Identifier: `'"order"'` (or `` `order` `` in MySQL); Value: `'2023-01-01'` | | Parentheses balance | Forgetting the closing `)` after a column list | Count opening and closing parentheses; most editors show matching pairs | | `INSERT INTO` column/value mismatch | Supplying 3 values for a table that has 4 columns without specifying the column list | Either list the columns you are providing or give a value for every column (or use `DEFAULT`) |

Best practices for writing robust SQL

> **Remember:** Choose clear, consistent names for tables and columns. > Use lowercase letters and underscores (snake_case) like `customer_orders` instead of `CustomerOrders` or `custOrders`. This avoids case■sensitivity surprises on databases that treat identifiers differently.

> **Note:** Add comments to explain why a table exists or why a column has a special constraint. > In SQL, a comment starts with `--` for a single line or `/* ... */` for a block. Example:

```sql -- Stores each order placed by a customer CREATE TABLE customer_orders (order_id INT PRIMARY KEY, customer_id INT NOT NULL, order_date DATE NOT NULL); ```

Comments do not affect execution, but they make the script readable for you and for anyone else who later maintains the database.

> **In daily life:** Test each statement before adding it to a larger script. > Just as you might taste a sauce before adding the next ingredient, run `CREATE TABLE` by itself, then run a few `INSERT` statements, and finally try a `SELECT` to verify the data looks correct. If an error appears, you can isolate the problem to the most recent command.

Additional tips:

- **Use explicit column lists** in `INSERT` statements. Even if the table's column order seems obvious, writing `INSERT INTO customers (id, name, email) VALUES (1, 'Alice', 'a@example.com');` protects you from future schema changes. - **Avoid reserved words** as identifiers. Words like `order`, `group`, or `select` have special meanings in SQL. If you must use them, enclose them in double quotes or backticks, but it is better to rename the column (e.g., `order_number`). - **Keep statements short and readable**. Break long `SELECT` queries onto multiple lines, aligning keywords (`SELECT`, `FROM`, `WHERE`) at the start of each line. This makes it easier to spot missing commas or misplaced clauses. - **Check data types** before inserting. Trying to put a string into an `INT` column will raise a conversion error. Cast values explicitly if needed, using `CAST(value AS INT)`.

By paying attention to the exact wording of error messages, following a consistent naming style, commenting your code, and testing step by step, you will write SQL that is both correct and easy to maintain. The more you practice spotting these small details, the faster you will become at turning error messages into quick fixes.

Figure (described): A spreadsheet■style view showing rows as records and columns as fields.

Figure (described): Visual representation of a CREATE TABLE statement as a blueprint for a new table.

Figure (described): Sequence of Create, Read, Update, Delete operations on a table.

Key terms

Table: A collection of rows (records) and columns (fields) that stores related data in a database.

Row: A single record in a table, analogous to a line in a spreadsheet.
```

Column: A field that holds a specific type of data for every row, similar to a spreadsheet heading.

Primary Key: A column (or set of columns) that uniquely identifies each row in a table.

CREATE TABLE: SQL statement that defines a new table's name, columns, data types, and primary key.

INSERT INTO: SQL command that adds new rows of data to an existing table.

UPDATE: SQL command that changes the values of existing rows based on a condition.

DELETE: SQL command that permanently removes rows from a table based on a condition.

6. Querying Data

In this lesson students learn how to retrieve and sort data from a database using SELECT, WHERE, and ORDER BY clauses. They practice writing basic queries and understand how filtering and ordering shape the results they see.

What you will be able to do

- Write a SELECT statement to list specific columns from a table
- Add a WHERE clause to filter rows based on conditions
- Use logical operators (AND, OR, NOT) in a WHERE clause
- Apply ORDER BY to sort query results in ascending or descending order
- Combine SELECT, WHERE, and ORDER BY in a single query

SELECT - Choosing Columns

When you ask a database for information, the first thing you decide is **what** you want to see. The `SELECT` clause is the part of a SQL query that tells the database exactly which columns (the vertical slices of a table) to return.

> **Definition:** `SELECT` lists the column names you want to retrieve from a table.

Imagine a spreadsheet that records every book in a school library. Each row is a different book, and the columns might be `Title`, `Author`, `Year`, `Genre`, and `ISBN`. If you only need the titles and authors for a display board, you don't have to pull the whole spreadsheet - just those two columns. In SQL you would write:

```sql SELECT Title, Author FROM Books; ```

Why `SELECT` matters - **Speed:** The database reads less data, so the answer comes back faster. - **Clarity:** You see only the information you need, which makes the result easier to understand. - **Security:** By limiting columns, you avoid exposing sensitive data (like a student's social■security number) to users who don't need it.

Breaking down the syntax 1. **`SELECT` keyword** - tells the database "I am choosing columns." 2. **Column list** - one or more column names separated by commas. - If you write `*` (asterisk) instead of names, you ask for *all* columns: `SELECT * FROM Books;`. 3. **`FROM` keyword** - tells the database which table to look in. - A table is like a single sheet in a spreadsheet; it holds rows (records) and columns (fields).

Putting these pieces together gives the basic pattern:

$$ \text{SELECT column1, column2, ... FROM table\_name;} $$

Step■by■step example Suppose we have a table called `Students` with columns `StudentID`, `FirstName`, `LastName`, `Grade`, and `Email`.

StudentID	FirstName	LastName	Grade	Email
101	Maya	Lee	10	maya.lee@example.com
102	Omar	Patel	11	omar.patel@example.com
```

If we only need each student's first and last name, we write:

```sql SELECT FirstName, LastName FROM Students; ```

The result will be:

| FirstName | LastName | |-----------|----------| | Maya | Lee | | Omar | Patel |

Notice that the other columns (`StudentID`, `Grade`, `Email`) are not shown, even though they still exist in the table.

Using expressions in `SELECT` You can also ask the database to calculate something on the fly. For example, to show a student's full name in one column, you can concatenate the first and last names:

```sql SELECT FirstName || ' ' || LastName AS FullName FROM Students; ```

Here `||` is the string■concatenation operator (different databases use `+` or `CONCAT`). The keyword `AS` gives the new column a friendly name (`FullName`).

> **Note:** The column list can contain plain column names, expressions, or functions, but each item must be separated by a comma.

Common pitfalls - **Misspelling a column name** - the database will return an error because it cannot find that column. - **Using `SELECT *` when you only need a few columns** - this can waste time and expose data unintentionally.

> **Common mistake:** Forgetting the comma between column names, e.g., `SELECT FirstName LastName FROM Students;` will cause a syntax error.

Quick checklist before running a `SELECT` 1. List every column you really need. 2. Separate column names with commas. 3. Verify each column name matches the table definition exactly (including case if the DB is case■sensitive). 4. Decide whether you need all columns (`*`) or just a subset.

> **Remember:** `SELECT` is your first filter; it decides the shape of the data you receive. Getting it right makes the rest of your query easier to read and faster to execute.

WHERE - Filtering Rows

When you write a `SELECT` statement you are asking the database to give you some rows from a table. Sometimes you only want a *part* of the table - for example, the customers who live in **London**, or the products that cost more than $20. That is what the **WHERE** clause does: it tells the database *which rows* are allowed to appear in the result.

> **Definition:** The `WHERE` clause is a part of a `SELECT` statement that filters rows by applying one or more conditions. Only rows that satisfy all the conditions are returned.

Think of a spreadsheet that contains every student in a school. If you want to see only the students who scored above 80 in math, you would look at the column "MathScore" and pick out the rows where the number is larger than 80. The `WHERE` clause works exactly the same way, but the work is done by the database engine instead of by your eyes.

Comparison operators

To compare the value in a column with a constant (or with another column) you use *comparison operators*. The most common ones are:

| Operator | Meaning | |----------|---------| | `=` | equal to | | `<>` or `!=` | not equal to | | `<` | less than | | `>` | greater than | | `<=` | less than or equal to | | `>=` | greater than or equal to |

If you have a table called `Employees` with a column `salary`, the condition

```sql salary > 50000 ```

means "keep only the rows where the employee's salary is greater than 50 000". You can also compare text values:

```sql department = 'HR' ```

means "keep only the rows where the department column exactly matches the word **HR**".

String matching

Text data often needs more flexible matching. SQL provides two useful operators for this:

* `LIKE` - matches a pattern using the wildcard characters `%` (any sequence of characters) and `_` (any single character). * `NOT LIKE` - the opposite; rows that do **not** match the pattern are kept.

Example with `LIKE`:

```sql WHERE name LIKE 'A%' ```

The pattern `'A%'` reads "any string that starts with the letter **A**". The `%` can stand for "nothing", "B", "Bob", "Alison", etc., as long as the first character is **A**.

If you want names that contain the letters **an** anywhere, you would write:

```sql WHERE name LIKE '%an%' ```

The `%` on both sides says "any characters before, any characters after, but **an** must appear somewhere inside".

> **Note:** `LIKE` is case■insensitive in most databases, but some systems (e.g., PostgreSQL) treat it as case■sensitive unless you use `ILIKE` or apply a function like `LOWER()`.

Combining conditions

Often you need more than one test. You can join conditions with the logical operators `AND` and `OR`.

* `AND` means *both* conditions must be true. * `OR` means *at least one* condition must be true.

Example: find products that are either cheap **or** belong to the "Books" category, but only if they are in stock.

```sql WHERE (price < 10 OR category = 'Books') AND in_stock = true ```

The parentheses make the order of evaluation clear: first evaluate the `OR`, then apply the `AND`.
```

> **In daily life:** Imagine a library catalog. You might ask the librarian, "Give me all books that are either written by **Smith** *or* published after 2015, but only if they are currently available." The librarian's answer follows the same logic as the `WHERE` clause above.

Putting it together

A complete `SELECT` with a `WHERE` clause looks like this:

```sql SELECT name, salary FROM Employees WHERE department = 'Engineering' AND salary >= 60000; ```

The database reads the `WHERE` part, checks each row, and returns only those rows that satisfy **both** conditions.

> **Remember:** Every condition you write inside `WHERE` must be a *boolean expression* - something that can be true or false. If the expression is true for a row, that row passes the filter; otherwise it is omitted.

Common mistake

> **Common mistake:** Forgetting to put quotation marks around text literals. > Example of the error: `WHERE city = London` (SQL thinks *London* is a column name). > Correct form: `WHERE city = 'London'`.

By mastering the `WHERE` clause you gain precise control over which rows you see, turning a huge table into exactly the slice of data you need for analysis, reporting, or further processing.

Logical Operators in WHERE

When you write a `WHERE` clause you are telling the database which rows you want to keep and which ones to discard. Sometimes a single condition is enough, for example `price > 20`. Often you need to look at **more than one** condition at the same time. That is where *logical operators* come in.

The three basic logical operators

| Operator | Meaning | Example in a sentence | |----------|---------|-----------------------| | `AND` | both conditions must be true | "Show students who are **both** in grade 10 **and** have a GPA higher than 3.5." | | `OR` | at least one condition must be true | "Show books that are **either** published after 2015 **or** have more than 500 pages." | | `NOT` | reverses the truth value of a condition | "Show employees **not** working in the Sales department." |

> **Definition:** A *logical operator* combines two or more Boolean expressions (expressions that evaluate to **true** or **false**) into a single Boolean result.

How they work together

Think of each condition as a light switch that can be **ON** (true) or **OFF** (false).

- `AND` is like a circuit that only lights a bulb when **all** switches are ON. - `OR` lights the bulb when **any** switch is ON. - `NOT` flips the switch: ON becomes OFF, OFF becomes ON.

In SQL you write them exactly as you would say them in English, but you must respect the order of evaluation. The database evaluates `NOT` first, then `AND`, then `OR`. If you want a different order you use parentheses `( )` to group expressions, just like you would in arithmetic.
```

Simple examples

1. **Two conditions with `AND`**

```sql SELECT name, age FROM students WHERE grade = 10 AND gpa > 3.5; ```

This returns only rows where *both* `grade = 10` **and** `gpa > 3.5` are true.

2. **Two conditions with `OR`**

```sql SELECT title, pages FROM books WHERE year > 2015 OR pages > 500; ```

A row is kept if *either* the book was published after 2015 **or** it has more than 500 pages.

3. **Using `NOT`**

```sql SELECT employee_id, department FROM employees WHERE NOT department = 'Sales'; ```

This selects every employee whose department is **not** Sales.

Combining all three

Suppose you want customers who bought **either** a laptop **or** a tablet, but you **do not** want those whose order total is less than $100.

```sql SELECT customer_id, product, total FROM orders WHERE (product = 'Laptop' OR product = 'Tablet') AND NOT total < 100; ```

The parentheses force the database to evaluate the `OR` part first, then apply the `AND NOT` condition to the result.

Practical tip: think in steps

1. Write each condition on its own line. 2. Decide whether a row must satisfy **all** of them (`AND`) or **any** of them (`OR`). 3. Use `NOT` to exclude what you don't want. 4. Add parentheses whenever the natural order (NOT -> AND -> OR) does not match your intention.

> **In daily life:** Imagine you are planning a movie night. You will watch a film **if** it is a comedy **or** an action movie **and** it is rated PG■13 **and** you do **not** have to finish a homework assignment first. Each part of that sentence is a condition, and the words "or", "and", "not" are exactly the logical operators you use in SQL.

Common pitfalls

> **Common mistake:** Forgetting parentheses when mixing `AND` and `OR`. Without them, the database may evaluate `AND` before `OR`, giving a different result than you expect. Always test a query with a small data set to verify the logic.

Quick reference formula

When you need to express "A and not B, or C", write it as:

$$ (A \; \text{AND} \; \text{NOT} \; B) \; \text{OR} \; C $$

Remember that `NOT` applies only to the condition immediately after it, unless you group it with parentheses.

> **Remember:** Use parentheses to make your intention clear, and read your `WHERE` clause aloud as a sentence. If it sounds confusing, add more parentheses until it reads naturally. This habit prevents logical errors and makes your queries easier for others (and your future self) to understand.

ORDER BY - Sorting Results

When you ask a database for information, the rows it returns come back in the order the system finds them. Often you want the results to follow a pattern that makes sense to you - for example, the newest orders first, or a list of students from the highest grade down to the lowest. That is what the **ORDER BY** clause does: it tells the database how to arrange (sort) the rows before they are sent back to you.

> **Definition:** `ORDER BY` is a keyword that you place at the end of a `SELECT` statement to specify one or more columns that determine the sequence of the output rows.

How sorting works

Think of a stack of index cards. Each card has a name and a score. If you spread the cards on a table and ask a friend to arrange them from the highest score to the lowest, they will look at the *score* column on each card and reorder the cards accordingly. In SQL, the *score* column is the one you list after `ORDER BY`.

The database can sort in two directions:

* **Ascending** - from the smallest value to the largest. This is the default order, and you can write it explicitly with the keyword `ASC`. * **Descending** - from the largest value to the smallest, written with the keyword `DESC`.

If you do not write `ASC` or `DESC`, the database assumes **ascending**.

Simple example

Suppose we have a table called `Products`:

| product_id | name | price | |-----------|---------------|-------| | 1 | Pencil | 0.50 | | 2 | Notebook | 2.30 | | 3 | Backpack | 25.00 | | 4 | Eraser | 0.20 |

To see the products from the cheapest to the most expensive, we write:

```sql SELECT name, price FROM Products ORDER BY price ASC; ```

The result will be:

| name | price | |--------|-------| | Eraser | 0.20 | | Pencil | 0.50 | | Notebook | 2.30 | | Backpack | 25.00 |

If we want the opposite - most expensive first - we change `ASC` to `DESC`:

```sql SELECT name, price FROM Products ORDER BY price DESC; ```

Now the rows appear in reverse order.

Sorting by more than one column

Sometimes a single column is not enough to give a clear order. Imagine a table `Students` with columns `grade` (numeric) and `last_name`. You might want all students sorted by grade, and

within each grade alphabetically by last name. You list the columns in the order you want them considered:

```sql SELECT last_name, grade FROM Students ORDER BY grade DESC, last_name ASC; ```

The database first looks at `grade`. All rows with the same grade are then ordered by `last_name`. The comma separates the sorting criteria.

How the database does the work

Behind the scenes, the database builds a temporary list of the rows that match your `SELECT` and `WHERE` clauses. It then applies a sorting algorithm (often a quicksort or mergesort) based on the values in the columns you specified. The algorithm compares two rows at a time, decides which should come first according to the chosen direction, and rearranges them until the whole list is ordered.

> **Note:** Sorting can be slower on very large tables because the database must look at every row that will appear in the result. Adding an index on the column(s) you sort by can make the operation much faster.

Practical tips

1. **Be explicit** - Even though `ASC` is the default, writing it makes your intention clear to anyone reading the query later. 2. **Use column aliases** - If you rename a column in the `SELECT` list, you can still sort by the original column name or by its position number (e.g., `ORDER BY 2 DESC` sorts by the second selected column). 3. **Watch for NULL values** - By default, many databases treat `NULL` as the lowest possible value in ascending order and the highest in descending order. If you need a different behavior, you can add `NULLS FIRST` or `NULLS LAST` after `ASC`/`DESC` (supported in most modern systems).

> **Remember:** `ORDER BY` only changes the *presentation* of the data; it does not alter the data stored in the table.

Putting it together

A full query that selects specific columns, filters rows, and then sorts them might look like this:

$$ \text{SELECT } name, price \\ \text{FROM } Products \\ \text{WHERE } price > 1.00 \\ \text{ORDER BY } price DESC, name ASC; $$

Read it from top to bottom:

1. Choose the columns `name` and `price`. 2. Keep only rows where `price` is greater than 1.00. 3. Show the remaining rows starting with the highest price, and if two products share the same price, list them alphabetically by `name`.

With `ORDER BY`, you gain control over how the answer is displayed, making it easier to spot trends, find the top■scoring items, or simply present the data in a way that matches the story you want to tell.

Putting It All Together

When you write a query that does everything at once, you are telling the database three things in a single sentence:
```

1. **Which columns** you want to see (the *SELECT* part). 2. **Which rows** are interesting (the *WHERE* part). 3. **How the rows should be ordered** when they are returned (the *ORDER BY* part).

Think of it like planning a road trip. First you decide **what sights** you want to visit (columns), then you choose **which roads** you will actually drive on (filtering rows), and finally you decide **the order** in which you will see the sights (sorting). By writing all three decisions in one instruction, the database can do the work efficiently, just as a GPS can give you the best route when it knows all your preferences at once.

Step■by■step construction

1. **Start with SELECT** - list the columns you need, separated by commas. Example: `SELECT name, age, city`

2. **Add FROM** - tell the database which table holds those columns. Example: `FROM students`

3. **Add WHERE** - write conditions that rows must satisfy. Use comparison operators (`=`, `>`, `<`, `>=`, `<=`) and logical operators (`AND`, `OR`, `NOT`). Example: `WHERE age >= 18 AND city = 'London'`

4. **Finish with ORDER BY** - choose one or more columns to sort by, and specify `ASC` (ascending) or `DESC` (descending). Example: `ORDER BY age DESC, name ASC`

Putting the pieces together gives a complete query:

```sql SELECT name, age, city FROM students WHERE age >= 18 AND city = 'London' ORDER BY age DESC, name ASC; ```

> **Definition:** *SELECT* chooses columns, *WHERE* filters rows, and *ORDER BY* sorts the result set.

Why the order of clauses matters

SQL has a fixed syntax order: `SELECT -> FROM -> WHERE -> ORDER BY`. If you swap them, the database will not understand the statement. It is like writing a recipe with the steps out of order - the cake will not bake correctly.

The logical processing order is actually different from the written order, but you do not need to worry about that when you write simple queries. The database internally does:

1. **FROM** - locate the table. 2. **WHERE** - discard rows that do not meet the condition. 3. **SELECT** - pick the requested columns from the remaining rows. 4. **ORDER BY** - arrange the final rows.

You can remember this with the phrase **"Find, Filter, Pick, Arrange."**

> **Remember:** The written order (`SELECT ... FROM ... WHERE ... ORDER BY ...`) is fixed; the internal order is different but produces the same result.

A complete example with real data
```

Suppose we have a table **orders** with columns `order_id`, `customer_name`, `total_amount`, `order_date`. We want to see the three most recent orders over $100, showing only the order ID and amount, sorted from highest amount to lowest.

```sql SELECT order_id, total_amount FROM orders WHERE total_amount > 100 ORDER BY order_date DESC, total_amount DESC LIMIT 3; ```

Explanation of each part:

- `SELECT order_id, total_amount` - we only need these two columns. - `FROM orders` - the data lives in the *orders* table. - `WHERE total_amount > 100` - keep rows where the amount exceeds $100. - `ORDER BY order_date DESC, total_amount DESC` - first sort by newest date, then by biggest amount. - `LIMIT 3` - return only the top three rows (this clause is optional but often useful).

> **In daily life:** Think of a library catalog. You ask for *titles* (SELECT), only books published after 2015 (WHERE), and you want them listed from newest to oldest (ORDER BY). The librarian gives you a single list that satisfies all three requests.

Common pitfalls to avoid

- **Missing commas** between column names in SELECT. - **Using the wrong comparison operator** (`=` vs. `==`). SQL uses a single `=` for equality. - **Forgetting ASC/DESC** - the default is ascending, but you may need descending. - **Putting ORDER BY before WHERE** - the query will raise a syntax error.

> **Common mistake:** Writing `WHERE age > 18 AND city = London` without quotes around *London*. Text values must be enclosed in single quotes (`'London'`).

Practice exercise

Write a query that shows the `product_name` and `price` of all products in the **products** table that are cheaper than $20, belong to the category `'Books'`, and should be listed alphabetically by product name.

```sql SELECT product_name, price FROM products WHERE price < 20 AND category = 'Books' ORDER BY product_name ASC; ```

Try changing the conditions and the sort order to see how the result set changes. The more you practice combining SELECT, WHERE, and ORDER BY, the more comfortable you will become writing powerful, single■statement queries.

Figure (described): A spreadsheet view of a school library catalog showing columns like Title, Author, Year, Genre, ISBN.

Figure (described): Illustration of rows being kept or removed based on a condition such as "City = London".

Figure (described): A set of student records ordered from highest to lowest grade after applying ORDER BY.

Key terms

SELECT: Specifies the columns to retrieve from a table.

WHERE: Filters rows by applying one or more conditions.

ORDER BY: Sorts the result set according to one or more columns.

AND: Logical operator that requires all combined conditions to be true.

OR: Logical operator that requires at least one of the combined conditions to be true.

NOT: Logical operator that negates a condition.

Column: A vertical slice of a table representing a specific attribute.

Row: A horizontal slice of a table representing a single record.

7. Joining Tables

In this lesson students learn how to combine data from multiple tables using inner and outer joins, and understand how to retrieve related information efficiently. They will practice writing join queries and see how joins are used in real■world databases.

What you will be able to do

- Write an inner join query to combine rows that match in two tables
- Write a left outer join query to include all rows from one table and matching rows from another
- Explain the difference between inner, left, right, and full outer joins
- Choose the appropriate join type for a given data■retrieval problem
- Combine data from three or more tables using multiple joins

What Is a Join?

When we talk about a **join** we are talking about a way to put together information that lives in two different tables. Imagine you have one table that stores every student's ID and name, and another table that stores every student's ID and the club they belong to. Each table on its own tells only part of the story. By joining them we can see, for each student, both the name **and** the club in a single view.

> **Definition:** A join is an operation that combines rows from two tables by matching values in one or more columns that are related to each other.

Why do we need this? In a relational database, data is deliberately split into separate tables to avoid duplication and to keep the data tidy. However, when we want to answer a question that involves more than one piece of information, we must bring the relevant rows together. The join does exactly that: it looks for rows where the values in the chosen columns are the same, and then stitches those rows together into a new result set.

Step■by■step picture

1. **Identify the common column** - This is the column that appears in both tables and holds matching values. In our example the common column is `student_id`. 2. **Match rows** - For each row in the first table, the database searches the second table for rows whose `student_id` is identical. 3. **Combine columns** - When a match is found, the columns from both rows are placed side by side, forming a single combined row. 4. **Collect all matches** - The process repeats for every row, and the database returns the collection of combined rows.

If we write this idea as a simple mathematical expression, we can think of each table as a set of rows, say A and B, and the join as the set of all pairs (a,b) where the join column has the same value:

$$ A \bowtie_{A.column = B.column} B = \{\, (a,b) \mid a \in A,\; b \in B,\; a.column = b.column \,\} $$

The symbol $\bowtie$ is read "join".

> **In daily life:** Think of a school's yearbook. One page lists students with their photos, another page lists students with their favorite subjects. To create a page that shows each student's photo **and** favorite subject, the editor matches the student names on both pages - just like a join matches IDs in two tables.

What the result looks like

After the join, the result table contains **all columns** from both original tables (unless we explicitly ask for only some). Using our example, the joined table would have columns: `student_id`, `name`, `club`. Each row now tells us everything we need about a particular student.

When a join does not find a match

If a student appears in the first table but not in the second, the basic (inner) join will simply leave that student out of the result. This is because an inner join only keeps rows that have a matching partner in both tables. Later we will see other kinds of joins that handle missing matches differently, but for now remember that a plain join is about **matching**.

> **Remember:** A join works only when there is at least one column that both tables share and that contains comparable values. Without a common column, the database would have no basis for pairing rows.

Why the term "join"

The word "join" comes from the idea of **joining** two pieces of a puzzle together. Each table is a puzzle piece; the matching column is the shape that lets the pieces fit. When the pieces fit, we see a bigger picture.

> **Common mistake:** Trying to join tables on columns that have different data types (e.g., joining an integer `student_id` with a text column that stores the same numbers). The database will either error out or return no matches because the values are not comparable.

In summary, a join is the fundamental tool that lets us combine related data spread across multiple tables, turning separate lists into a richer, unified view. This ability is what makes relational databases powerful for answering real■world questions that involve many pieces of information.

Inner Join Basics

Imagine two lists of information that belong together, like a class roster and a list of exam scores. The roster tells us *who* is in the class (student ID, name), and the score sheet tells us *what* each student earned (student ID, score). If we want to see each student's name next to their score, we need to line up the rows that share the same student ID. That lining■up operation is what a **join** does, and an **inner join** keeps only the pairs that actually match.

> **Definition:** An inner join returns rows that have matching values in both tables involved in the join.

How the inner join works

1. **Choose the tables** - You write the names of the two tables you want to combine. 2. **Specify the matching column(s)** - In the `ON` clause you tell the database which column(s) must be equal for a pair of rows to be considered a match. 3. **Select the columns you need** - After the join, you can pick any columns from either table to appear in the result.

Only rows where the condition in the `ON` clause is true survive the join. If a student appears in the roster but has no score, that student disappears from the result; likewise, a score with no matching student is also dropped. This is why inner joins are called "inner" - they look only at the *inner* intersection of the two sets.

Basic syntax

The most common form uses the `INNER JOIN` keyword:

```sql SELECT t1.column_a, t2.column_b, t1.column_c FROM table1 AS t1 INNER JOIN table2 AS t2 ON t1.common_field = t2.common_field; ```

- `SELECT` lists the columns you want to see. - `FROM table1 AS t1` names the first table and gives it a short alias (`t1`). - `INNER JOIN table2 AS t2` tells the database to join `table2` (alias `t2`) to `t1`. - `ON t1.common_field = t2.common_field` is the **join condition**; it says "keep only those rows where the value of `common_field` in `t1` equals the value of `common_field` in `t2`".

The join condition can involve more than one column, using `AND` to require several matches:

$$ t1.id = t2.id \; \text{AND} \; t1.year = t2.year $$

Everyday comparison

Think of two puzzle pieces with matching tabs. Each piece has many tabs, but only the tabs that fit together let you snap the pieces. An inner join is like pressing the two pieces together and keeping only the parts where the tabs interlock. Anything that doesn't fit is left out of the picture.

> **In daily life:** Joining a contact list with a phone■call log is an inner join - you see only the calls made by people who are already in your contacts.

Why inner joins are useful

- **Combine related data** without duplicating information. - **Filter out irrelevant rows** automatically; you don't need a separate `WHERE` clause to discard non■matching rows. - **Simplify reporting** - a single query can pull together fields from many tables, giving you a complete view in one result set.

Quick example

Suppose we have:

| students (id, name) | |---------------------| | 1, Alice | | 2, Bob | | 3, Carol |

| scores (student_id, exam) | |---------------------------| | 1, 85 | | 2, 92 | | 4, 78 |

The query:

```sql SELECT s.name, sc.exam FROM students AS s INNER JOIN scores AS sc ON s.id = sc.student_id; ```

produces:

| name | exam | |-------|------| | Alice | 85 | | Bob | 92 |

Notice that Carol (no score) and the score for student 4 (no student record) are omitted.

> **Remember:** The `ON` clause decides which rows match; if the condition is never true, the result set will be empty.

> **Common mistake:** Writing `WHERE t1.id = t2.id` instead of using `ON` after `INNER JOIN`. The `WHERE` clause works, but it forces the database to create a Cartesian product first, which can be far slower for large tables.

With this foundation, you can now move on to outer joins, which keep the non■matching rows as well, or to joining more than two tables. The inner join remains the simplest and most frequently used tool for combining related data.

Outer Joins Overview

When you join two tables with an **inner join**, the result only contains rows that have matching values in both tables. An **outer join** works a little differently: it keeps rows that do **not** find a match in the other table, filling the missing columns with `NULL` (a special marker that means "no data").

> **Definition:** An outer join returns all rows from one (or both) tables, inserting `NULL` for columns that have no matching row in the other table.

Left Outer Join (LEFT JOIN)

Imagine two lists:

| **Students** | | |--------------|------------| | student_id | name | | 1 | Alice | | 2 | Bob | | 3 | Carol |

| **Enrollments** | | |----------------|------------| | student_id | course | | 1 | Math | | 2 | History |

A **left join** says: "Give me every row from the **left** table (the first one you write), and attach data from the right table when the keys match. If there is no match, still keep the left row and put `NULL` for the right columns."

SQL syntax:

$$ \text{SELECT } s.student_id, s.name, e.course \\ \text{FROM } Students\ s \\ \text{LEFT JOIN } Enrollments\ e \\ \text{ON } s.student_id = e.student_id; $$

Result:

| student_id | name | course | |------------|-------|--------| | 1 | Alice | Math | | 2 | Bob | History| | 3 | Carol | NULL |

Why useful? You often need a complete list of the primary entities (students, customers, products) even if some of them have not yet been linked to secondary data (enrollments, orders, reviews). The left join guarantees that none of the primary rows disappear.

> **In daily life:** Think of a school attendance sheet (left table) and a list of students who turned in homework (right table). A left join shows every student, marking "no homework" for those who didn't turn it in.

Right Outer Join (RIGHT JOIN)

A **right join** is the mirror image of a left join: it keeps every row from the **right** table and adds matching rows from the left table, inserting `NULL` when the left side lacks a match.

If we swap the order in the previous example:

$$ \text{SELECT } s.student_id, s.name, e.course \\ \text{FROM } Students\ s \\ \text{RIGHT JOIN } Enrollments\ e \\ \text{ON } s.student_id = e.student_id; $$

Result:

| student_id | name | course | |------------|-------|--------| | 1 | Alice | Math | | 2 | Bob | History|

Notice that Carol disappears because she is not present in the **right** table (Enrollments).

When to use it? When the table you write second (the right side) is the one you consider "complete" and you want to see any missing information from the first table.

> **Note:** Many developers prefer to write left joins only, simply swapping the table order, because it avoids confusion about which side is "right".

Full Outer Join (FULL JOIN)

A **full outer join** combines the effects of left and right joins: it returns **all** rows from both tables, matching where possible and filling with `NULL` where there is no counterpart.

SQL syntax:

$$ \text{SELECT } s.student_id, s.name, e.course \\ \text{FROM } Students\ s \\ \text{FULL JOIN } Enrollments\ e \\ \text{ON } s.student_id = e.student_id; $$

Result:

| student_id | name | course | |------------|-------|--------| | 1 | Alice | Math | | 2 | Bob | History| | 3 | Carol | NULL | | NULL | NULL | Science| *(if a course existed without a student)*

Why would you need this? When you want a complete picture of two related datasets, even the parts that don't line up. For example, a report that lists every employee **and** every contractor, showing which projects they are on, while still listing those who have no project assignments.

> **Common mistake:** Assuming a full join behaves like two inner joins stacked together. It actually *adds* rows, not removes them.

Comparing to Inner Join

| Join type | Rows kept when there is a match | Rows kept when there is **no** match | |-----------|---------------------------------|--------------------------------------| | **Inner** | Yes (both sides) | No | | **Left** | Yes (left side) | Yes (left side only) | | **Right** | Yes (right side) | Yes (right side only) | | **Full** | Yes (both sides) | Yes (both sides) |

> **Remember:** Use an inner join when you only care about records that exist in **both** tables. Use an outer join when you need to keep records from one or both tables even if the partner data is missing.

By choosing the appropriate outer join, you can answer questions like "Which students have not enrolled in any course?" (left join with `NULL` check) or "Which courses have no students yet?"

(right join or full join with `NULL` on the student side). This flexibility makes outer joins a powerful tool for real■world data analysis.

Writing Inner and Outer Join Queries

When you write a join query you are telling the database how to line up rows from two (or more) tables so that related information appears side■by■side. Think of two spreadsheets: one lists **students** and their IDs, the other lists **exam scores** together with the same student IDs. A join is like taking a pair of scissors, cutting out the matching rows from each sheet, and taping them together on the common ID column.

Below is a step■by■step walk■through for the four most common join types. We will use two example tables:

| **students** | | |--------------|----------------| | `student_id` | `name` | | 1 | Alice | | 2 | Bob | | 3 | Carol |

| **scores** | | |--------------|----------------| | `student_id` | `exam` | `score` | | 1 | Math | 85 | | 2 | Math | 78 | | 2 | English| 90 | | 4 | Math | 92 |

We will give each table a short **alias** (`s` for *students*, `c` for *scores*) so the query stays readable.

---

1. Inner Join - keep only rows that match in both tables

1. Start with `SELECT` and list the columns you want to see. 2. Write `FROM students AS s` - this tells the database to start with the *students* table and call it `s`. 3. Add `INNER JOIN scores AS c` - this says "join the *scores* table (alias `c`) to the rows we already have". 4. Specify the matching condition with `ON s.student_id = c.student_id`.

```sql SELECT s.student_id, s.name, c.exam, c.score FROM students AS s INNER JOIN scores AS c ON s.student_id = c.student_id; ```

The result contains only the students that have at least one score record (Alice, Bob). Carol disappears because she has no scores, and the score for student 4 disappears because there is no matching student.

> **Definition:** An **inner join** returns rows where the join condition is true for *both* tables.

---

2. Left Outer Join - keep every row from the left table

A **left outer join** (often just "left join") is useful when you want all records from the first table, even if there is no matching row in the second table. Missing values are filled with `NULL`.

```sql SELECT s.student_id, s.name, c.exam, c.score FROM students AS s LEFT JOIN scores AS c ON s.student_id = c.student_id; ```

Now Carol appears with `NULL` for `exam` and `score`, because she has no scores. The row for student 4 is still absent because it belongs to the right table only.

> **In daily life:** Imagine a class roster (left table) and a list of submitted assignments (right table). A left join shows every student, marking "no submission" for those who didn't turn in work.

---

3. Right Outer Join - keep every row from the right table

A **right join** does the opposite: keep all rows from the second (right) table. It is written the same way, just swapping the keywords.

```sql SELECT s.student_id, s.name, c.exam, c.score FROM students AS s RIGHT JOIN scores AS c ON s.student_id = c.student_id; ```

Now the row for student 4 appears, with `NULL` for `student_id` and `name` because there is no matching student record.

> **Note:** Many databases treat `RIGHT JOIN` the same as `LEFT JOIN` with the tables reversed, so you can always rewrite a right join as a left join if you prefer.

---

4. Full Outer Join - keep everything from both sides

A **full outer join** returns all rows from both tables, filling `NULL` where there is no match on the other side. Not all SQL engines support it directly; PostgreSQL and SQL Server do, while MySQL requires a workaround using `UNION`.

```sql SELECT s.student_id, s.name, c.exam, c.score FROM students AS s FULL OUTER JOIN scores AS c ON s.student_id = c.student_id; ```

Result: Alice, Bob, Carol (with NULL scores) **and** the score for student 4 (with NULL name).

> **Remember:** Use a full outer join when you need a complete picture of two related datasets, such as merging customer lists from two different departments.

---

5. Selecting only the columns you need

After the join, you can pick any columns you like. If two tables have a column with the same name (e.g., both have `student_id`), you must qualify it with the alias, otherwise the database will not know which one you mean.

```sql SELECT s.name, c.exam, c.score FROM students AS s LEFT JOIN scores AS c ON s.student_id = c.student_id; ```

Here we omitted `s.student_id` because we already know the name identifies the student.

---

6. Putting it all together

A generic template for any join type looks like this:

$$ \text{SELECT } \text{alias1.columnA}, \text{alias2.columnB}, \dots \text{FROM } \text{table1} \text{ AS } \text{alias1} \text{[JOIN\_TYPE] } \text{table2} \text{ AS } \text{alias2} \text{ON } \text{alias1.key} = \text{alias2.key}; $$

Replace **[JOIN_TYPE]** with `INNER JOIN`, `LEFT JOIN`, `RIGHT JOIN`, or `FULL OUTER JOIN` as needed.
```

> **Common mistake:** Forgetting the `ON` clause or using the wrong column for the condition will produce a Cartesian product (every row paired with every other row), which quickly becomes a huge, meaningless result set. Always double■check that the columns you compare truly represent the same entity.

Joining More Than Two Tables

When you need information that lives in three or more tables, you simply **chain** the joins - one after another - just like linking a chain of paper clips. Each join connects two tables at a time, but the result of the first join becomes the left■hand side for the next join.

> **Definition:** *Chaining joins* means writing several `JOIN` clauses in a single `SELECT` statement, each one using the result of the previous join as its starting point.

Step■by■step example

Imagine a small school database with three tables:

| Table | Columns | |-------|---------| | `Students` | `student_id`, `name`, `class_id` | | `Classes` | `class_id`, `teacher_id`, `room` | | `Teachers` | `teacher_id`, `teacher_name` |

We want a list that shows each student's name, the room where their class meets, and the teacher's name.

1. **First join** `Students` to `Classes` on the column that relates them (`class_id`). 2. **Second join** the result to `Teachers` on the column that relates `Classes` to `Teachers` (`teacher_id`).

The SQL looks like this:

$$ \text{SELECT s.name, c.room, t.teacher_name} \\ \text{FROM Students AS s} \\ \text{JOIN Classes AS c ON s.class_id = c.class_id} \\ \text{JOIN Teachers AS t ON c.teacher_id = t.teacher_id;} $$

Notice that after the first `JOIN`, the query already knows about columns from both `Students` and `Classes`. The second `JOIN` can refer to any column that is already in the current result set - in this case `c.teacher_id`.

> **In daily life:** Think of planning a road trip. First you decide which city to start from (Students). Then you pick the highway that connects that city to the next town (Classes). Finally you choose the final destination that the highway leads to (Teachers). Each step builds on the previous one.

Why join order matters

SQL does not always execute the joins in the order you write them, but the **logical order** you specify helps the database optimizer understand relationships. If you join a very large table first and then filter with a small table later, the engine may have to process many rows unnecessarily.

A good practice is to start with the table that **restricts** the result set the most (usually the one with the most selective `WHERE` conditions) and then add the other tables.

> **Note:** The order of `JOIN` clauses does not change the final result as long as you keep the same join conditions, but a sensible order can make the query run faster.

Keeping the query readable

When you have three or more joins, the statement can become long and hard to follow. Use these tricks:

* **Alias** each table with a short name (`s`, `c`, `t`). This reduces typing and makes column references clear. * Put each `JOIN` on its own line, aligned under the previous one. * Add comments (using `--`) to explain why a particular join is needed.

```sql SELECT s.name, c.room, t.teacher_name FROM Students AS s JOIN Classes AS c ON s.class_id = c.class_id -- which class each student takes JOIN Teachers AS t ON c.teacher_id = t.teacher_id -- who teaches that class WHERE c.room = '101'; -- only classes in room 101 ```

Extending to more tables

Suppose we also have a `Subjects` table that tells us the subject name for each class:

| `Subjects` | `class_id`, `subject_name` |

We can add another join:

```sql JOIN Subjects AS sub ON c.class_id = sub.class_id ```

Now the `SELECT` list can include `sub.subject_name`. The pattern is the same: each new table is linked by a column that already exists in the current result set.

> **Common mistake:** Forgetting to use the correct alias when referring to a column, e.g., writing `class_id = class_id` instead of `c.class_id = sub.class_id`. This creates a tautology that returns every row, defeating the purpose of the join.

Quick checklist

1. Identify the **key columns** that relate each pair of tables. 2. Choose short **aliases** for readability. 3. Write the first `JOIN` between the base table and the next most selective table. 4. Add additional `JOIN`s one by one, always using a column that is already present in the intermediate result. 5. Test the query with a small `WHERE` clause to verify you are getting the expected rows.

By chaining joins carefully, you can pull together any number of related pieces of information, just as you would link together several pieces of a puzzle to see the whole picture.

Common Pitfalls and Best Practices

When you start joining tables, it's easy to get a few hidden problems that make your result look wrong or run very slowly. Below we look at the three most common pitfalls - duplicate rows, missing join conditions, and performance - and give you simple habits that keep your queries clear and fast.

> **Common mistake:** *Leaving out a join condition or using the wrong column* If you write a join without telling the database how the two tables are related, the system will combine **every**

row of the first table with **every** row of the second table. This is called a **Cartesian product**. Imagine you have a list of 10 students and a list of 5 clubs. Without a condition, the database would produce 10 x 5 = 50 rows, even though each student only belongs to a few clubs. The result is a huge table filled with meaningless combinations.

Why it happens: The `ON` clause (or the `WHERE` clause for older syntax) tells the database which columns must match. If you forget it, the default is "match everything".

How to avoid it: 1. Always write an explicit `ON` clause. 2. Double■check that the columns you compare have the same data type and meaning (e.g., `student.id = enrollment.student_id`).

> **Note:** *Duplicate rows often appear when you join a one■to■many relationship* Suppose each order can have many line items. If you join `Orders` to `LineItems` and also join `Orders` to `Payments`, the same order row will appear once for each line item **and** once for each payment, producing a multiplication of rows. This is not an error in the database; it's a result of joining more than one "many" side to the same "one" side.

Every new word: - **One■to■many relationship** - one row in the first table matches many rows in the second table. - **Many■to■many relationship** - rows in both tables can match many rows in the other; usually handled with a junction table.

Everyday comparison: Think of a classroom roster (students) and a list of books each student borrowed (borrowings). If you also join a list of extracurricular activities, a student who borrowed three books and joined two clubs will appear 3 x 2 = 6 times in the final result.

How to keep duplicates under control: - Use `DISTINCT` only when you truly need to remove duplicates; it forces the database to scan all rows again, which can be slow. - Prefer aggregating before joining: compute totals per student first, then join the aggregated result to other tables.

> **Remember:** *Performance depends on indexes and the order of joins* An **index** is a data structure that lets the database find rows quickly, much like a book's index lets you locate a topic without reading every page. If you join on a column that isn't indexed, the database may have to scan the whole table (a **full table scan**), which is much slower.

Tips for efficient joins:

1. **Index the join columns** - put an index on every column that appears in an `ON` clause, especially foreign keys. 2. **Join the smallest tables first** - the query planner often starts with the table that returns the fewest rows, reducing the amount of data it has to carry forward. 3. **Select only needed columns** - writing `SELECT *` forces the database to retrieve every column, even those you never use. Specify the exact columns you need.

$$ \text{Cost} \approx \text{Rows}_{\text{left}} \times \text{Rows}_{\text{right}} \; \text{if no index} $$

If an index exists, the cost drops to roughly the number of matching rows, not the product of the two table sizes.

Practical habit: Write your query in this order:

```sql SELECT o.order_id, c.customer_name, SUM(li.quantity) AS total_qty FROM Orders AS o JOIN Customers AS c ON o.customer_id = c.id -- indexed foreign key JOIN LineItems AS li ON li.order_id = o.id -- indexed foreign key WHERE o.order_date >= '2024-01-01' -- filter early
```

GROUP BY o.order_id, c.customer_name; ```

Notice how the `WHERE` clause narrows the data **before** the `GROUP BY`, and each `JOIN` uses an indexed key.

By always checking for a proper join condition, watching out for unintended duplication, and making sure the columns you join on are indexed, you'll write queries that are both correct and fast. Keep these habits, and the "join" part of your database work will feel as smooth as pairing socks from two drawers that are already organized.

Practice Exercise and Summary

Imagine a small online bookstore. The store keeps two tables:

* **Customers** - each row stores a customer's `customer_id`, `name`, and `city`. * **Orders** - each row stores an `order_id`, the `customer_id` of the buyer, the `book_title`, and the `order_date`.

Your task is to write the queries you have learned about joins to answer three questions.

1. **List every order together with the name of the customer who placed it.** This needs an **inner join** because we only want orders that have a matching customer.

2. **Show all customers and, if they have placed an order, the title of the book they bought.** Here we need a **left outer join** so that customers without any orders still appear, with `NULL` in the book column.

3. **Find every city that has at least one order, and also list the cities that have no orders at all.** This requires a **full outer join** (or two separate queries combined) to keep rows from both tables even when there is no match.

Write the SQL for each question, using the column names given above. Test your queries on the sample data below (you can create the tables with `CREATE TABLE` statements if you like).

```sql INSERT INTO Customers (customer_id, name, city) VALUES (1, 'Alice', 'Springfield'), (2, 'Bob', 'Rivertown'), (3, 'Carol', 'Lakeside');

INSERT INTO Orders (order_id, customer_id, book_title, order_date) VALUES (101, 1, 'SQL Basics', '2024-09-01'), (102, 1, 'Data Modeling', '2024-09-15'), (103, 2, 'Python for Kids', '2024-10-05'); ```

---

> **Definition:** A **join** combines rows from two (or more) tables by matching values in specified columns, producing a new temporary table that you can query.

> **Note:** An **inner join** returns only rows where the join condition is true for *both* tables. An **outer join** (left, right, or full) keeps rows from one or both tables even when there is no matching row, filling missing values with `NULL`.

> **In daily life:** Think of a join like matching two lists of friends. If you list everyone who attended a party (list A) and everyone who brought a gift (list B), an inner join shows only those who both attended *and* brought a gift. A left outer join would show every party attendee, marking "no gift" for those who didn't bring one.
```

> **Common mistake:** Forgetting to use the correct join type. Using an inner join when you need a left outer join will silently drop rows you actually wanted to keep, leading to incomplete reports.

Now, recap the most important ideas from today's lesson.

1. **Join types** - inner, left outer, right outer, and full outer - each control which rows are preserved when there is no match. 2. **Join condition** - the `ON` clause tells the database which columns to compare, usually primary key ↔ foreign key pairs. 3. **Multiple joins** - you can chain joins to connect three or more tables, remembering to use parentheses or clear alias names to keep the query readable. 4. **NULL handling** - outer joins introduce `NULL` for missing data; functions like `COALESCE` can replace `NULL` with a default value.

A useful formula for an inner join looks like this:

$$ \text{Result} = \{\, r_1 \cup r_2 \mid r_1 \in \text{Table1},\; r_2 \in \text{Table2},\; r_1.key = r_2.key \,\} $$

For a left outer join, the set is extended with rows from the left table that have no match:

$$ \text{Result} = \text{InnerJoin} \;\cup\; \{\, r_1 \mid r_1 \in \text{Table1},\; \nexists r_2 \in \text{Table2},\; r_1.key = r_2.key \,\} $$

Remember: Always check the result of a join by looking at a few rows. If you see unexpected `NULL`s or missing rows, revisit the join type and the `ON` condition.

When you finish the three queries, compare the output with the expected tables below. If any rows are missing or extra, identify which join type caused the issue and correct it. This practice will help you choose the right join automatically in real■world projects.

Figure (described): Shows which rows are kept by inner and outer joins.

Figure (described): Combines student names with their clubs using a join.

Figure (described): Three tables linked together to form a single query result.

Key terms

Join: An operation that merges rows from two tables based on related column values.

Inner Join: Returns only rows where the join condition matches in both tables.

Outer Join: Returns all rows from one or both tables, using NULL for missing matches.

Left Join: Keeps all rows from the left table and adds matching rows from the right table, or NULL if none match.

Right Join: Keeps all rows from the right table and adds matching rows from the left table, or NULL if none match.

Full Outer Join: Keeps all rows from both tables, inserting NULL where there is no match.

Cartesian Product: Result of joining tables without a condition, pairing every row of one table with every row of the other.

Chaining Joins: Writing multiple JOIN clauses in one SELECT so that each join builds on the previous result.

8. Grouping and Aggregates

This lesson introduces aggregate functions and how to group rows in SQL queries. Students learn to count, sum, average, and filter groups using GROUP BY and HAVING.

What you will be able to do

- Explain what an aggregate function does
- Write queries that use COUNT, SUM, and AVG
- Group rows with GROUP BY
- Filter groups with HAVING
- Interpret the results of grouped queries

What Is an Aggregate Function?

Imagine you have a list of many numbers, like the scores of all students in a class exam. If you look at each score one by one, you see many separate values. An **aggregate function** is a special tool that looks at all those values together and squeezes them down to **one single result**.

> **Definition:** An aggregate function is a built■in operation that takes a set of rows (or a column of values) as input and returns a single scalar value that summarizes that set.

Why do we need such a function? Often we are not interested in every individual row; we want to know something about the whole collection. For example, a teacher may want to know the **average** score of the class, not each student's exact mark. The average tells the teacher quickly how the class performed overall. That single number is the output of an aggregate function applied to many rows.

How it works, step by step

1. **Select the column** that contains the values you want to summarize. 2. **Apply the aggregate function** to that column. 3. The database engine scans every row that matches the query, feeds each value into the function, and finally returns one number.

Think of it like a kitchen blender. You put many fruit pieces (the rows) into the blender, press the button (the aggregate function), and out comes a smooth drink (the single result). The blender does not give you each fruit back; it gives you the blended mixture, which represents the whole batch.

Common aggregate functions you will meet

- `COUNT` - tells you **how many rows** there are. - `SUM` - adds up all the numbers. - `AVG` - calculates the **average** (mean) value. - `MAX` and `MIN` - give the largest and smallest values.

Each of these follows the same pattern: many inputs -> one output.

A tiny example in SQL

Suppose we have a table called `sales` with a column `amount`. To find the total sales we write:

$$ \text{SELECT SUM(amount) FROM sales;} $$

The `SUM` function looks at every `amount` value, adds them together, and returns just one number - the total.

If we only want to know how many sales records exist, we use `COUNT`:

$$ \text{SELECT COUNT(*) FROM sales;} $$

The asterisk `*` means "all rows". The result is a single integer representing the row count.

> **In daily life:** Counting the number of apples in a basket, adding up the price of all items on a receipt, or finding the average temperature over a week are all everyday versions of aggregate functions.

What makes an aggregate different from ordinary functions?

A regular (scalar) function, like `ABS(x)` which returns the absolute value of a single number, works on **one value at a time**. You must call it separately for each row if you want every row's result. An aggregate function, on the other hand, automatically processes the whole set in one go and never returns a separate value for each row - only the single summary.

> **Remember:** Aggregate functions always produce **one** result per query (or per group, when we later use `GROUP BY`). They never return a list of values.

Why the result is always a single value

The mathematics behind an aggregate is a reduction operation: it reduces a collection to a single element. For `SUM`, the reduction rule is addition; for `AVG`, it is addition followed by division by the count. Because the reduction rule combines every element into one, the final output cannot be a list - it must be a single scalar.

> **Note:** When you combine aggregates with other columns that are not aggregated, you must tell the database how to group rows (using `GROUP BY`). Otherwise the database would not know which rows belong together for the reduction.

Understanding aggregate functions is the first step toward summarizing data efficiently. Once you can turn many rows into one meaningful number, you can answer questions like "How many students passed?", "What is the total revenue this month?", or "Which product sold the most?" - all without manually inspecting each individual record.

COUNT - Counting Rows

COUNT is the simplest aggregate function. It looks at a set of rows and tells you **how many** rows are there. If you add a column name inside the parentheses, COUNT only counts the rows where that column is **not NULL** (i.e., where a real value exists).

> **Definition:** `COUNT(*)` returns the total number of rows in the selected set, while `COUNT(column)` returns the number of rows with a non■NULL value in *column*.

Why is this useful? Imagine you run a school and you want to know how many students are enrolled, or a shop that wants to know how many orders actually received a discount. Counting rows gives you a quick summary without having to look at each record individually.

> **In daily life:** Counting rows is like counting the number of people standing in a line. If you only count the people who are wearing a hat, you are doing the same as `COUNT(column)` where the column records "has_hat".

How COUNT works step by step

1. **Select the rows** - The `FROM` clause tells the database which table to look at, and any `WHERE` clause can filter the rows first. 2. **Apply COUNT** - * `COUNT(*)` simply adds 1 for every row that survived the filter. * `COUNT(col)` adds 1 only when the value in *col* is not NULL. 3. **Return a single number** - The result is a single scalar value, not a list of rows.

Because COUNT never looks at the actual data values (except to test for NULL), it is very fast. The database can often compute it using internal statistics without scanning every column.

Example 1 (Easy): Count how many students are in the table.

We have a table `Students(id, name, grade)` and we want the total number of students.

1. **Choose the table** - `FROM Students`. 2. **Apply COUNT to all rows** - use `COUNT(*)`.

$$\text{total_students} = \text{COUNT(*)}$$

3. **Run the query** -

```sql SELECT COUNT(*) AS total_students FROM Students; ```

Answer: The query returns a single number, e.g., `total_students = 124`. This means there are 124 rows (students) in the `Students` table.

Example 2 (Medium): Count how many orders have a non■NULL discount column.

Table `Orders(order_id, customer_id, discount)` stores a discount amount, but some orders have no discount (NULL). We need the count of orders where a discount was actually recorded.

1. **Select the table** - `FROM Orders`. 2. **Filter to non■NULL discounts** - use `COUNT(discount)`.

$$\text{discounted\_orders} = \text{COUNT(discount)}$$

3. **Run the query** -

```sql SELECT COUNT(discount) AS discounted_orders FROM Orders; ```

Answer: If the result is `discounted_orders = 37`, then 37 orders have a discount value; the remaining orders either have a discount of 0 or a NULL (meaning no discount was applied).

> **Note:** `COUNT(discount)` ignores rows where *discount* is NULL. If you wrote `COUNT(*)` you would get the total number of orders, not just the discounted ones.

When to prefer `COUNT(*)` vs. `COUNT(column)`

- Use `COUNT(*)` when you need the size of the whole result set, regardless of any column values. - Use `COUNT(column)` when you are interested only in rows that contain meaningful data in that column.

> **Common mistake:** Writing `COUNT(column)` expecting it to count all rows, even those where *column* is NULL. The function will skip NULLs, leading to a smaller number than intended.

Quick checklist

- Make sure the column you count is the one that matters for your question. - Remember that `NULL` means "unknown" or "missing"; COUNT treats it as invisible. - If you need to count rows that meet a condition (e.g., orders with a discount **greater than 0**), combine `WHERE` with `COUNT(*)` instead of relying on `COUNT(column)`.

> **Remember:** `COUNT` always returns a single integer value; it never returns NULL itself. Even if the result set is empty, `COUNT(*)` returns 0.

With this foundation, you can now use COUNT as a building block for more complex queries, such as grouping students by grade or finding the number of orders per customer. The next sections will show how to combine COUNT with `GROUP BY` and `HAVING` to answer those kinds of questions.

SUM and AVG - Adding and Averaging

When we talk about **adding** numbers in a database, we use the **SUM** function. When we want the **average** (the "typical" value) of a set of numbers, we use the **AVG** function. Both functions look at a column that contains numeric data (like prices, scores, or ages) and return a single number that summarizes the whole set.

> **Definition:** `SUM(column)` adds together every value that appears in *column* for the rows that are being considered. > **Definition:** `AVG(column)` adds the same values together and then divides by the count of those values, giving the arithmetic mean.

Why are these useful? Imagine you run a small shop and you keep a table called **Sales** with a column **price** for each item sold. At the end of the day you want to know how much money you made - that is exactly what `SUM(price)` tells you. Now think of a teacher who records test scores for each class. She may want to know how well the class performed on average, which is what `AVG(score)` provides.

How SUM works, step by step

1. **Select the rows** you are interested in (maybe all rows, or only those that meet a condition). 2. **Take the numeric value** from the chosen column for each row. 3. **Add them together**.

Mathematically this is written as

$$ \text{SUM} = \sum_{i=1}^{n} v_i $$

where v_i is the value in the column for row i, and n is the number of rows being summed.

How AVG works, step by step

1. **Select the rows** you want to average. 2. **Add the values** exactly as `SUM` does. 3. **Count how many values** were added. 4. **Divide the total sum by the count**.

The formula looks like

$$ \text{AVG} = \frac{\displaystyle\sum_{i=1}^{n} v_i}{n} $$

If any row has a *NULL* (unknown) value, that row is ignored - the functions only use the numbers that actually exist.

> **In daily life:** Adding up the prices of groceries in your cart is like `SUM`. Dividing that total by the number of items gives you the average price per item, just like `AVG`.

Worked Example - Easy: Total price of all items in the **Sales** table

Example 3 (Easy): Sum the total price of all items in the Sales table.

We have a table **Sales** with a column **price** that contains the following values: 12.50, 7.99, 15.00, 3.45.

1. **Identify the column** to sum: `price`. 2. **List the values**: 12.50, 7.99, 15.00, 3.45. 3. **Add them together**

$$ \text{Total} = 12.50 + 7.99 + 15.00 + 3.45 $$

Answer: The total sales amount is **$38.94**.

Worked Example - Medium: Average score of each class

Example 4 (Medium): Find the average score of each class.

Assume a table **Results** with columns **class_id** and **score**. Data:

| class_id | score | |----------|-------| | A | 78 | | A | 85 | | A | 92 | | B | 66 | | B | 74 | | B | 80 |

We want the average score for class A and for class B.

1. **Group rows by class** so that each class is treated separately. 2. **For class A**, list the scores: 78, 85, 92. - Compute the sum

$$ \text{SUM}_A = 78 + 85 + 92 = 255 $$

- Count the rows: $n_A = 3$. - Divide to get the average

$$ \text{AVG}_A = \frac{255}{3} = 85 $$

3. **For class B**, list the scores: 66, 74, 80. - Compute the sum

$$ \text{SUM}_B = 66 + 74 + 80 = 220 $$

- Count the rows: $n_B = 3$. - Divide to get the average

$$ \text{AVG}_B = \frac{220}{3} \approx 73.33 $$

Answer: - Class A average score = **85** points. - Class B average score ≈ **73.33** points.

When to use SUM vs. AVG

- Use **SUM** when you need a total amount, such as total revenue, total distance traveled, or total inventory cost. - Use **AVG** when you need a typical value, like average test score, average temperature, or average salary.

> **Remember:** `AVG` is essentially `SUM` divided by `COUNT`. If you already have the sum and the count, you can compute the average yourself, but the database does it for you automatically.

Common pitfalls

> **Common mistake:** Forgetting that `NULL` values are ignored. If a column contains a `NULL`, `SUM` and `AVG` treat it as if the row does not exist for that calculation. This can lead to a lower count than you expect, especially for `AVG`. Always check for `NULL`s or use `COALESCE(column, 0)` if you want to treat missing numbers as zero.

By mastering `SUM` and `AVG`, you gain two of the most frequently used tools for turning raw numbers into meaningful insights. They are the foundation for many reports, dashboards, and data■driven decisions.

GROUP BY - Creating Groups

When we use an aggregate function such as `COUNT`, `SUM` or `AVG`, the database looks at **all** rows that match the query and produces a single number. Often we do not want one number for the whole table, but a separate number for each category - for example, the total sales for each store, or the average salary for each department. That is exactly what the **`GROUP BY`** clause does: it tells the database to **split** the rows into groups that share the same value(s) in one or more columns, then apply the aggregate function **inside each group**.

> **Definition:** `GROUP BY` divides a result set into groups that have identical values in the specified column(s). After grouping, any aggregate function is calculated separately for each group.

Why is this useful? Think of a classroom where the teacher wants to know the average test score for each grade level. If the teacher simply averages all scores together, the result hides the differences between grades. By first separating the students into grade■level groups, the teacher can compute a meaningful average for each grade. The same idea works for data stored in a database.

How `GROUP BY` works, step by step

1. **Select the columns that will define the groups.** These columns appear after the `GROUP BY` keyword. All rows that have the same value in these columns belong to the same group.

2. **Collect rows into groups.** The database scans the table, compares the values of the grouping columns, and puts matching rows together.

3. **Apply aggregate functions to each group.** For every group, the database evaluates functions like `SUM`, `COUNT`, `AVG`, `MAX`, `MIN`. The result is one row per group.

4. **Return one output row per group.** The output contains the grouping column(s) and the calculated aggregates.

> **In daily life:** Imagine a grocery store manager who wants to know how many apples were sold each day. The manager first sorts the sales receipts by date (the grouping column) and then adds up the apple quantities for each date. The sorted dates are the "groups," and the addition is the aggregate.

Important rule about columns in the `SELECT` list

When you write a `SELECT` statement with `GROUP BY`, **every column that is not inside an aggregate function must appear in the `GROUP BY` clause**. This rule ensures the database knows how to assign each non■aggregated value to a specific group.

> **Common mistake:** Forgetting to list a non■aggregated column in `GROUP BY`. The query will either error out or give unexpected results.

Worked Example 1 - Total sales per store for June (Hard)

Example 5 (Hard): Total sales per store for June

Question: From the table **Sales** (`store_id`, `sale_date`, `amount`), find the total amount sold by each store during June 2023.

Steps

1. **Identify the grouping column.** We want totals per store, so we group by `store_id`.

$$\text{GROUP BY store_id}$$

2. **Restrict rows to June 2023.** Use a `WHERE` clause to keep only rows where `sale_date` falls in June.

$$\text{WHERE sale_date BETWEEN '2023-06-01' AND '2023-06-30'}$$

3. **Sum the amounts inside each group.** Apply the aggregate `SUM(amount)` to get the total for each store.

$$\text{SELECT store_id, SUM(amount) AS total_june}$$

Answer: | store_id | total_june | |----------|------------| | 101 | 45 230.00 | | 102 | 38 750.00 | | 103 | 52 410.00 |

(Values are illustrative; each row shows the total sales for that store in June.)

Worked Example 2 - Average salary per department (Hard)

Example 6 (Hard): Average salary for each department

Question: From the table **Employees** (`dept_id`, `salary`), compute the average salary for every department.

Steps

1. **Choose the grouping column.** Departments are identified by `dept_id`, so we group by that column.

$$\text{GROUP BY dept_id}$$

2. **Apply the average aggregate.** Use `AVG(salary)` to calculate the mean salary inside each department.

$$\text{SELECT dept_id, AVG(salary) AS avg_salary}$$

3. **(Optional) Order the result for readability.** Sort by `dept_id` or by `avg_salary` if we want the highest■paid department first.

$$\text{ORDER BY dept_id}$$

Answer: | dept_id | avg_salary | |---------|------------| | 10 | 68 200.00 | | 20 | 74 500.00 | | 30 | 59 750.00 |

Each row tells us the average salary of the employees in that department.

Putting it all together

A full query that combines the ideas above looks like this:

```sql SELECT store_id, SUM(amount) AS total_june FROM Sales WHERE sale_date BETWEEN '2023-06-01' AND '2023-06-30' GROUP BY store_id ORDER BY store_id; ```

Notice how the `GROUP BY` appears **after** the `WHERE` clause (filtering rows first) and **before** the `ORDER BY` clause (sorting the final groups).

> **Remember:** `GROUP BY` works on the rows that remain after `WHERE` has filtered them. The order of clauses is `SELECT ... FROM ... WHERE ... GROUP BY ... HAVING ... ORDER BY ...`.

Quick checklist when writing a `GROUP BY` query

- Identify the column(s) that define the groups. - Put those column(s) in the `GROUP BY` list. - Make sure every column in `SELECT` that is not aggregated appears in `GROUP BY`. - Use `WHERE` to filter rows **before** grouping, and `HAVING` to filter groups **after** aggregation.

By mastering `GROUP BY`, you can turn a flat list of records into meaningful summaries, just like sorting receipts by date to see daily totals or grouping students by class to compare their average scores. This ability is at the heart of data analysis and reporting.

HAVING - Filtering Groups

When we group rows with **GROUP BY**, the database first collects rows that share the same values in the chosen columns. After the groups are built, we can compute aggregate values such as `COUNT`, `SUM`, or `AVG` for each group. Sometimes we only want to keep the groups that satisfy a certain condition - for instance, "only departments whose average salary is above $70 000". That is what the **HAVING** clause does.

> **Definition:** `HAVING` is a clause that filters groups after the `GROUP BY` step, keeping only those groups that meet a specified condition on aggregate results.

Why do we need a separate clause? The `WHERE` clause filters rows **before** they are grouped, while `HAVING` works **after** the grouping. Think of a classroom: `WHERE` decides which students are allowed to sit in the room (e.g., only those who arrived on time). `HAVING` looks at the groups of students that formed later (e.g., teams for a project) and decides which teams stay in the competition (e.g., only teams whose average score exceeds a threshold).

How `HAVING` fits into a query

A typical query with aggregation looks like this:

```sql SELECT department, AVG(salary) AS avg_sal FROM employees GROUP BY department HAVING AVG(salary) > 70000; ```

1. `FROM` tells the database where to read the rows. 2. `WHERE` (if present) would discard rows that do not meet a condition. 3. `GROUP BY department` builds one group for each distinct department. 4. `SELECT` computes `AVG(salary)` for each group and names it `avg_sal`. 5. `HAVING AVG(salary) > 70000` removes any department whose average salary is **not** greater than $70 000.

> **Note:** The condition inside `HAVING` can use any aggregate function (`COUNT`, `SUM`, `MAX`, `MIN`, `AVG`) and can also combine them with ordinary columns that appear in the `GROUP BY` list.

Step■by■step illustration

Suppose we have a table **employees**:

| emp_id | department | salary | |--------|------------|--------| | 1 | Sales | 65000 | | 2 | Sales | 72000 | | 3 | HR | 58000 | | 4 | HR | 61000 | | 5 | IT | 90000 | | 6 | IT | 85000 |

If we run the query above, the database will:

1. Group rows by `department` -> three groups: Sales, HR, IT. 2. Compute `AVG(salary)` for each group -> Sales = 68 500, HR = 59 500, IT = 87 500. 3. Apply `HAVING AVG(salary) > 70000` -> keep only the IT group.

The final result is a single row: `IT | 87500`.

> **In daily life:** Imagine you are sorting a pile of fruits by type (apples, oranges, bananas). After sorting, you only keep the piles that weigh more than 5 kg. The sorting step is like `GROUP BY`; the weighing and discarding step is like `HAVING`.

Worked examples

Example 7 (Hard): Departments with average salary greater than $70,000

Question: List each department whose average employee salary exceeds $70 000, and show that average.

1. **Group rows** by `department`. $$\text{Groups} = \{\text{Sales}, \text{HR}, \text{IT}\}$$ 2. **Compute the average salary** for each group using `AVG(salary)`. $$\text{Avg}_{\text{Sales}} = \frac{65000 + 72000}{2} = 68500$$ $$\text{Avg}_{\text{HR}} = \frac{58000 + 61000}{2} = 59500$$ $$\text{Avg}_{\text{IT}} = \frac{90000 + 85000}{2} = 87500$$ 3. **Apply HAVING** to keep groups where the average is > 70000. $$\text{Keep } \text{IT} \text{ because } 87500 > 70000$$

Answer: The only department meeting the condition is **IT** with an average salary of **$87,500**.

Example 8 (Very Hard): Products sold more than 100 times and whose total revenue exceeds $5,000

Question: From a `sales` table containing `product_id`, `quantity`, and `price_per_unit`, find the product IDs that have been sold in more than 100 transactions **and** whose total revenue (`SUM(quantity * price_per_unit)`) is greater than $5 000.

1. **Group rows** by `product_id`. $$\text{Groups} = \{p_1, p_2, \dots, p_n\}$$ 2. **Count the number of transactions** per product with `COUNT(*)`. $$\text{TransCount}_{p_i} = \text{COUNT(*) for } p_i$$ 3. **Compute total revenue** per product using `SUM(quantity * price_per_unit)`. $$\text{Revenue}_{p_i} = \sum (\text{quantity} \times \text{price_per_unit})$$ 4. **Apply HAVING** with two conditions combined by `AND`. $$\text{HAVING } \text{TransCount}_{p_i} > 100 \; \text{AND} \; \text{Revenue}_{p_i} > 5000$$ 5. **Select** the `product_id` that satisfy both.

Assume the table contains the following aggregated results after steps 2■3:

| product_id | TransCount | Revenue | |------------|------------|---------| | 101 | 120 | 4800 | | 102 | 95 | 6200 | | 103 | 130 | 7200 | | 104 | 150 | 4500 |

Applying the `HAVING` condition:

- 101: 120 > 100 **but** 4800 <= 5000 -> discard. - 102: 95 <= 100 -> discard. - 103: 130 > 100 **and** 7200 > 5000 -> keep. - 104: 150 > 100 **but** 4500 <= 5000 -> discard.

Answer: Only **product 103** meets both criteria; it was sold in 130 transactions and generated $7,200 in revenue.

> **Common mistake:** Writing the condition in `WHERE` instead of `HAVING`. `WHERE` cannot see aggregate results, so the query would either error out or return no rows because the condition would be evaluated on individual rows, not on the grouped totals.

> **Remember:** Use `HAVING` whenever you need to filter **after** aggregation. The columns you reference in a `HAVING` clause must either be aggregate expressions or appear in the `GROUP BY` list. This keeps the query logical and the database engine happy.

Combining Multiple Aggregates

When you write a query you can ask the database to give you many different summaries at the same time. Instead of asking "how many rows?" you can also ask "what is the total?" and "what is the average?" in one go. The database evaluates each aggregate function for the same set of rows (or for each group if you use **GROUP BY**) and then returns a single row that contains all the results.

> **Definition:** An *aggregate function* is a special kind of function that takes many input rows and returns a single value, such as `COUNT`, `SUM`, `AVG`, `MIN`, or `MAX`.

Why combine them? Think of a school report card. One line shows the number of subjects taken, the total marks earned, and the average mark. All three numbers come from the same list of marks, but you read them together because they describe the same student. In a database the same idea lets you see a complete picture of each group without running several separate queries.

Step■by■step logic

1. **Select the columns you want to keep** - usually the columns that identify the group (e.g., city name). 2. **Add each aggregate you need** - write `COUNT(*)`, `SUM(amount)`, `AVG(amount)`, etc., and give each a clear alias with `AS`. 3. **Group the rows** - use `GROUP BY` with the identifier columns. 4. **Order the result** - if you want the biggest totals first, add `ORDER BY` with the appropriate aggregate alias and the keyword `DESC` (descending).

All of these clauses are written in the same `SELECT` statement, so the database does the work only once.

Worked Example

Example 9 (Very Hard): City■wide sales summary *For each city, display the number of customers, total purchases, and average purchase amount, sorted by total purchases descending.*

1. **Identify the tables and columns** - suppose we have a table `Customers` with `city` and `customer_id`, and a table `Orders` with `customer_id` and `order_total`. 2. **Join the tables** - we need the city for each order, so we join on `customer_id`.

$$\text{FROM Customers c JOIN Orders o ON c.customer_id = o.customer_id}$$

3. **Group by city** - this creates one group per city.

$$\text{GROUP BY c.city}$$

4. **Add the three aggregates** - count distinct customers, sum all order totals, and compute the average order total.

$$\text{SELECT c.city, COUNT(DISTINCT c.customer_id) AS num_customers,}$$
$$\text{SUM(o.order_total) AS total_purchases, AVG(o.order_total) AS avg_purchase}$$

5. **Order by total purchases descending** - the city with the highest sales appears first.

$$\text{ORDER BY total_purchases DESC}$$

Putting it all together:

```sql SELECT c.city, COUNT(DISTINCT c.customer_id) AS num_customers, SUM(o.order_total) AS total_purchases, AVG(o.order_total) AS avg_purchase FROM Customers c JOIN Orders o ON c.customer_id = o.customer_id GROUP BY c.city ORDER BY total_purchases DESC; ```

Answer: The query returns a table like

| city | num_customers | total_purchases | avg_purchase |
|---|---|---|---| | New York | 124 | 1 254 300.00 | 10 115.73 | | Chicago | 98 | 987 450.00 | 10 080.71 | | ... | ... | ... | ... |

The rows are sorted so the city with the largest `total_purchases` appears at the top.

Using Multiple Aggregates without GROUP BY

Sometimes you want several summaries of the whole table, not per group. The same principle applies; just omit `GROUP BY`.

```sql SELECT COUNT(*) AS total_rows, SUM(salary) AS total_salary, AVG(salary) AS avg_salary FROM Employees; ```

The result is a single row with three numbers: how many employees, how much money the company pays in total, and the average salary.

> **Note:** All aggregates in the same `SELECT` must refer to the same set of rows. If you mix aggregated columns with non■aggregated columns, you must use `GROUP BY` to tell the database how to split the rows.

Ordering by an Aggregate Alias

When you give an aggregate an alias (e.g., `total_purchases`), you can refer to that alias in `ORDER BY`. This makes the query easier to read.

```sql SELECT department, COUNT(*) AS emp_cnt, AVG(salary) AS avg_sal FROM Employees GROUP BY department ORDER BY emp_cnt DESC; -- shows departments with most employees first ```
```

The database first computes `emp_cnt` for each department, then sorts the rows using that value.

> **In daily life:** Imagine you are sorting a list of classrooms by the number of students they contain. You first count the students in each room, then you arrange the rooms from biggest to smallest. The database does exactly the same with `ORDER BY emp_cnt DESC`.

Common Pitfalls

> **Common mistake:** Forgetting to give each aggregate a distinct alias and then trying to order by the original function name. Some SQL dialects allow `ORDER BY COUNT(*)`, but many require the alias, leading to an error.

Quick Checklist

- Write every aggregate you need in the same `SELECT`. - Give each aggregate a clear alias (`AS ...`). - If you need separate rows per category, add `GROUP BY` with the category columns. - Use `ORDER BY <alias> DESC` (or `ASC`) to sort by any aggregate result.

By combining aggregates, you get a richer, more informative result set with a single, efficient query. This saves time, reduces server load, and keeps your code tidy.

Common Pitfalls and Tips

> **Common mistake:** Mixing aggregated and non■aggregated columns without using **GROUP BY** will cause an error because the database does not know how to combine the raw values with the summary values.

When you write a query that contains an aggregate function such as `COUNT`, `SUM`, or `AVG`, every column that appears in the `SELECT` list must either be inside an aggregate function **or** be listed in a `GROUP BY` clause. Think of it like a classroom: if the teacher asks each group of students to report the total number of books they read, the teacher also needs to know which group each total belongs to. The group name (e.g., "Group A") is the **GROUP BY** part; the total number of books is the **aggregate** part. Without the group name, the teacher cannot match a total to a specific group, and the request is ambiguous.

> **Remember:** `WHERE` filters rows **before** aggregation, while `HAVING` filters groups **after** aggregation.

This distinction is crucial. `WHERE` works on the original table rows, so it can only refer to columns that exist in each row. `HAVING` works on the results of aggregation, so it can refer to aggregate expressions like `COUNT(*)` or `SUM(sales)`. An everyday analogy is sorting laundry: you first **remove** any dirty socks (`WHERE`) before you **count** how many clean pairs you have (`HAVING`). If you tried to count socks before removing the dirty ones, you would get a misleading total.

> **Note:** `HAVING` can be used without `GROUP BY` when you want to apply an aggregate condition to the whole table as a single group.

Pitfall 1 - Forgetting to Group

Suppose we have a table `Orders`:

| order_id | customer_id | amount | |----------|-------------|--------| | 1 | 101 | 50 | | 2 | 102 | 30 | | 3 | 101 | 20 |

If we write:

```sql SELECT customer_id, SUM(amount) FROM Orders; ```

the database will raise an error because `customer_id` is not aggregated and not grouped. The correct query adds `GROUP BY`:

```sql SELECT customer_id, SUM(amount) FROM Orders GROUP BY customer_id; ```

Now each `customer_id` is paired with the sum of its orders.

Pitfall 2 - Using `WHERE` Instead of `HAVING` for Aggregates

Imagine we want customers whose total spending exceeds $60. A wrong attempt is:

```sql SELECT customer_id, SUM(amount) AS total FROM Orders WHERE SUM(amount) > 60 GROUP BY customer_id; ```

`WHERE` cannot see `SUM(amount)` because the sum is calculated **after** the rows are filtered. The database will reject the query. The right way is:

```sql SELECT customer_id, SUM(amount) AS total FROM Orders GROUP BY customer_id HAVING SUM(amount) > 60; ```

Now the `HAVING` clause evaluates the sum for each group and keeps only those groups that satisfy the condition.

> **In daily life:** Think of `WHERE` as a sieve that removes unwanted items **before** you count them, while `HAVING` is a filter that discards counted groups you don't need **after** you have the totals.

Pitfall 3 - Mixing Aggregates and Non■Aggregates in `HAVING`

A subtle error is to place a non■aggregated column in `HAVING` without also grouping by it. For example:

```sql SELECT customer_id, SUM(amount) AS total FROM Orders GROUP BY customer_id HAVING customer_id > 100 AND SUM(amount) > 60; ```

Here `customer_id` is already part of the `GROUP BY`, so it is allowed. If we omitted `customer_id` from the `GROUP BY`, the query would be invalid because `HAVING` would be trying to use a column that is not uniquely defined for each group.

Worked Example - Fixing a Mixed■Up Query

Example 10 (Intermediate): Wrong use of WHERE with an aggregate

We have a table `Sales`:

| sale_id | region | revenue | |---------|--------|---------| | 1 | North | 120 | | 2 | South | 80 | | 3 | North | 150 | | 4 | East | 60 | | 5 | South | 200 |

We want to list each region whose total revenue is **greater than 200**.

Steps

1. **Identify the aggregate needed.** We need the sum of `revenue` per `region`, so we will use `SUM(revenue)`.
```

2. **Group the rows by region.** Write the `GROUP BY` clause to create one group for each region.

$$ \text{GROUP BY region} $$

3. **Filter groups with HAVING.** Apply the condition on the aggregated sum.

$$ \text{HAVING SUM(revenue) > 200} $$

4. **Combine into a full query.**

```sql SELECT region, SUM(revenue) AS total_rev FROM Sales GROUP BY region HAVING SUM(revenue) > 200; ```

Answer: The query returns

| region | total_rev | |--------|-----------| | North | 270 | | South | 280 |

Both North and South have total revenue exceeding 200.

Pitfall 4 - Assuming `COUNT(*)` Counts Only Non■NULL Values

`COUNT(*)` counts **all rows**, even if some columns are `NULL`. If you need to count only rows where a specific column has a value, use `COUNT(column_name)`. For example, `COUNT(email)` counts only rows where `email` is not `NULL`. Forgetting this can lead to over■counting.

Pitfall 5 - Overusing `HAVING` When `WHERE` Suffices

If a condition does not involve an aggregate, place it in `WHERE`. This lets the database eliminate rows early, which is usually more efficient. For instance, to ignore orders with a negative amount before summing:

```sql SELECT customer_id, SUM(amount) FROM Orders WHERE amount >= 0 -- filter rows first GROUP BY customer_id; ```

Putting `amount >= 0` in `HAVING` would still work, but it would be evaluated after the grouping, wasting resources.

Quick Checklist

- **Every column in SELECT** must be either aggregated **or** listed in `GROUP BY`. - **Use `WHERE`** for row■level conditions that do **not** involve aggregates. - **Use `HAVING`** for conditions that involve aggregates or that need to filter whole groups. - **Remember** `COUNT(*)` counts all rows; `COUNT(column)` ignores `NULL`s. - **Test** your query with a small data set to see how groups are formed before scaling up.

By keeping these rules in mind, you can avoid the most common errors when working with aggregates and groups, and write clear, correct SQL that behaves exactly as you expect.

Figure (described): Rows of a student■score table grouped by class

Figure (described): Counting rows and summing scores in a small table

Figure (described): Only groups with average score above 80 remain

Key terms

Aggregate function: A built■in operation that takes a set of rows and returns one summarized scalar value (e.g., COUNT, SUM, AVG).

COUNT: Counts rows; COUNT(*) counts all rows, COUNT(column) counts only rows where the column is not NULL.

SUM: Adds together all numeric values in a column for the selected rows.

AVG: Calculates the arithmetic mean of numeric values by dividing SUM by COUNT.

GROUP BY: Divides a result set into groups that share the same values in specified columns, then applies aggregates to each group.

HAVING: Filters groups after they have been formed by GROUP BY, based on aggregate conditions.

WHERE: Filters individual rows before any grouping or aggregation occurs.

NULL: A special marker indicating missing or unknown data; COUNT(column) ignores NULLs.

9. Subqueries and Views

In this lesson students learn how to write nested queries and create reusable views in SQL. They will see how subqueries and views simplify complex data retrieval and promote clean database design.

What you will be able to do

- Write a SELECT statement that contains a nested subquery
- Explain the difference between correlated and non■correlated subqueries
- Create a view that encapsulates a frequently used query
- Use a view in subsequent SELECT statements
- Identify situations where a view improves query readability and maintenance

What Is a Subquery?

A **subquery** is simply a query that lives inside another query. Think of it as a small question you ask the database first, and then you use the answer of that small question as part of a bigger question.

> **Definition:** A subquery (also called an inner query or nested query) is a SELECT statement placed inside the SELECT, FROM, WHERE, or HAVING clause of another SELECT statement.

Why would we do this? Often we need information that is not directly stored in a single table, or we need to compute a value before we can filter rows. By letting the database solve the inner query first, we can reuse its result without writing extra code or temporary tables.

Where a subquery can appear

1. **Inside the SELECT list** - you can ask for a computed column that comes from another query. Example:

```sql SELECT s.student_id, s.name, (SELECT AVG(score) FROM Exams e WHERE e.student_id = s.student_id) AS avg_score FROM Students s; ```

Here the inner `SELECT AVG(score)...` runs for each row of `Students` and returns a single number that becomes the column `avg_score`.

2. **Inside the FROM clause** - the inner query acts like a temporary table that the outer query can join to. Example:

```sql SELECT t.department, t.total_sales FROM (SELECT department, SUM(sales) AS total_sales FROM Orders GROUP BY department) AS t WHERE t.total_sales > 100000; ```

The subquery creates a derived table `t` containing each department's total sales; the outer query then filters it.

3. **Inside the WHERE clause** - this is the most common use. The inner query supplies a value (or a set of values) that the outer query compares against. Example:

```sql SELECT name, salary FROM Employees WHERE salary > (SELECT AVG(salary) FROM Employees); ```
```

The inner query finds the average salary; the outer query returns every employee earning more than that average.

4. **Inside the HAVING clause** - similar to WHERE, but it works after grouping has taken place. Example:

```sql SELECT department, COUNT(*) AS emp_cnt FROM Employees GROUP BY department HAVING COUNT(*) > (SELECT AVG(cnt) FROM (SELECT COUNT(*) AS cnt FROM Employees GROUP BY department) AS d); ```

The inner query calculates the average number of employees per department; the outer query keeps only departments with a count above that average.

> **Note:** The inner query is executed first, and its result is then used by the outer query. The database engine decides the most efficient order, but logically you can think of it as "solve the inner problem, then the outer one."

Everyday comparison

Imagine you are planning a road trip. First, you check the weather forecast for each city you might visit (the inner query). After you know which cities will have rain, you decide which route to take to avoid the rainy spots (the outer query). The weather check is a sub■question that informs your main decision.

> **In daily life:** A subquery is like looking up a phone number in a directory before calling someone. The lookup (inner query) gives you the number you need to place the call (outer query).

Things to keep in mind

- The inner query must return a result type that makes sense for the place it is used. For a SELECT list or WHERE clause expecting a single value, the subquery must return exactly one column and, usually, one row. - If the inner query returns multiple rows where only one is allowed, the database will raise an error. - Subqueries can be **scalar** (return one value), **row** (return one row with multiple columns), or **table** (return a set of rows). The context (SELECT, FROM, WHERE, HAVING) determines which kind is allowed.

> **Remember:** Use a subquery when you need a result that depends on data not directly available in the current row, or when you want to avoid creating a separate temporary table. It keeps your SQL concise and often improves readability.

Non■Correlated Subqueries

A **non■correlated subquery** is a query that lives inside another query but does not depend on any column of the outer query. Because it is independent, the database can run it **once**, get the result, and then use that result while evaluating the outer query.

> **Definition:** A non■correlated subquery is a SELECT statement placed inside another SELECT, INSERT, UPDATE or DELETE statement that can be executed on its own, without referring to columns of the surrounding query.

Why does this matter?

When the subquery can be solved on its own, the database engine can treat it like a temporary table. It first computes the subquery, stores the result (often in memory), and then uses that
```

stored result while scanning the rows of the outer query. This usually makes the whole statement faster than writing the same logic with many separate statements.

Simple example

Suppose we have two tables:

``` Students(id, name, class_id) Classes(id, name, teacher) ```

We want to list the names of all students who are in a class taught by **Mrs. Lee**.

A non■correlated subquery can do this:

```sql SELECT name FROM Students WHERE class_id IN (SELECT id FROM Classes WHERE teacher = 'Mrs. Lee'); ```

The inner query (`SELECT id FROM Classes ...`) does **not** look at any column from `Students`. It can be run by itself, producing a list of class IDs taught by Mrs. Lee, e.g. `{3,7,12}`. The outer query then checks each student's `class_id` against that list.

What the result looks like

A non■correlated subquery may return:

* **A single value** - for example `SELECT MAX(salary) FROM Employees`. The outer query can compare a column to that single value: `WHERE salary > (SELECT MAX(salary) FROM Employees)`. * **A set of values** - like the list of class IDs above. The outer query uses `IN`, `ANY`, or `ALL` to compare its column to the whole set.

Everyday comparison

Think of a non■correlated subquery as a **shopping list** you write before you go to the store. You first write down everything you need (the subquery result). Then, while you walk through the aisles (the outer query), you simply check whether each item you see is on your list. You never have to go back and rewrite the list based on what you see in the store; the list is fixed beforehand.

> **In daily life:** A non■correlated subquery is like preparing a fixed checklist before starting a task, then using that checklist repeatedly without changing it.

Step■by■step execution

1. **Parse** the whole SQL statement and identify the subquery. 2. **Execute** the subquery alone; store its result in a temporary structure. 3. **Scan** the outer query's rows. For each row, apply the condition that references the stored result. 4. **Return** the rows that satisfy the condition.

Because step 2 is done only once, the cost of the subquery does not grow with the number of rows in the outer query.

When to use a non■correlated subquery

* When you need a **constant reference** for many rows, such as the highest price, the earliest date, or a list of IDs that meet a certain criterion. * When you want to keep the query readable by separating a complex filter into its own block.

Small caution

If the subquery returns many rows, the database may need to store a large temporary table, which can affect performance. In such cases, consider rewriting the query with a **JOIN** instead.

> **Remember:** Non■correlated subqueries are evaluated once, so they are ideal for constants or fixed sets that many outer rows will compare against.

Correlated Subqueries

A **correlated subquery** is a sub■query that depends on a value from the outer query. In other words, the inner query cannot be run on its own - it needs a piece of data that the outer query supplies for each row it processes. Because of this dependency the database must evaluate the inner query **once for every row** produced by the outer query.

> **Definition:** A correlated subquery is a subquery that references one or more columns from the outer query, causing the inner query to be re■executed for each row of the outer result set.

Why does this matter?

When you need to compare each row to a set of values that are calculated **relative to that row**, a correlated subquery is a natural tool. It lets you write a single SQL statement that expresses a "for each row, find something else that matches this row's data". The database engine handles the repeated execution behind the scenes, so you do not have to write a loop in a programming language.

Everyday comparison

Think of a teacher grading a class of essays. The teacher reads each essay (the outer query) and, for each essay, looks up the rubric criteria that apply to that particular student's level (the inner query). The rubric lookup cannot be done once for the whole class; it must be done **for each essay** because different students may have different criteria. The teacher's process mirrors a correlated subquery: the outer loop supplies the student level, and the inner lookup uses that level to find the appropriate points.

Step■by■step example

Suppose we have two tables:

```sql Students(student_id, name, class_year) Grades(student_id, subject, score) ```

We want to list every student together with the **highest score they have received** in any subject. The inner query must know which student we are currently looking at, so we write:

```sql SELECT s.student_id, s.name, (SELECT MAX(g.score) FROM Grades g WHERE g.student_id = s.student_id) AS best_score FROM Students s; ```

1. The outer query (`SELECT ... FROM Students s`) produces one row for each student. 2. For each of those rows, the inner query `SELECT MAX(g.score) FROM Grades g WHERE g.student_id = s.student_id` is executed. 3. The condition `g.student_id = s.student_id` links the inner query to the current outer row, making it **correlated**. 4. The result of the inner query (the maximum score) is returned as the column `best_score`.

Because the inner query uses `s.student_id`, it cannot be run once for the whole table; it must be run once per student.

How the database evaluates it

The database engine typically follows this logical flow:

1. Fetch the first row from `Students`. 2. Plug the value of `student_id` into the inner query's `WHERE` clause. 3. Execute the inner query, obtain the maximum score, and attach it to the outer row. 4. Move to the next row of `Students` and repeat steps 2■3.

This row■by■row evaluation can be slower than a non■correlated subquery, especially on large tables, because the inner query may be executed thousands of times.

> **Note:** Correlated subqueries are often less efficient than joins or window functions, because they require repeated execution. Use them when the logic is clearer or when a join would be more complex.

A slightly more complex case

Imagine we need to find all students whose score in **any** subject is higher than the average score of their class year. Here the inner query must compute the average for the **class year of the current student**, so it is also correlated:

```sql SELECT s.student_id, s.name, g.subject, g.score FROM Students s JOIN Grades g ON s.student_id = g.student_id WHERE g.score > (SELECT AVG(g2.score) FROM Grades g2 JOIN Students s2 ON g2.student_id = s2.student_id WHERE s2.class_year = s.class_year); ```

The inner `SELECT AVG(g2.score) ... WHERE s2.class_year = s.class_year` uses `s.class_year` from the outer query, making the whole `WHERE` clause a correlated condition.

When to prefer other techniques

If you can rewrite the same logic with a **JOIN** or a **window function**, you often gain performance because the database can process the data in a set■based manner rather than row■by■row. However, correlated subqueries are still valuable for quick ad■hoc queries or when the logic is naturally expressed as "for each row, find something that matches this row".

> **Remember:** Use a correlated subquery when you need a value that depends on each individual row of the outer query, but consider alternatives (JOINs, window functions) for large data sets to keep queries fast.

Creating Views

A view is simply a **saved query**. When you write a `SELECT` statement, the database runs it and returns a result set. If you write exactly the same `SELECT` many times, you are repeating work and risking mistakes. A view lets you give that `SELECT` a name and store it in the database, so you can reuse it just by referring to the name.

> **Definition:** A view is a virtual table defined by a `SELECT` statement. It does not store data itself; it shows the data that the underlying tables contain at the moment you query the view.

How a view is created

The SQL command to create a view looks like this:

```sql CREATE VIEW view_name AS SELECT column1, column2, ... FROM table1 JOIN table2 ON ... WHERE condition; ```

- `CREATE VIEW` tells the database you want to define a new view. - `view_name` is the identifier you will use later, just like a table name. - The word `AS` separates the name from the query that defines the view. - Everything after `AS` is a normal `SELECT` statement. It can include joins, aggregates, `WHERE` clauses, and even other views.

Because a view is only a stored query, the database does not duplicate the rows. When you ask for data from the view, the database runs the underlying `SELECT` at that moment and returns the current result.

Why store a query as a view?

Think of a view like a **recipe card** in a kitchen. The recipe (the `SELECT`) tells you how to combine ingredients (the tables) to make a dish (the result). If you need the same dish many times, you keep the card on the wall instead of rewriting the whole recipe each time. If the ingredients change (rows are added or updated), the dish automatically reflects the new ingredients because the recipe is executed fresh each time.

Naming conventions for views

A clear name makes it obvious that you are dealing with a view, not a real table. Common practices are:

1. **Prefix** the name with `v_` or `vw_`. Example: `v_customer_orders` or `vw_active_students`. 2. Use **lowercase** letters and **underscores** to separate words. This matches the style often used for table names and avoids case■sensitivity issues in some databases. 3. Make the name **descriptive** of the result set. If the view shows only active employees, `v_active_employees` tells the reader exactly what to expect.

> **Note:** A view name must be unique within the same schema, just like a table name. Trying to create a view with a name that already exists will raise an error.

Simple example

Suppose you have a table `orders` and you frequently need the total amount per customer. You can create a view:

```sql CREATE VIEW v_customer_totals AS SELECT customer_id, SUM(order_amount) AS total_spent FROM orders GROUP BY customer_id; ```

Now, instead of writing the whole `SELECT ... GROUP BY ...` each time, you simply query:

```sql SELECT * FROM v_customer_totals WHERE total_spent > 1000; ```

The database runs the stored `SELECT` behind `v_customer_totals`, groups the orders, and then applies the outer `WHERE` clause.

When a view is not enough

Because a view does not store data, every query that uses it must re■execute the underlying `SELECT`. If the query is very complex and runs on a huge table, performance can suffer. In such cases you might consider a **materialized view** (a view that stores the result) or a temporary table, but those are advanced topics.

> **Remember:** A view is a virtual table - it shows data *as it exists now*, and it is defined by a single `SELECT` statement saved with `CREATE VIEW`. Use clear, prefixed names like `v_` or

`vw_` so anyone reading the schema knows they are looking at a view, not a physical table.

Using Views in Queries

When you create a view, the database stores **just the definition** of that view - the `SELECT` statement that builds it - not a separate copy of the data. Because of that, a view behaves like a virtual table: you can write a query that reads from the view exactly the same way you read from a real table.

```sql SELECT * FROM TopCustomers; ```

The line above asks the database to run the `SELECT` that defines `TopCustomers`, then return the rows it produces. Nothing special is required; the syntax is identical to querying a normal table.

> **Definition:** A *view* is a saved query that can be referenced in other SQL statements as if it were a table.

Why treating a view like a table is useful

Think of a view as a **recipe card** in a kitchen. The card tells you how to combine ingredients (rows from underlying tables) to make a dish (the result set). Whenever you need that dish, you just follow the card - you don't have to rewrite the whole recipe each time. In SQL, the view's definition is the recipe, and the query that reads the view is the act of cooking.

This saves you from repeating complex joins, filters, or calculations. If the logic ever changes (for example, you decide to include a new column), you only edit the view definition; every query that uses the view automatically gets the updated result.

Adding other SQL features to a view query

A view can be combined with **all the usual SQL clauses**: `WHERE`, `GROUP BY`, `ORDER BY`, `JOIN`, and even other views. The database first expands the view into its underlying `SELECT`, then applies the outer query's clauses. Let's see step■by■step.

1. **Simple filter** - Suppose `TopCustomers` shows every customer who spent more than $10,000 last year.

```sql SELECT customer_id, total_spent FROM TopCustomers WHERE total_spent > 15000; ```

The outer `WHERE` clause narrows the already■filtered list from the view.

2. **Aggregation** - You can group the view's rows just like a normal table.

```sql SELECT region, COUNT(*) AS num_customers FROM TopCustomers GROUP BY region; ```

Here the view supplies the rows, and `GROUP BY` counts how many top customers live in each region.

3. **Joining with another table** - Views can be part of a larger join.

```sql SELECT t.customer_id, t.total_spent, a.account_manager FROM TopCustomers t JOIN AccountManagers a ON t.account_manager_id = a.manager_id; ```

The alias `t` lets us treat the view as a table and join it to `AccountManagers`.

4. **Using a view inside another view** - Views can be stacked.

```sql CREATE VIEW HighValueRegions AS SELECT region, SUM(total_spent) AS region_sales FROM TopCustomers GROUP BY region HAVING SUM(total_spent) > 500000; ```

`HighValueRegions` builds on `TopCustomers`, showing only regions whose top customers together exceed half a million dollars in sales.

> **Note:** The database resolves nested views by expanding each definition in turn, so deep nesting can affect performance. Keep view hierarchies reasonably shallow.

When the view behaves like a table, but isn't

Even though you write `SELECT * FROM ViewName`, the view does **not** store its own rows. Every time you query it, the database executes the underlying `SELECT`. This means:

- **Up■to■date data:** Changes in the base tables are instantly reflected in the view. - **Potential cost:** If the view's definition is complex, the query may be slower than a simple table scan. Indexes on the underlying tables still help, but the view itself cannot have its own indexes (except for materialized views, which are a special case not covered here).

> **Common mistake:** Assuming a view is a cached copy of data. Remember, a standard view is just a saved query; it does not improve speed by itself.

Quick checklist for using a view in a query

1. Write the view name in the `FROM` clause just like a table name. 2. Add any additional `WHERE`, `GROUP BY`, `ORDER BY`, or `JOIN` clauses you need. 3. If you need only a subset of columns, list them after `SELECT` - the view may contain many columns you don't use. 4. Test performance; if the view is very heavy, consider simplifying it or using a materialized view.

> **Remember:** Treat a view as a reusable piece of SQL logic. It lets you write cleaner queries, keep business rules in one place, and ensures consistency across the whole database.

When to Prefer Views Over Repeated Subqueries

When you write a query that needs the same piece of logic many times, you have two main options: repeat the subquery each place it is needed, or create a **view** that contains that subquery once and then refer to the view. Deciding which approach is better is a matter of readability, performance, and security.

First, let's recall what a view is. A view is a virtual table defined by a `SELECT` statement. It does **not** store data itself; whenever you query the view, the database runs the underlying `SELECT` again and presents the result as if it were a real table. Think of a view as a saved recipe: the ingredients and steps are written down once, and you can call the recipe whenever you want to make the dish, instead of rewriting the whole list of ingredients each time.

> **Definition:** A *view* is a named query that can be used like a table. It encapsulates a `SELECT` statement so you can reuse its result without storing duplicate data.

Readability - making the query easier to understand

Imagine you have to list all customers who placed an order in the last 30 days and also calculate each customer's total spending. The subquery that finds "orders in the last 30 days" might be several lines long, involving date functions and joins. If you need that same condition in three
```

different parts of a larger report, writing the subquery three times makes the overall query hard to read. By putting the condition into a view called `RecentOrders`, the main query becomes:

```sql SELECT c.name, ro.total_spent FROM Customers c JOIN RecentOrders ro ON c.id = ro.customer_id; ```

Now anyone reading the query sees a clear, high■level description (`RecentOrders`) instead of a wall of technical details.

> **In daily life:** This is like using a shortcut on your phone's home screen. Instead of opening a long series of apps each time, you tap one icon that launches everything you need.

Performance - letting the database do the work smarter

Databases can sometimes reuse the execution plan of a view, especially if the view is *materialized* (stored on disk). Even for regular (non■materialized) views, the optimizer may recognize that the same subquery appears multiple times and compute it once, then share the result. However, this is not guaranteed. When you repeat a complex subquery, the optimizer must treat each occurrence as a separate piece of work, which can lead to redundant calculations and slower response times.

If you notice that a particular subquery is the bottleneck of many reports, creating a view and then **indexing** the underlying tables appropriately can give the optimizer more clues to speed things up. In some systems you can also create a *materialized view* that stores the result physically, turning the repeated subquery into a simple table lookup.

> **Note:** Views improve performance only when the database can reuse the execution plan or when you use a materialized view. Otherwise, the benefit may be modest.

Security - controlling who can see what

Views can act as a security filter. Suppose a table `EmployeeSalaries` contains a column `salary`. You want most users to see only the employee name and department, never the salary. By creating a view `EmployeePublicInfo` that selects only the allowed columns, you grant users permission to query the view but not the base table. Even if a user writes a subquery that tries to pull `salary`, they will be blocked because they lack rights on the original table.

> **Common mistake:** Assuming that hiding a column in a view automatically encrypts the data. The view only hides the column from users who lack permission on the base table; the data is still stored in clear text.

When to choose a view over repeated subqueries

1. **The logic is used in many places** - a view centralises the code, making maintenance easier. 2. **You want a clearer, self■documenting query** - the view name tells the reader what the subquery does. 3. **You need to restrict access** - grant rights on the view, not on the underlying tables. 4. **Performance testing shows a benefit** - if the optimizer can reuse the plan or you can materialize the view, you gain speed.

Conversely, if the subquery is tiny, used only once, or you need the result to be different each time (e.g., a subquery that depends on a parameter passed to the outer query), writing it directly may be simpler.
```

> **Remember:** Use a view when it makes the query easier to read, can be reused safely, or provides a security layer. Otherwise, a single, well■written subquery is perfectly fine.

Best Practices and Limitations of Views

When you create a view, you are asking the database to store a **named query** - a saved recipe that tells the system how to pull data from one or more tables. Think of a view as a reusable "window" on the data: every time you read from the view, the database runs the underlying query behind the scenes and shows you the result as if it were a regular table.

> **Definition:** A *view* is a virtual table defined by a SELECT statement. It does not hold its own data (unless it is a materialized view); it only shows data that lives in the base tables.

Because a view does not store data itself, there are rules about when you can change (UPDATE, INSERT, DELETE) the rows you see through the view. The database must be able to translate your change back to the original tables unambiguously. If the view joins several tables, groups rows, or uses calculations, the system cannot know which underlying row to modify, so it blocks the update.

> **Common mistake:** Trying to UPDATE a view that contains aggregates (e.g., `SUM`) or `DISTINCT`. The database will reject the statement because it cannot map the aggregated result back to a single source row.

Update Restrictions

1. **Simple projection** - a view that selects columns directly from one table without any calculations can usually be updated. 2. **Joins** - if the view joins two tables, you can only update columns that belong to one side of the join, and only if the join is a one■to■one relationship. 3. **Aggregates, GROUP BY, DISTINCT** - these create summary rows; the database cannot know which original rows to change, so updates are disallowed. 4. **WITH CHECK OPTION** - you can add this clause to a view definition to force the database to reject any INSERT or UPDATE that would cause a row to fall outside the view's filter condition.

Dependency Tracking

When you create a view, the database records a **dependency** on the tables and columns it uses. If later you rename a column, drop a table, or change a column's data type, the view becomes **invalid**. The system will refuse to run the view until you either drop it or modify its definition.

> **Note:** Always check for dependent objects before altering a table. Most DBMSs provide a command like `SHOW DEPENDENCIES` or a system view that lists which views rely on a given table.

Keeping dependencies in mind helps avoid surprise breakages in large projects where many developers share the same schema.

Performance Considerations

A regular view is just a stored query; each time you read from it, the database rewrites the view's SELECT into the outer query and executes the whole thing. This can be convenient but may add overhead if the view is complex and used frequently.

A **materialized view** (sometimes called a snapshot) stores the result set physically, like a cached table. The database updates this cache at defined times (e.g., on commit, nightly, or on demand). Because the data is already computed, reads are fast, but you pay the cost of keeping the cache synchronized.

The performance trade■off can be expressed simply:

$$ \text{Query time (regular view)} \approx \text{time to run underlying SELECT each use} $$ $$ \text{Query time (materialized view)} \approx \text{time to read cached rows} + \text{refresh cost} $$

If the underlying data changes rarely but the view is queried often, a materialized view is usually worth the extra storage and refresh work. If the data changes constantly, the refresh cost may outweigh the read speed benefit, and a regular view is preferable.

Best■Practice Checklist

- **Design views for readability**, not for hiding performance problems. Use them to encapsulate business logic, but still monitor the execution plan of queries that use the view. - **Avoid updates through complex views**; instead, issue DML (Data Manipulation Language) statements directly on the base tables. - **Document dependencies**: keep a simple list of which tables each view depends on, and review it whenever you change the schema. - **Choose materialized views only when** the data is relatively static and the view is a bottleneck. Schedule refreshes at off■peak times to reduce impact on the main workload.

> **Remember:** A view is a convenience tool. It simplifies code and enforces consistent logic, but it does not magically make a slow query fast. Understanding its limitations - especially around updates, dependencies, and performance - lets you use views wisely without surprising side effects.

Figure (described): Shows how a subquery runs before the outer query uses its result.

Figure (described): Illustrates a view acting like a table that shows data from underlying tables.

Key terms

Subquery: A SELECT statement nested inside another SQL statement.

Non■correlated subquery: A subquery that does not reference columns from the outer query and can be executed independently.

Correlated subquery: A subquery that references columns from the outer query and is evaluated once per outer row.

View: A virtual table defined by a stored SELECT statement; it does not hold its own data.

Materialized view: A view that stores its result set physically for faster access.

SELECT list: The column expressions listed after SELECT, where subqueries can appear as computed columns.

HAVING clause: A filter applied after GROUP BY, which can also contain subqueries.

Updatable view: A view that allows INSERT, UPDATE, or DELETE operations that can be mapped back to the base tables.

10. Transactions

This lesson introduces database transactions and explains how they ensure reliable data changes using ACID properties, commit, and rollback. It also covers how transactions handle concurrent access to keep data consistent.

What you will be able to do

- Explain the ACID properties of transactions
- Differentiate between commit and rollback operations
- Describe how transactions manage concurrent access
- Identify situations where transactions are needed in applications

What Is a Transaction?

A transaction is simply a **single unit of work** that groups together one or more actions you want the database to perform. Think of it as a tiny "mini■program" that the database runs from start to finish without interruption.

> **Definition:** A transaction is a set of one or more database operations that are treated as a single, indivisible unit.

Why do we need this idea? Imagine you are buying a ticket for a concert online. The system must do two things:

1. Check that a seat is still free. 2. Reserve that seat for you and charge your credit card.

If the first step succeeds but the second fails (maybe the payment gateway is down), you would end up with a seat marked as taken but no payment recorded - a messy, inconsistent state. A transaction guarantees that either **both** steps happen, or **none** happen.

The "All■or■Nothing" Rule

When a transaction runs, the database follows the **all■or■nothing** rule:

$$ \text{Result of a transaction} = \begin{cases} \text{All operations succeed} & \rightarrow \text{commit} \\ \text{Any operation fails} & \rightarrow \text{rollback} \end{cases} $$

* **Commit** means the database makes all the changes permanent. * **Rollback** means the database undoes everything that happened inside the transaction, returning to the state before the transaction began.

> **Note:** The commit/rollback decision is made automatically by the database when the transaction ends, based on whether any error occurred.

Everyday Analogy

Consider a **bank teller** handling a money transfer between two accounts. The teller must:

* Debit $100 from Alice's account. * Credit $100 to Bob's account.

If the teller forgets to credit Bob after debiting Alice, the bank's records become wrong. In real life, the teller would not finish the transaction until both steps are verified, or they would cancel the

whole process and put the money back into Alice's account. This is exactly what a database transaction does, but it happens in milliseconds and without a human watching.

> **In daily life:** A transaction works like a "double■check" before any change is accepted - similar to a vending machine that only dispenses a snack after it has successfully taken your money.

Building a Transaction Step by Step

1. **Start** - The application tells the database, "I'm beginning a transaction." 2. **Execute operations** - The database runs the individual commands (INSERT, UPDATE, DELETE, etc.). 3. **Check for errors** - If any command fails, the database marks the transaction as "failed." 4. **End** - The application either issues a **COMMIT** (if everything succeeded) or a **ROLLBACK** (if something went wrong).

During steps 2 and 3, the database keeps a hidden copy of the original data. This copy lets it restore the original values if a rollback is needed.

> **Common mistake:** Assuming that simply grouping commands together in code makes them a transaction. You must explicitly start and end a transaction using the database's commands (e.g., `BEGIN`, `COMMIT`, `ROLLBACK`).

Why Transactions Matter

* **Consistency** - The database never ends up in a partially updated state. * **Reliability** - Applications can recover gracefully from failures because the database can undo incomplete work. * **Simplicity for programmers** - Developers can think in terms of logical units (e.g., "place order") rather than worrying about each low■level step.

> **Remember:** A transaction is the database's way of saying, "Treat these actions as one whole thing, and either accept all of them or reject all of them."

By grouping related operations into a transaction, you protect the integrity of your data and make your programs more robust, just as you would double■check a critical real■world task before finalizing it.

ACID Properties

When a database handles a transaction, it promises four guarantees that together are called **ACID**. Think of ACID as a safety checklist that makes sure every "job" the database does finishes correctly, even if something goes wrong.

Atomicity - all■or■nothing A transaction is treated as a single, indivisible unit. Either every step of the transaction succeeds, or none of them do. > **Definition:** *Atomicity* means the transaction is "all■or■nothing".

Imagine you are buying a ticket online. The system must deduct money from your bank account **and** reserve a seat on the train. If the money is taken but the seat is not reserved, you have lost money for nothing. Atomicity guarantees that the system will either do both actions or roll back to the state before you started the purchase.

Why is this useful? It prevents half■finished work that could corrupt data. In a banking system, a transfer from account A to account B must debit A **and** credit B. If only the debit happens, the total amount of money in the bank would disappear.

Consistency - stay in a valid state After a transaction finishes, the database must still obey all the rules (called *constraints*) that define a correct state. > **Note:** *Consistency* ensures that every rule, such as "an account balance cannot be negative", remains true after the transaction.

Think of a puzzle where each piece must fit a specific shape. If you move one piece, you must also move the matching piece so the picture stays correct. In a library database, a rule may say "a book cannot be loaned to more than one person at the same time". When a loan transaction runs, the system checks this rule before allowing the loan and again after the transaction ends, guaranteeing the rule still holds.

Isolation - keep transactions apart When many users run transactions at the same time, each one should behave as if it were the only transaction running. This prevents them from seeing each other's intermediate results and from interfering with one another. > **In daily life:** *Isolation* is like two people writing on the same whiteboard at different times. Each person should see a clean board when they start, and the other person's unfinished scribbles should not affect their work.

Databases offer different *isolation levels* (e.g., Read Committed, Serializable) that trade off performance for how strictly they hide intermediate states. The strongest level, *Serializable*, makes the result exactly the same as if the transactions had been executed one after another.

Durability - never lose committed work Once a transaction reports success, its effects must survive any later failure, such as a power outage or system crash. The database writes the changes to stable storage (like a hard drive) before confirming the commit. > **Remember:** *Durability* means "once it's committed, it stays committed, even after a crash".

A simple analogy is writing a letter and then putting it in a safe deposit box. Even if the office building burns down, the letter is still safe because it was stored in a protected place. In a shopping cart system, once an order is marked as "paid", the record must remain even if the server restarts.

Putting the four guarantees together, we can see why ACID is essential for reliable data processing. The properties can be expressed compactly:

$$ \text{ACID} = \{\text{Atomicity},\ \text{Consistency},\ \text{Isolation},\ \text{Durability}\} $$

If any one of these is missing, the database can produce incorrect or unpredictable results. For example, without atomicity a money transfer could debit one account and forget to credit the other, breaking the *consistency* rule that total money stays constant. Without durability, a committed purchase could disappear after a crash, causing loss of revenue and customer trust.

Common mistake: Assuming that "the database will automatically fix errors". ACID does not correct bad data; it only protects the *process* of changing data. The application must still enforce correct business logic before the transaction starts.

By remembering these four ideas - *all■or■nothing*, *stay valid*, *work alone*, and *stay saved* - you can design and use databases that behave predictably, even under heavy load or unexpected failures.

Commit and Rollback

When a program talks to a database it usually does many steps: read a row, change a value, insert a new record, maybe delete something else. All those steps together are called a **transaction**. A transaction is finished in one of two ways:

* **COMMIT** - tell the database "everything I did is correct, make it permanent." * **ROLLBACK** - tell the database "something went wrong, throw away everything I did and go back to the way it was before I started."

> **Definition:** *COMMIT* is the command that permanently saves all changes made in the current transaction. After a COMMIT, other users can see the new data and the changes cannot be undone by the database.

> **Definition:** *ROLLBACK* is the command that discards all changes made in the current transaction, restoring the database to the state it had before the transaction began.

Why we need both commands

Imagine you are writing a short story on a computer. You type a paragraph, then decide the plot isn't working and delete everything you just wrote. You don't want the unwanted paragraph to stay in the file. In a database, the same idea applies: sometimes the work you have done is good and should be kept (COMMIT); sometimes an error occurs and you must erase the partial work (ROLLBACK).

If the database only had COMMIT, a single mistake could corrupt the whole set of data. If it only had ROLLBACK, you could never make any permanent change. Having both gives you control: you can try a series of operations, check that everything succeeded, and then decide whether to keep them.

How the database does it internally

When a transaction starts, the database creates a **savepoint** - a hidden copy of the data as it existed at that moment. Every change is written to a temporary area called the **transaction log**.

If you issue COMMIT, the database copies the changes from the log to the main data files and marks the savepoint as no longer needed.

If you issue ROLLBACK, the database discards the log entries and restores the data from the savepoint.

Mathematically we can think of it as:

$$ \text{Final State} = \begin{cases} \text{Initial State} + \text{Changes}, & \text{if COMMIT} \\ \text{Initial State}, & \text{if ROLLBACK} \end{cases} $$

When to use COMMIT

1. **All operations succeeded** - every INSERT, UPDATE, DELETE returned "OK". 2. **Business rules are satisfied** - for example, the total amount of a bank transfer matches the sum of debit and credit entries. 3. **You are ready for others to see the result** - after a COMMIT, other users can query the new rows.

When to use ROLLBACK

1. **An error occurs** - a constraint violation (e.g., trying to insert a duplicate primary key) or a runtime exception. 2. **Validation fails** - you check the data after the operations and discover it does not meet required conditions. 3. **User cancels** - a web form has a "Cancel" button that should leave the database unchanged.

Everyday comparison

Think of a **shopping cart** on an online store. Adding items to the cart is like making changes inside a transaction. When you click **"Place Order"**, the system issues a COMMIT: the order is recorded, inventory is reduced, and you receive a confirmation. If you click **"Empty Cart"** or the payment fails, the system performs a ROLLBACK: the cart is cleared and no inventory is deducted. The store never ends up with half■filled orders.

> **In daily life:** COMMIT is like sealing an envelope after you finish writing a letter; ROLLBACK is like tearing the paper back up before you ever send it.

Practical tips

* **Always end a transaction** - forgetting to COMMIT or ROLLBACK leaves the transaction open, which can lock rows and slow down the whole system. * **Check return codes** - most database APIs return a status after each statement; if any status indicates failure, issue a ROLLBACK. * **Use explicit transactions** - even if your language defaults to auto■commit, wrap related statements in `BEGIN TRANSACTION ... COMMIT` (or `ROLLBACK` on error) so the intent is clear.

> **Common mistake:** Assuming that if a program crashes the database will automatically roll back. In many systems the transaction stays open until the server detects the loss of the client and then aborts it, which may take time and still hold locks.

By understanding COMMIT and ROLLBACK, you gain precise control over when changes become part of the permanent data and when they are discarded. This control is the foundation of reliable, safe database programming.

Concurrent Access and Isolation Levels

When several people (or programs) use a database at the same time, each of them runs a **transaction** - a group of operations that must be treated as a single unit. If the database let every transaction see every change immediately, the results could become confusing or even wrong. That is why databases offer **isolation levels**: rules that decide how much a transaction is allowed to see of the work that other transactions are doing.

> **Definition:** *Isolation level* is the degree to which a transaction must be isolated from the effects of concurrent transactions. It determines which intermediate results are visible and which anomalies are prevented.

Why isolation matters

Imagine a library where two friends, Alice and Bob, are borrowing books at the same time. - Alice checks the catalogue and sees that *"Data Structures"* is available, so she decides to borrow it. - Before she finishes the checkout, Bob also checks the catalogue, sees the same book as available, and takes it.

If the library system allowed both to see the same "available" status, they would both think they have the book, leading to a conflict. In a database, similar conflicts can cause **lost updates**, **dirty reads**, or **non■repeatable reads** - all situations where the data a transaction works with is no longer reliable.

Isolation levels let us choose how strictly we want to avoid these problems, trading off safety against performance (more strict isolation usually means slower execution because the system has to lock more data).

The four standard isolation levels

| Level | What it allows | Typical anomalies prevented | |-------|----------------|-----------------------------| | **Read Uncommitted** | Transactions can read data that other transactions have written but not yet committed. | Prevents nothing; dirty reads can happen. | | **Read Committed** | A transaction only sees data that other transactions have committed. | Stops dirty reads; non■repeatable reads may still occur. | | **Repeatable Read** | Once a transaction reads a row, it will see the same value for that row for the rest of the transaction. | Prevents dirty and non■repeatable reads; phantom rows can still appear. | | **Serializable** | Transactions are executed as if they were run one after another, not at the same time. | Prevents all three major anomalies (dirty reads, non■repeatable reads, phantoms). |

> **In daily life:** Think of **Read Uncommitted** as a draft of a document that anyone can peek at, even before the author saves it. **Read Committed** is like only looking at the final printed version. **Repeatable Read** is like locking a page so you always see the same text while you work on it. **Serializable** is like waiting your turn in a single■person line; no one else can start until you finish.

How the database enforces isolation

The most common mechanism is **locking**. When a transaction reads or writes a row, the database may place a *shared lock* (others can also read) or an *exclusive lock* (no one else can read or write) on that row. The lock behavior depends on the chosen isolation level. For example, at **Read Committed** the database releases a shared lock as soon as the read is done, while at **Repeatable Read** it holds the lock until the transaction ends.

Sometimes databases use **multiversion concurrency control (MVCC)** instead of locks. MVCC keeps several versions of a row: a transaction sees the version that was committed before it started. This approach can give the same isolation guarantees without blocking other transactions as much.

Choosing the right level

- **High safety, low concurrency:** Use **Serializable** for financial transfers or inventory updates where any inconsistency could cause loss. - **Balanced safety and speed:** **Read Committed** is the default in many systems (e.g., PostgreSQL, Oracle) and works well for most web applications. - **Maximum performance, accept risk:** **Read Uncommitted** may be used for reporting queries where exact accuracy is not critical.

> **Common mistake:** Assuming that "higher isolation = always better." Higher isolation can dramatically reduce throughput because transactions wait for each other. Pick the lowest level that still protects the data you need.

Quick checklist

1. Identify the possible anomalies for your application (dirty reads, non■repeatable reads, phantoms). 2. Pick the isolation level that blocks those anomalies. 3. Test under realistic concurrent load to see if performance meets your needs. 4. If needed, fine■tune by using explicit locks or by redesigning the transaction logic.

> **Remember:** Isolation levels are a tool to balance **correctness** and **concurrency**. Understanding what each level allows and forbids lets you build databases that are both reliable and fast.

Practical Use Cases and Best Practices

When a program changes data in a database, it usually does more than one step. Imagine a simple online shop where a customer buys a book. The system must:

1. Decrease the stock count for that book. 2. Record the payment in the orders table. 3. Create a shipping record.

If any one of these steps fails and the others stay, the database ends up in an impossible state - the stock might be reduced but the order never appears, or the payment is recorded but the stock is unchanged. A **transaction** groups these steps so they all succeed together or all disappear together.

> **Definition:** A transaction is a set of database operations that are treated as a single logical unit. Either every operation in the set is permanently saved (committed) or none of them are (rolled back).

Real■world scenarios where transactions are essential

Bank transfers - Moving money from Alice's account to Bob's account needs two updates: debit Alice, credit Bob. If the debit succeeds but the credit fails, the total amount of money in the bank changes, which is illegal. A transaction guarantees that both updates happen together.

Ticket reservations - When a user books a seat for a concert, the system must mark the seat as taken and create a reservation record. If two users try to book the same seat at the same time, a transaction with proper isolation prevents both from succeeding; only one will be committed.

Inventory management - A warehouse may receive shipments and ship orders at the same time. Each shipment adds items, each order removes items. Transactions keep the inventory count accurate even when many processes run concurrently.

User registration - Creating a new user often involves inserting a row in a `users` table, adding a default profile, and possibly sending a welcome email. If the profile insert fails after the user row is already saved, the system would have a half■created account. Wrapping the inserts in a transaction avoids this.

Best practices for safe transaction code

1. **Keep transactions short** - The longer a transaction runs, the more it blocks other users and the higher the chance of deadlocks (situations where two transactions wait for each other forever). Do only the necessary reads and writes inside the transaction, and move any heavy computation outside.

2. **Handle errors explicitly** - Always check the result of each database call. If any step returns an error, issue a `ROLLBACK` immediately. Do not assume that later steps will "fix" earlier problems.

3. **Use the appropriate isolation level** - Isolation determines how visible a transaction's intermediate changes are to other transactions. The strictest level, *serializable*, prevents all anomalies but can be slow. For many web applications, *read committed* or *repeatable read* gives a good balance between safety and performance. Choose the level that matches the consistency you need.

4. **Avoid mixing transaction scopes** - Do not start a transaction, call a function that opens its own transaction, and then continue the outer one. This can lead to nested transactions that some databases treat as separate commits, breaking atomicity. Instead, pass the existing transaction object to helper functions so they use the same scope.

5. **Release resources promptly** - After a `COMMIT` or `ROLLBACK`, close the connection or return it to the pool. Leaving idle transactions open consumes memory and can lock rows indefinitely.

> **Note:** Even with a transaction, you must still think about *concurrency* - multiple users trying to modify the same data at the same time. Proper locking or optimistic concurrency control (checking a version number before updating) works together with transactions to keep data correct.

Simple code pattern (pseudo■code)

```sql BEGIN TRANSACTION; -- start the atomic block INSERT INTO orders ...; -- step 1 UPDATE stock SET qty = qty - 1 WHERE book_id = 42; -- step 2 INSERT INTO shipping ...; -- step 3 IF any_error THEN ROLLBACK; -- undo everything ELSE COMMIT; -- make all changes permanent END IF; ```

The `BEGIN TRANSACTION` statement tells the database, "from now on, treat everything as one unit." The `COMMIT` finalizes it, while `ROLLBACK` discards all changes made since the `BEGIN`.

> **Remember:** A transaction is only as safe as the code that controls it. Always test edge cases (network failure, timeout, duplicate requests) to be sure the rollback path works correctly.

Figure (described): Shows the steps of a database transaction from start to commit or rollback.

Figure (described): Visual checklist of the four ACID properties for transactions.

Figure (described): Illustrates how different isolation levels affect visibility of concurrent transactions.
```

Key terms

Transaction: A set of one or more database operations treated as a single, indivisible unit.

Atomicity: All‑or‑nothing execution: either every step succeeds or none do.

Consistency: A transaction moves the database from one valid state to another, preserving all rules.

Isolation: Concurrent transactions do not interfere with each other beyond defined limits.

Durability: Once a transaction is committed, its changes survive system failures.

COMMIT: Command that permanently saves all changes made in the current transaction.

ROLLBACK: Command that discards all changes made in the current transaction, restoring the prior state.

Isolation Level: The degree to which a transaction is isolated from the effects of other concurrent transactions.

11. Indexes and Performance

Indexes are data structures that let databases find rows quickly, making queries run faster. However, they also add overhead for writes and can slow down performance if used improperly.

What you will be able to do

- Explain how an index works internally
- Identify which types of queries benefit from indexes
- Describe situations where indexes can degrade performance
- Choose appropriate columns to index in a given scenario

What Is an Index?

Imagine you have a huge table of student records. Each row stores a student's ID, name, birthdate, grade, and many other columns. If you want to find the row for a student named *Alice*, the database would have to look at every row until it finds the right one. With millions of rows, that would take a long time.

> **Definition:** An *index* is a separate, sorted list of key values (such as names or IDs) together with pointers that tell the database where the matching rows are stored.

Think of the index as a shortcut map. The map does not contain the full data, only the pieces you need to locate the data quickly. When the database receives a query like "find all rows where `name = 'Alice'`", it first checks the index. Because the index is sorted, it can jump directly to the part of the list that contains "Alice" and then follow the pointers to the exact rows in the main table. This avoids scanning every row.

Why does sorting matter? In a sorted list, items are arranged from smallest to largest (or alphabetically). This order lets the database use a technique called *binary search*: it repeatedly halves the search space until it finds the target. Binary search needs only about $\log_2 N$ comparisons for a list of N items, which is dramatically fewer than the N comparisons required by a full table scan.

> **Note:** The pointer stored in the index is not the actual data row; it is a reference (often a physical address or a row identifier) that the database can use to fetch the row instantly.

Everyday comparison

Consider a textbook with an index at the back. The index lists important terms alphabetically and tells you the page numbers where each term appears. If you want to read about "photosynthesis", you don't flip through every page; you look up "photosynthesis" in the index, see that it appears on pages 42, 87, and 123, and turn directly to those pages. The book's index works exactly like a database index: a sorted list of keys (terms) pointing to locations (page numbers) in the main content.

> **In daily life:** A phone's contact list is another familiar example. The contacts are sorted by name, and each entry stores a pointer to the full contact record (phone number, email, etc.). Searching for "Bob" is fast because the list is ordered.

Building an index step by step

1. **Choose a key column** - This is the column whose values will be stored in the index (e.g., `name`). 2. **Sort the key values** - The database arranges the values in order, just like alphabetical order in a book index. 3. **Create pointers** - For each key value, the database records where the corresponding row lives in the main table. 4. **Store the structure** - The index is saved as a separate data structure (often a B■tree) that the database can search efficiently.

Because the index is separate from the main table, it can be kept in memory or on disk in a form that is optimized for quick look■ups. The most common structure is the **B■tree**, which maintains sorted order while allowing fast insertion, deletion, and search operations.

> **Remember:** An index does not replace the original table; it only helps the database find rows faster. The original data remains unchanged.

When an index helps

- **Equality searches** - Finding rows where a column equals a specific value (`WHERE id = 123`). - **Range queries** - Retrieving rows within a range (`WHERE salary BETWEEN 50000 AND 60000`). - **Sorting** - If a query asks for results ordered by a column (`ORDER BY name`), an index on that column can provide the rows already sorted, saving an extra sorting step.

When an index does not help

If a query never uses the indexed column, the database will ignore the index and perform a normal scan. Also, an index on a column with many duplicate values (e.g., a column that is always "yes") provides little benefit because the sorted list does not narrow the search much.

In summary, an index is a sorted, lightweight guide that points the database to the exact rows it needs, much like a book's index points you to the right pages. By reducing the amount of data the system must examine, indexes are a fundamental tool for making database queries fast and efficient.

How Indexes Speed Up Searches

Imagine you have a huge phone book with millions of names. If you want to find the phone number for "Laura Smith", you could start at the first page and read every entry until you see her name. That would take a long time because you might have to look at almost every line - this is what we call a **linear search**, and its cost grows proportionally to the number of entries, written as $O(n)$.

> **Definition:** *Linear search* means checking each item one after another until you find the one you need. Its time grows with the size of the list (n).

Indexes let the database avoid this slow walk■through. An index is a separate, smaller structure that stores the values of one (or more) columns **in sorted order**, together with a pointer to the full row where that value lives. Because the index is sorted, we can use a technique called **binary search** to locate a value much faster.

Binary search step by step

1. **Start in the middle** - Look at the value that sits halfway through the index. 2. **Compare** - If the middle value is exactly the one you want, you are done. 3. **Discard half** - If the middle value is smaller than the target, you know the target must be in the right half; otherwise it must be in the left half. 4. **Repeat** - Take the remaining half and again look at its middle element.

Continue until you either find the target or run out of elements.

Each step cuts the remaining search space in half. After the first step you have $n/2$ items left, after the second $n/4$, after the third $n/8$, and so on. The number of steps needed is the smallest integer k such that $n/2^{k} \le 1$. Solving for k gives

$$k = \lceil \log_{2} n \rceil$$

Thus the time to locate a single value in a sorted index is $O(\log n)$, which grows very slowly even when n is huge. For a table with one million rows, $\log_{2}(1{,}000{,}000) \approx 20$, meaning only about twenty comparisons are needed instead of a million.

> **Note:** *$O(\log n)$* means "order of log n"; the work grows with the logarithm of the number of items, not with the number itself.

Range scans - finding many rows quickly

Often we need not just one exact match but all rows whose values fall inside a range, for example "all orders placed between 2023■01■01 and 2023■01■31". With a linear scan we would still have to read every row and test the date, costing $O(n)$. With an index we can do two binary searches:

1. Find the first entry that is **greater than or equal to** the start of the range. 2. From that point, walk forward through the index entries until we pass the end of the range.

The first step is $O(\log n)$, and the second step touches only the rows that actually belong to the range. If the range contains r rows, the total cost is $O(\log n + r)$. When r is much smaller than n, this is dramatically faster than scanning the whole table.

> **In daily life:** Think of a library's catalogue sorted by author. To find all books by "Jane Austen", you first locate the first "Austen" entry (binary search) and then walk down the list until the author changes. You only look at the Austen books, not every book in the library.

Why the index must be sorted

Binary search works only because the index is ordered. If the index stored values in random order, we would have no way to decide which half to discard, and we would fall back to a linear scan. That is why most database systems build **B■tree** or **B■■tree** indexes: these are balanced tree structures that keep the data sorted while allowing fast insertion and deletion.

> **Common mistake:** Assuming any index speeds up every query. An index helps only when the query's condition uses the indexed column(s) in a way that can exploit the sorted order (equality or range predicates). Searching on a non■indexed column still requires a full table scan.

Putting it together

- **Equality search** (`WHERE col = value`) -> one binary search -> $O(\log n)$. - **Range search** (`WHERE col BETWEEN a AND b`) -> two binary searches + sequential read of matching entries -> $O(\log n + r)$. - The larger the table, the bigger the advantage of $O(\log n)$ over $O(n)$.

By keeping a small, sorted copy of the column values, an index turns a potentially endless walk through the whole table into a quick "divide■and■conquer" hunt. This is the core reason indexes are essential for good query performance.

Types of Indexes

When a database needs to find rows quickly it uses a special data structure called an **index**. Think of an index as a shortcut map that tells the system where to look without scanning every page of the table. Different kinds of shortcuts work better for different kinds of searches, just like different tools are best for different jobs around the house.

B■tree indexes

A **B■tree** (short for *balanced tree*) is the most common index type. It stores the indexed column values in a sorted order inside a tree■like hierarchy. Each node of the tree holds a small range of values and pointers to child nodes, so the tree stays *balanced* - all leaf nodes are at the same depth.

Because the values are ordered, the database can perform a **binary search**: it repeatedly halves the range it looks at until it reaches the exact leaf that contains the target value. This gives a search cost of $$O(\log N)$$ where *N* is the number of rows, meaning the time grows only slowly as the table gets larger.

> **Definition:** A B■tree index keeps keys in sorted order and uses a balanced tree structure to locate rows in logarithmic time.

When to use it - Columns that are searched with range conditions (`>`, `<`, `BETWEEN`) or with equality (`=`). - Queries that need the results already sorted, because the index order can be reused. - Tables that receive a mix of reads and writes; B■trees handle inserts and deletes reasonably well.

Everyday comparison Imagine a phone book sorted alphabetically. To find "Smith, John" you open to the middle, see if the name is before or after, then repeat. You never read every entry - just a few steps down the book. That is exactly how a B■tree works.

Hash indexes

A **hash index** stores a *hash value* for each indexed key. A hash function takes the original value (e.g., a customer ID) and converts it into a fixed■size number that determines where the row's pointer will be placed in a hash table.

When you look up a row with an equality condition (`WHERE id = 12345`), the same hash function is applied to the search value, and the database jumps directly to the bucket that should contain the pointer. This gives an average search cost of $$O(1)$$ - constant time - because it does not need to walk through a tree.

> **Note:** Hash indexes only help with exact matches. They cannot be used for range queries because the hash function scrambles the order of the original values.

When to use it - Columns that are queried almost always with `=` (e.g., primary keys, unique identifiers). - Very large tables where the constant■time lookup saves a lot of CPU. - Workloads that are read■heavy and have few updates, because each insert requires recomputing the hash and may cause bucket splits.

Everyday comparison Think of a set of lockers numbered 0■99. To store a book, you compute a simple rule (e.g., sum of the digits of the book's ISBN) and put the book in the locker that matches the result. To retrieve the book you apply the same rule and go straight to that locker. No

need to search through other lockers.

Bitmap indexes

A **bitmap index** represents the presence of each distinct value of a column with a bit (0 or 1) for every row. If a column has only a few possible values (low cardinality), the index builds one bitmap per value. For example, a column `gender` with values 'M' and 'F' gets two bitmaps: the first has a 1 for every male row, the second for every female row.

When a query asks for rows that match a value, the database simply reads the corresponding bitmap and knows exactly which rows qualify. For multiple conditions, bitmaps can be combined with fast bitwise operations (`AND`, `OR`).

> **In daily life:** Imagine a spreadsheet where each row is a person and each column is a yes/no question. A bitmap index is like a column of check■marks that instantly tells you who answered "yes".

When to use it - Columns with very few distinct values (e.g., gender, status flags, boolean fields). - Data warehouses where queries often involve many such low■cardinality columns combined together. - Read■only or rarely updated tables, because changing a row requires updating several bitmaps, which can be costly.

Common mistake: Using a bitmap index on a high■cardinality column (like a unique email address) creates huge bitmaps that waste space and slow down updates.

Putting it together

| Index type | Best for | Typical query pattern | Update cost |
|---|---|---|---| | B■tree | Ordered or range searches | `=`, `<`, `BETWEEN`, `ORDER BY` | Moderate | | Hash | Exact■match lookups only | `=` on key columns | Low for reads, higher for inserts | | Bitmap | Low■cardinality columns in analytics | Multiple `=` combined with `AND/OR` | High for frequent writes |

> **Remember:** Choose the index that matches the way you ask questions of the data. A B■tree is the all■purpose tool, a hash is the fast■draw for exact keys, and a bitmap shines when you need to combine many simple yes/no filters.

When Indexes Slow Down Writes

When you add a new row to a table, change a value that already exists, or remove a row, the database does more than just touch the main data file. It also has to keep every index that refers to that table in sync. Think of an index as a separate, sorted list that points back to the rows in the table. Whenever the underlying data changes, the list must be updated so that later searches still find the right rows quickly.

INSERT - putting a new book on the shelf

Imagine a library that keeps a catalogue (the index) of all books sorted by title. When a new book arrives (an **INSERT** operation), the librarian must:

1. Write the book's record into the main collection. 2. Find the correct place in the catalogue where the new title belongs. 3. Insert a new entry that points to the book's location.

The extra work is the second and third steps. In a database, the index is usually stored as a balanced tree (often a B■tree). Adding a new entry means navigating the tree from the root to the leaf, possibly splitting a leaf node, and then updating parent nodes. Each split may cause further splits up the tree, so a single INSERT can touch several pages on disk.

UPDATE - moving a book to a different shelf

An **UPDATE** changes the value of one or more columns. If the changed column is part of an index, the database must treat the operation like a delete■then■insert:

1. Locate the old index entry that points to the row. 2. Remove that entry from the index structure. 3. Insert a new entry with the updated key value.

If the column is not indexed, the index does not need to change. This is why you often hear the advice "only index columns you search on, not columns you change a lot."

DELETE - taking a book off the shelf

When a row is removed (**DELETE**), the database must also remove every index entry that points to that row. The steps are:

1. Find the row in the main table and mark it as deleted (or physically remove it). 2. Walk the index tree to the leaf that contains the entry for that row. 3. Delete the entry and, if the leaf becomes too empty, merge it with a sibling node and adjust parent nodes.

Again, several pages of the index may be rewritten.

> **Note:** Each of these extra steps requires reading and writing additional pages on disk, which is much slower than working with memory alone. The more indexes a table has, the more work the database must do for every write.

Why the overhead matters

Disk I/O (input/output) is the slowest part of most database operations. When a write touches multiple index pages, the database may need to:

- **Read** the relevant index pages into memory. - **Modify** them (add, delete, or split nodes). - **Write** the changed pages back to disk.

If a table has three indexes, a single INSERT might cause three separate tree traversals and up to three page writes. The total time can be several times longer than inserting the row alone.

> **In daily life:** Think of updating a phone's contact list that is also synced to several cloud services. Changing a name means the phone updates its local list **and** sends the change to each cloud service. The more services you use, the longer the update takes.

Balancing reads and writes

Because indexes speed up **reads** (searches) but slow down **writes** (INSERT, UPDATE, DELETE), database designers must choose a sensible set of indexes. A common rule of thumb is:

- Keep indexes that are used frequently in queries. - Remove indexes that are rarely used or that involve columns that change often.

> **Remember:** Adding an index is like adding a shortcut for finding data, but it also creates extra work every time the data changes. Choose shortcuts that you will actually use.

Choosing Good Indexes

When you decide which columns should have an index, think of the index as a shortcut that the database can use to find rows quickly. The shortcut is only helpful if you often need to walk down that path.

> **Definition:** *Selectivity* is a measure of how many different values a column has compared to the total number of rows. A column with high selectivity returns only a few rows for each value, while a column with low selectivity returns many rows for each value.

A column with high selectivity makes a good candidate for an index because the shortcut points to a small set of rows, so the database saves time. For example, a column `email` in a user table usually contains a unique address for each user, so its selectivity is close to 1 (every value is different). An index on `email` lets the database jump straight to the single row you need.

In contrast, a column like `country` often has only a handful of distinct values (e.g., "USA", "Canada", "UK") even though the table may contain millions of rows. Its selectivity is low, and an index on `country` would point to a huge list of rows, giving little speed■up for most queries.

How to spot good columns

1. **Look at your most common queries.** Write down the `WHERE`, `JOIN`, `ORDER BY`, and `GROUP BY` clauses you use most often. The columns that appear there are the ones the database checks repeatedly, so they are the first places to consider for indexing.

2. **Check the selectivity.** Estimate the fraction of rows returned by a condition. A simple rule of thumb: if a condition returns less than about 5 % of the rows, the column is usually selective enough to benefit from an index.

$$\text{Selectivity} = \frac{\text{Number of distinct values}}{\text{Total rows}}$$

3. **Consider the query pattern.** - **Equality searches** (`WHERE col = value`) profit most from a single■column index. - **Range searches** (`WHERE col > value` or `BETWEEN`) also use indexes, but the index must be ordered, so a B■tree index works well. - **Composite queries** that filter on several columns at once can use a *composite* (multi■column) index, but the order of columns in that index matters: the most selective column should come first.

4. **Think about write frequency.** Every time a row is inserted, updated, or deleted, the database must also update any indexes on the affected columns. If a table is written to many times per second, adding many indexes can become a bottleneck.

> **Note:** For tables with heavy write activity, keep the number of indexes low and focus on the columns that are truly essential for read performance.

Everyday comparison

Imagine a library where books are arranged only by title on the shelves. If you often look for books by the author's name, you would ask the librarian to create a separate list sorted by author. That list speeds up author searches (high selectivity) but takes extra effort to keep up to date whenever a new book arrives (writes). If most visitors only search by title, the extra author list would be unnecessary work.

Practical checklist

- ■ Identify the top 3-5 queries that run most often. - ■ For each query, note the columns used in `WHERE`, `JOIN`, `ORDER BY`, `GROUP BY`. - ■ Calculate or estimate selectivity; prioritize columns with selectivity > 0.05. - ■ Create single■column indexes for high■selectivity equality searches. - ■ Build composite indexes when a query filters on multiple columns, placing the most selective column first. - ■ Review the write load: if inserts/updates are frequent, limit indexes to those that give the biggest read benefit.

> **Remember:** An index is a trade■off - fast reads versus slower writes. Choose indexes that give the biggest speed■up for the queries you run most, and avoid indexing columns that are rarely used or have very few distinct values. This balanced approach keeps your database responsive without paying too high a cost on data modifications.

Index Maintenance and Monitoring

Imagine a bookshelf that you keep adding new books to every day. Over time the books get shuffled, some shelves become crowded, and a few books are left in the wrong place. If you keep looking for a title without ever straightening the shelves, finding a book will take longer and longer. An **index** in a database works the same way as the catalogue of that bookshelf. When data changes - rows are inserted, updated, or deleted - the catalogue can become messy. **Index maintenance** is the regular cleaning■up that keeps the catalogue tidy so searches stay fast.

> **Definition:** *Index maintenance* is the set of actions (checking, reorganising, rebuilding, or dropping indexes) that keep index structures efficient as the underlying data changes.

Why we need to check index usage

Not every index is useful all the time. Some may be consulted by many queries, while others sit idle and just consume space. Most database systems provide commands that tell you how often an index is used.

* In **SQL Server**, the dynamic■management view `sys.dm_db_index_usage_stats` records counts of seeks, scans, lookups, and updates for each index. * In **PostgreSQL**, the function `pg_stat_user_indexes` shows similar statistics. * In **MySQL**, the `information_schema.STATISTICS` table together with the `performance_schema` can be queried for usage numbers.

Running a simple query like the one below (SQL Server example) gives you a quick picture:

```sql SELECT OBJECT_NAME(i.object_id) AS TableName, i.name AS IndexName, s.user_seeks, s.user_scans, s.user_updates FROM sys.indexes i JOIN sys.dm_db_index_usage_stats s ON i.object_id = s.object_id AND i.index_id = s.index_id WHERE OBJECTPROPERTY(i.object_id,'IsUserTable') = 1; ```

If `user_seeks` and `user_scans` are near zero while `user_updates` is high, the index is rarely helping queries but is being touched every time data changes. That is a sign you might consider dropping it.

Spotting fragmentation

When rows are added or removed, the physical pages that store index entries can become **fragmented** - meaning the logical order of keys no longer matches the physical order on disk.
```

Fragmentation forces the database engine to jump around the storage media, slowing reads.

* **SQL Server** reports fragmentation with `sys.dm_db_index_physical_stats`. * **PostgreSQL** uses the `pgstattuple` extension, and you can call `pgstattuple('index_name')` to see the *bloat* percentage. * **MySQL** (InnoDB) provides `ANALYZE TABLE` and the `INFORMATION_SCHEMA.INNODB_INDEX_STATS` view.

A typical SQL Server command to see fragmentation looks like:

```sql SELECT index_id, avg_fragmentation_in_percent FROM sys.dm_db_index_physical_stats (DB_ID(), OBJECT_ID('dbo.Orders'), NULL, NULL, 'LIMITED') WHERE index_id > 0; ```

If `avg_fragmentation_in_percent` is above **30 %**, the index is considered heavily fragmented and should be rebuilt; between **10 %** and **30 %**, a simple **reorganise** may be enough.

> **Note:** *Reorganise* is a lightweight, online operation that defragments the leaf pages. *Rebuild* creates a fresh copy of the index, optionally changing its fill factor, and can be done offline or online depending on the engine.

When to rebuild or drop an index

1. **High fragmentation** (>= 30 %) -> **Rebuild** the index. 2. **Moderate fragmentation** (10 %-30 %) -> **Reorganise** the index. 3. **Very low usage** (e.g., < 5 % of total query plans) and **high update cost** -> **Drop** the index.

Rebuilding can be scheduled during low■traffic windows because it may lock the table (unless you use an online rebuild feature). Monitoring tools such as **SQL Server Agent**, **pg_cron**, or **MySQL Event Scheduler** can automate these checks.

> **In daily life:** Think of rebuilding an index like re■printing a phone book after many pages have been torn out and added. It takes time, but the new book is compact and fast to search.

Keeping an eye on the health of indexes

Most DBAs set up a regular **maintenance plan**:

* **Weekly**: Run usage■stats queries, flag unused indexes. * **Monthly**: Check fragmentation levels, decide between reorganise or rebuild. * **Quarterly**: Review the overall index strategy - add new indexes for emerging query patterns, drop those that have become obsolete.

Many monitoring dashboards (e.g., **SQL Server Management Studio** reports, **pgAdmin** dashboards, or **Percona Monitoring and Management** for MySQL) visualise these metrics, making it easier to spot trends before performance degrades.

> **Remember:** An index that is never used is just wasted space and extra work for every data change. Regularly checking usage and fragmentation lets you keep only the indexes that truly help, and keep them in good shape.

Common Pitfalls and Best Practices

Indexes are like shortcuts in a huge library. Imagine you have a book with thousands of pages, and you want to find every page that mentions the word **"dragon."** If you read the whole book each time, it will take a long time. If the book has an index at the back that lists every page where "dragon" appears, you can jump straight to those pages.
```

The same idea works for databases, but shortcuts can also become obstacles if they are built the wrong way. Below are the most common pitfalls students run into, followed by simple rules to avoid them.

> **Common mistake:** **Over■indexing** - creating many indexes on a table. Every index must be kept up■to■date whenever the table changes (INSERT, UPDATE, DELETE). The database has to write extra data for each index, so write operations become slower. If you have ten indexes on a table that is updated many times per second, the extra work can overwhelm the system, just like a library that prints a new index page after every single word is added to a book.

> **Common mistake:** **Indexing low■cardinality columns** - columns that contain only a few distinct values. Cardinality means "how many different values a column can have." A column that stores only "yes" or "no" has a cardinality of 2, which is very low. An index on such a column is usually useless because the database still has to scan many rows to find the ones you need, much like an index that lists every page where the word "the" appears - the list is so long that it doesn't help you find anything quickly.

> **Remember:** **Keep statistics fresh**. Statistics are numbers the database collects about how data is distributed (for example, how many rows have each value in a column). The query optimizer uses these numbers to decide whether to use an index. If statistics are out■of■date, the optimizer may choose a bad plan, ignoring a useful index or using a useless one. Think of statistics as a map that tells you which shortcuts are still valid; an old map can lead you down dead■ends.

> **Note:** **Choose indexes that match the query pattern**. When you write a SELECT statement, look at the columns used in the **WHERE** clause, the **JOIN** conditions, and the **ORDER BY** clause. An index that starts with those columns can speed up the query. For example, if most queries filter by `city` and then by `zipcode`, a composite index on `(city, zipcode)` is helpful. The order matters: an index on `(zipcode, city)` would not help a query that only filters by `city`. This is similar to a phone book that is sorted by last name first; you can quickly find someone if you know the last name, but not if you only know the first name.

> **In daily life:** **Avoid "index bloat."** Just as you would not keep a separate index page for every single word in a short story, do not create indexes on columns that are rarely searched or that change frequently. Periodically review your indexes: drop those that are never used (you can see usage statistics in most database tools) and keep the ones that give a clear performance benefit.

Best practice checklist

1. **Identify hot queries** - the queries that run most often or take the most time. 2. **Create indexes that start with the most selective columns** (the columns that filter out the most rows). 3. **Limit the number of indexes** - aim for a small, well■chosen set. 4. **Schedule regular statistics updates** (often automatically after a certain amount of data change). 5. **Monitor index usage** and remove unused indexes.

By following these simple steps, you keep your database fast for both reading and writing, just as a well■organized library stays easy to use for both visitors and staff.

Figure (described): Shows how a B■tree index stores keys and pointers in a balanced hierarchy.

Figure (described): Comparing linear search of a phone book with an indexed lookup.

Figure (described): How an index can become fragmented after many inserts and deletions.

Key terms

Index: A separate, sorted list of key values with pointers to the rows where those values are stored.

Linear search: Checking each item one by one until the desired one is found; its cost grows with the number of items (O(n)).

Binary search: A search method that repeatedly halves a sorted list to locate a value, giving logarithmic time (O(log n)).

B■tree: A balanced tree data structure used for most database indexes; it keeps data sorted and allows fast search, insert, and delete.

Selectivity: A measure of how many distinct values a column has relative to the total rows; high selectivity means few rows match each value.

Index maintenance: Activities such as reorganising or rebuilding indexes to keep them efficient as data changes.

Over■indexing: Creating too many indexes, which harms write performance because each change must update all indexes.

INSERT: An operation that adds a new row to a table and also updates any relevant indexes.

12. Database Design in Practice

In this final lesson students will design a small, real■world database from start to finish, applying all concepts learned earlier. They will see how each step - requirements gathering, conceptual modeling, logical design, and physical implementation - fits together in practice.

What you will be able to do

- Identify the real■world problem and list the data requirements
- Create an Entity■Relationship diagram for the problem
- Convert the ER diagram into relational tables with primary and foreign keys
- Write the SQL statements to create the tables and insert sample data
- Explain how the design supports typical queries and future changes

1. Define the Problem and Gather Requirements

When you start a database project, the very first step is to **understand the real■world problem** you want the computer to help solve. Think of it like planning a road trip: before you drive, you decide where you are going, who is coming, what you need to bring, and which roads are best. In the same way, a database designer must decide *what* the system will store, *who* will use it, and *how* the information will be used.

1. Pick a simple, familiar scenario Choose a situation that you (or your classmates) already know well. A classic example is a **school library** that keeps track of books, students, and loans. Because the domain is small, you can see all the parts clearly without getting lost in complexity.

2. Ask the right questions Treat the problem like an interview. Write down questions for each group of people who will interact with the system (called *stakeholders*). Stakeholders might be librarians, students, or teachers. Typical questions include:

- **What information do we need to record?** Example: For each book we need its title, author, ISBN, and whether it is currently on loan.

- **How do people use the information?** Example: A student wants to find out which books are available, and a librarian wants to see which books are overdue.

- **What rules must the system follow?** Example: A student may borrow at most three books at a time, and a loan cannot last more than 21 days.

Write the answers in plain sentences; avoid technical jargon at this stage. This list becomes your **requirements** - a clear description of what the database must do.

3. List all needed data items From the answers, extract every piece of information that will be stored. These pieces are called **data items** or **attributes**. For the library scenario you might end up with:

| Entity (thing) | Data items (attributes) | |----------------|--------------------------| | Book | BookID, Title, Author, ISBN, PublicationYear, ShelfLocation | | Student | StudentID, Name, Email, Phone, YearOfStudy | | Loan | LoanID, BookID, StudentID, LoanDate, DueDate, ReturnDate |

Notice that some items, like *BookID* and *StudentID*, are special because they uniquely identify a single record. We will call them **keys** later, but for now just note that every entity needs at least one identifier.

4. Write a short problem statement Summarize the whole picture in one paragraph. This statement guides everyone who later reads the design.

> **Problem statement:** The library database must store information about books, students, and the loans that link them. It must allow librarians to add new books, register students, record when a book is borrowed or returned, and generate reports such as "books currently on loan" and "students with overdue books." The system must enforce the rule that a student cannot have more than three active loans and that loans cannot exceed 21 days.

5. Check completeness Review your list of data items and questions. Ask yourself:

- Have I captured every piece of information mentioned in the requirements? - Is there any data that seems unnecessary? - Do the rules make sense and cover all possible situations?

If something is missing, go back to the stakeholder questions and add it. This iterative checking prevents costly changes later.

> **Remember:** A well■defined problem and a complete set of requirements are the foundation of a good database. Skipping this step is like building a house without a blueprint - you may end up with rooms that don't fit together.

> **Common mistake:** Trying to design tables before you know exactly what data you need. This often leads to extra columns, duplicated information, or missing relationships that are hard to fix later.

> **In daily life:** When you shop online, the website first asks for your address, payment method, and items you want. Those questions are the "requirements" that let the system store the right data and process your order correctly.

Once you have a clear problem statement, a complete list of data items, and a set of business rules, you are ready to move on to the next step: drawing an Entity■Relationship (ER) diagram that visualises how these pieces fit together.

2. Sketch an Entity■Relationship (ER) Diagram

When we start to design a database we first need a picture of the "things" we will store and how those things are linked. That picture is called an **Entity■Relationship (ER) diagram**. Think of it as a map of a city: the **entities** are the landmarks (schools, students, books), the **attributes** are the details written on each landmark's sign■board (name, address, price), and the **relationships** are the roads that connect the landmarks (a student *enrolls in* a course, a book *belongs to* a category).

> **Definition:** An **entity** is anything we want to keep information about. > **Note:** An **attribute** is a property or characteristic of an entity. > **In daily life:** A **relationship** tells us how two entities are associated, just like "friendship" connects two people on a social network.

Step 1 - List the entities Read the problem statement carefully. Ask: *What nouns appear?* Each noun that represents a real object or concept usually becomes an entity. Example problem: "A library wants to keep track of books, members, and loans." - **Book** - each physical book in

the collection. - **Member** - a person who can borrow books. - **Loan** - the act of a member borrowing a book.

Step 2 - Identify attributes for each entity For every entity ask: *What facts do we need to know about it?* Write them as bullet points. - **Book**: `ISBN` (unique identifier), `Title`, `Author`, `Publisher`, `YearPublished`. - **Member**: `MemberID` (unique), `FirstName`, `LastName`, `Email`, `Phone`. - **Loan**: `LoanID` (unique), `LoanDate`, `DueDate`, `ReturnDate`.

The attributes that uniquely identify an entity are called **primary keys**. They are essential because they let us pick out one exact record from many.

Step 3 - Determine relationships and cardinalities Now ask how the entities interact. Use verbs from the description. - A **Member** *borrows* a **Book**. - A **Book** can be borrowed many times over its life, but each **Loan** involves exactly one **Member** and one **Book**.

Cardinality tells us the "how many" part of a relationship:

- **One■to■many (1:M)** - One member can have many loans, but each loan belongs to one member. - **Many■to■many (M:N)** - If the problem allowed a loan to contain several books at once, we would have a many■to■many link between Member and Book. In our simple design we avoid M:N by introducing the **Loan** entity, which resolves the many■to■many into two one■to■many relationships.

Step 4 - Draw the diagram with standard symbols Use the following conventions (the most common in textbooks):

- **Rectangle** = entity. - **Oval** = attribute; underline the primary key attribute. - **Diamond** = relationship; label it with a verb phrase. - **Lines** connect entities to relationships; a small "crow's foot" on a line indicates "many".

Below is a simple ER diagram written in **Mermaid** syntax, which many editors can render automatically:

```mermaid erDiagram BOOK { string ISBN PK string Title string Author string Publisher int YearPublished } MEMBER { int MemberID PK string FirstName string LastName string Email string Phone } LOAN { int LoanID PK date LoanDate date DueDate date ReturnDate }

MEMBER ||--o{ LOAN : makes BOOK ||--o{ LOAN : includes ```

In the diagram:

- `PK` marks the primary key attribute (underlined in a hand■drawn picture). - `||` means "one", `o{` means "many". - The relationship names "makes" and "includes" describe the verb phrases.

### Why this matters Creating the ER diagram forces you to think about the data before you write any SQL. It catches missing information early (e.g., we realized we need a `ReturnDate` attribute to know when a book was given back). It also shows you where to place foreign keys later: the **Loan** table will contain `MemberID` and `ISBN` as references to the other two tables.

> **Remember:** An ER diagram is a communication tool. Keep it simple, label everything clearly, and verify it with the person who gave you the requirements. If the diagram matches their mental picture, you are ready to move on to choosing keys and normalizing the design.

## 3. Choose Keys and Normalize the Design
```

When we start turning an ER diagram into a real database, the first thing we must decide is **which pieces of data will uniquely identify each record**. These unique identifiers are called **keys**.

> **Definition:** A **primary key** is a column (or a small set of columns) whose value is different for every row in a table. No two rows can share the same primary■key value, and a primary key can never be empty (NULL).

Why do we need a primary key? Imagine a classroom where every student writes their name on a piece of paper. If two students happen to have the same name, you could not tell their papers apart. Giving each student a unique student■ID solves the problem - the ID works exactly like a primary key.

Choosing a Primary Key

1. **Look at the entity's natural identifier.** In a *Student* entity, a student number assigned by the school is a natural choice because it is already unique.

2. **If no natural identifier exists, create a surrogate key.** A *surrogate key* is an artificial column, often called `ID`, that the database generates automatically (for example, an auto■increment integer).

3. **Make sure the key is minimal.** The key should contain the smallest number of columns that still guarantee uniqueness. Adding extra columns does not help and makes the design larger.

Defining Foreign Keys

When two tables are related, we need a way to link a row in one table to the corresponding row in another. This link is made with a **foreign key**.

> **Definition:** A **foreign key** is a column (or set of columns) in one table that refers to the primary key of another table. It creates a relationship between the two tables and enforces referential integrity - the database will not allow a foreign■key value that does not exist in the referenced primary■key column.

Think of a foreign key like a mailing address. Your house (the child table) has an address that points to a street (the parent table). If the street does not exist, the address would be meaningless.

Normalization: Removing Redundancy

Even with keys in place, a table can still store the same piece of information many times. This duplication wastes space and can cause inconsistencies when one copy is updated but another is not. **Normalization** is a step■by■step process that reorganises tables to eliminate such redundancy.

First Normal Form (1NF)

A table is in 1NF when every column holds **atomic** values - that is, each cell contains a single, indivisible piece of data.

> **Note:** If a column stores a list (e.g., "phone numbers: 555■1234, 555■5678"), split it into separate rows or a separate table.

Second Normal Form (2NF)

A table is in 2NF if it is already in 1NF **and** every non■key column depends on the whole primary key, not just part of it. This matters only for tables whose primary key is composite (made of more than one column).

For example, in a *CourseEnrollment* table with primary key `(StudentID, CourseID)`, the column `StudentName` depends only on `StudentID`. To satisfy 2NF, move `StudentName` to a separate *Student* table.

Third Normal Form (3NF)

A table is in 3NF if it is in 2NF **and** no non■key column depends on another non■key column. In other words, there should be no **transitive dependency**.

Consider a *Product* table with columns `ProductID`, `SupplierID`, and `SupplierName`. `SupplierName` depends on `SupplierID`, not directly on `ProductID`. To achieve 3NF, create a separate *Supplier* table with `SupplierID` as primary key and move `SupplierName` there.

Putting It All Together

After you have:

1. Chosen a primary key for each table, 2. Added foreign keys to represent relationships, and 3. Normalised the tables to at least 3NF,

the design will be free of unnecessary duplication and ready for reliable data entry.

$$ \text{Redundancy} = \text{Total data stored} - \text{Data needed for unique information} $$

If the redundancy value is zero, you have achieved a well■normalised design.

> **Remember:** Primary keys uniquely identify rows, foreign keys link tables, and normalization removes repeated data. Skipping any of these steps can lead to confusing, error■prone databases.

4. Translate the ER Diagram into Relational Tables

When you have finished drawing your ER diagram, the next step is to turn the picture into real tables that a database can store. Think of the ER diagram as a blueprint of a house: the blueprint shows rooms (entities) and doors (relationships). To actually build the house you must order bricks, windows and doors and tell the builder how they fit together. In a database, **bricks** are the columns of a table, **rooms** are the tables themselves, and **doors** are foreign■key columns that link one table to another.

1. Create a table for every entity

Start with the simplest part: each entity becomes its own `CREATE TABLE` statement. Choose a column that uniquely identifies each row - this is the **primary key**. Give each attribute a suitable SQL data type (e.g., `VARCHAR` for text, `INT` for whole numbers, `DATE` for dates).

```sql CREATE TABLE Student ( student_id INT PRIMARY KEY, -- unique identifier first_name VARCHAR(50) NOT NULL, last_name VARCHAR(50) NOT NULL, birth_date DATE, email VARCHAR(100) UNIQUE );

CREATE TABLE Course ( course_id INT PRIMARY KEY, title VARCHAR(100) NOT NULL, credits INT CHECK (credits > 0) ); ```
```

> **Definition:** *Primary key* - a column (or set of columns) whose value is different for every row in the table. It lets the database find a specific record quickly.

2. Represent relationships

Relationships become **foreign keys** - columns that store the primary■key value of another table. The type of relationship (one■to■many, many■to■many, one■to■one) tells us how many foreign■key columns we need.

One■to■many (e.g., each student can enroll in many courses, but each enrollment belongs to one student)

Add a foreign■key column to the "many" side:

```sql CREATE TABLE Enrollment ( enrollment_id INT PRIMARY KEY, student_id INT NOT NULL, course_id INT NOT NULL, enroll_date DATE, grade CHAR(2),

FOREIGN KEY (student_id) REFERENCES Student(student_id), FOREIGN KEY (course_id) REFERENCES Course(course_id) ); ```

#### Many■to■many (e.g., students can have many skills and each skill can belong to many students)

Create a **junction table** that contains two foreign keys, each pointing to one of the related entities. The combination of these keys is usually the primary key of the junction table.

```sql CREATE TABLE Skill ( skill_id INT PRIMARY KEY, name VARCHAR(50) NOT NULL );

CREATE TABLE StudentSkill ( student_id INT NOT NULL, skill_id INT NOT NULL, level VARCHAR(20),

PRIMARY KEY (student_id, skill_id), FOREIGN KEY (student_id) REFERENCES Student(student_id), FOREIGN KEY (skill_id) REFERENCES Skill(skill_id) ); ```

> **Note:** A junction table resolves many■to■many relationships because a relational table can only store one value per column in each row.

### 3. Add constraints for data integrity

Constraints are rules that the database checks automatically. Common ones are:

* `NOT NULL` - the column must have a value. * `UNIQUE` - no two rows can have the same value. * `CHECK` - a condition that must be true (e.g., `credits > 0`). * `FOREIGN KEY` - ensures the value exists in the referenced table.

These constraints keep the data clean, just like a rule "no two students can have the same email address" prevents confusion later.

### 4. Choose appropriate data types

Pick the smallest type that can hold all expected values; this saves space and speeds up queries.

| Concept | Typical SQL type | Example | |---------|------------------|---------| | Whole number (ID) | `INT` or `SMALLINT` | `student_id` | | Short text (name) | `VARCHAR(n)` | `first_name` | | Fixed■length code (grade) | `CHAR(n)` | `grade` | | Date | `DATE` | `enroll_date` |

> **In daily life:** Think of data types like different sized containers - you wouldn't store a gallon of water in a teaspoon. Using the right size avoids overflow and waste.

### 5. Verify the translation

After writing the `CREATE TABLE` statements, run them in your DBMS (e.g., MySQL, PostgreSQL). If any error appears, it usually points to a missing reference or a datatype mismatch. Correct the script, then use `DESCRIBE table_name;` (or `\d table_name` in PostgreSQL) to see the final structure.

> **Common mistake:** Forgetting to declare a foreign key as `NOT NULL` when the relationship is mandatory. This lets orphan rows appear, breaking the logical link between tables.

### 6. Summary of the conversion process

1. **Entity -> Table** with a primary key. 2. **One■to■many relationship -> foreign key on the "many" side.** 3. **Many■to■many relationship -> separate junction table with two foreign keys.** 4. **Add constraints** (`NOT NULL`, `UNIQUE`, `CHECK`, `FOREIGN KEY`). 5. **Choose sensible data types** for each column.

Following these steps turns the abstract ER diagram into concrete, query■ready tables that faithfully represent the real■world problem you are modeling.

## 5. Populate the Tables with Sample Data

When we have finished drawing the ER diagram and turning it into relational tables, the next step is to **fill the tables with some real■looking rows**. These rows are called *sample data* and they let us see how the tables connect to each other before we write any queries.

### Why insert sample data?

Think of a new bookshelf you are building. Before you put all your books on it, you might place a few test books on each shelf to check that the shelves are level and that the books fit. In a database, inserting a few rows works the same way: it shows whether the foreign■key links are correct, whether the data types accept the values, and whether the constraints (like "no two students can have the same ID") are enforced.

### Step■by■step insertion

Below we assume a simple university database with three tables that came from the ER diagram:

| Table | Columns | |-------|---------| | **Students** | `StudentID` (PK), `FirstName`, `LastName`, `Email` | | **Courses** | `CourseID` (PK), `CourseName`, `Credits` | | **Enrollments** | `EnrollmentID` (PK), `StudentID` (FK), `CourseID` (FK), `Grade` |

*PK* = primary key - a column (or set of columns) that uniquely identifies each row. *FK* = foreign key - a column that must match a value of a primary key in another table, creating a relationship.

#### 1. Insert students

```sql INSERT INTO Students (StudentID, FirstName, LastName, Email) VALUES (1001, 'Alice', 'Brown', 'alice.brown@example.edu'), (1002, 'Bob', 'Smith', 'bob.smith@example.edu'), (1003, 'Carol', 'Lee', 'carol.lee@example.edu'); ```
```

> **Definition:** *Primary key* values must be unique; they are like a student's ID card number that never repeats.

2. Insert courses

```sql INSERT INTO Courses (CourseID, CourseName, Credits) VALUES ('CS101', 'Introduction to Computer Science', 4), ('MA201', 'Calculus I', 3), ('EN105', 'English Literature', 2); ```

> **Note:** The `CourseID` is a short code that will be used later in the `Enrollments` table. Think of it as a course's nickname.

3. Insert enrollments

```sql INSERT INTO Enrollments (EnrollmentID, StudentID, CourseID, Grade) VALUES (1, 1001, 'CS101', 'A'), (2, 1001, 'MA201', 'B+'), (3, 1002, 'CS101', 'B-'), (4, 1003, 'EN105', 'A-'), (5, 1003, 'MA201', 'A'); ```

> **In daily life:** The `Enrollments` table works like a class register where each line says "Student X is taking Course Y and got Grade Z". The `StudentID` and `CourseID` columns are *foreign keys* that point back to the `Students` and `Courses` tables.

Checking that the relationships work

After the inserts, you can run a quick query to see the whole picture:

```sql SELECT s.FirstName, s.LastName, c.CourseName, e.Grade FROM Enrollments e JOIN Students s ON e.StudentID = s.StudentID JOIN Courses c ON e.CourseID = c.CourseID ORDER BY s.StudentID; ```

If the query returns rows without error, the foreign■key links are correct. If you try to insert a row with a non■existent `StudentID` or `CourseID`, the database will reject it, protecting the integrity of the data.

> **Common mistake:** Forgetting to insert the parent rows (`Students` or `Courses`) before the child rows (`Enrollments`). The database will raise a *foreign■key violation* because it cannot find the referenced key.

A few extra rows for realism

Adding more data helps test edge cases, such as a student taking many courses or a course with many students.

```sql -- More students INSERT INTO Students (StudentID, FirstName, LastName, Email) VALUES (1004, 'David', 'Kim', 'david.kim@example.edu'), (1005, 'Eva', 'Martinez','eva.martinez@example.edu');

-- More enrollments INSERT INTO Enrollments (EnrollmentID, StudentID, CourseID, Grade) VALUES (6, 1004, 'CS101', 'B'), (7, 1004, 'EN105', 'B+'), (8, 1005, 'MA201', 'A'), (9, 1005, 'CS101', 'A-'); ```

Now the database contains a small but varied set of records that mimic a real university's data. You can experiment with queries that count how many students are in each course, calculate average grades, or find students who have not yet enrolled in any class.

> **Remember:** Sample data is not the final production data, but it is essential for *testing* the design. By inserting realistic rows and watching how the tables interact, you gain confidence that

the schema you built will support the real world use cases you will later load.

## 6. Test the Design with Sample Queries

When we finish drawing our ER diagram and turning it into tables, the next step is to **check that the tables really work**. The easiest way to do that is to write a few simple `SELECT` statements - the SQL command that asks the database for information. If the results look right, we have confidence that the design can answer the questions users will ask.

---

> **Definition:** A **SELECT query** is a command that tells the database which columns (fields) to show, from which tables, and under what conditions (filters).

### 1. Start with a very basic query

Suppose our database is for a small library. One of the most common questions is:

> *"Which books are currently available?"*

In everyday life, this is like looking at a shelf and picking out the books that are not checked out. In SQL we write:

```sql SELECT BookID, Title, Author FROM Books WHERE IsAvailable = TRUE; ```

- `SELECT BookID, Title, Author` tells the database to show three columns. - `FROM Books` says the data comes from the **Books** table. - `WHERE IsAvailable = TRUE` filters the rows so we only see books that are marked as available.

If the result list matches the books you see on the shelf, the `Books` table and the `IsAvailable` attribute are set up correctly.

### 2. Join two tables to answer a slightly harder question

A user might also want to know **who has borrowed each book**. The information lives in two tables:

| Table | Important columns | |-------|-------------------| | **Books** | `BookID`, `Title` | | **Loans** | `LoanID`, `BookID`, `MemberID`, `LoanDate`, `ReturnDate` |

We need to combine (join) these tables on the common column `BookID`. Think of it as matching each book with its loan record, just like matching a name tag to a person in a crowd.

```sql SELECT B.Title, L.MemberID, L.LoanDate FROM Books AS B JOIN Loans AS L ON B.BookID = L.BookID WHERE L.ReturnDate IS NULL; ```

- `JOIN Loans AS L ON B.BookID = L.BookID` links each book (`B`) to its loan (`L`) where the IDs match. - `WHERE L.ReturnDate IS NULL` keeps only rows where the book has not yet been returned (i.e., it is still on loan).

If the query returns a list of titles together with the member who currently holds them, the foreign■key relationship between **Books** and **Loans** works.

### 3. Use an aggregate function to get a summary

Managers often need a quick summary, such as **how many books each member has borrowed**. This uses an **aggregate function** (`COUNT`) that counts rows, and a `GROUP BY`

clause that groups rows by a column.

```sql SELECT MemberID, COUNT(*) AS BooksBorrowed FROM Loans WHERE ReturnDate IS NULL GROUP BY MemberID; ```

- `COUNT(*)` counts all loan rows for each member. - `AS BooksBorrowed` gives the count column a readable name. - `GROUP BY MemberID` tells the database to calculate the count separately for each member.

If the numbers match the library's manual records, the `Loans` table correctly records each borrowing event.

### 4. Test a query that uses a sub■query (a query inside another query)

Sometimes we need information that depends on another query. For example, **find all members who have never borrowed a book**. First we find members who *have* borrowed books, then we exclude them.

```sql SELECT MemberID, Name FROM Members WHERE MemberID NOT IN ( SELECT DISTINCT MemberID FROM Loans ); ```

- The inner query `SELECT DISTINCT MemberID FROM Loans` produces a list of members who appear in the **Loans** table. - `NOT IN` removes those members from the outer result, leaving only those who never appear in **Loans**.

If the result matches the list of new members, the relationship between **Members** and **Loans** is correctly enforced.

### 5. Verify that constraints prevent bad data

A well■designed schema should reject impossible data. Try to insert a loan for a non■existent book:

```sql INSERT INTO Loans (LoanID, BookID, MemberID, LoanDate) VALUES (999, 9999, 1, CURRENT_DATE); ```

Because `BookID` 9999 does not exist in **Books**, the database should raise an error (a **foreign■key violation**). This test confirms that the foreign■key constraint we defined earlier is active.

---

> **Note:** When a query returns *no rows*, it does not always mean something is wrong; it may simply mean the condition you asked for does not exist in the data yet.

By running these kinds of queries - simple selects, joins, aggregates, sub■queries, and constraint checks - you can be sure that the relational schema you built from the ER diagram actually supports the information needs of the users. If any query gives an unexpected result, go back to the relevant table or relationship, adjust the design, and test again. This iterative testing is the final safety net before the database goes live.

## 7. Review and Refine the Design

When you finish building a database, the work is not yet done. Just like a carpenter checks a table for wobbling legs before calling it finished, you should **review** the design and **refine** it. This
```

step helps the database stay fast, flexible, and easy to keep up■to■date.

First, look at the **queries** that will be run most often. If a query repeatedly scans a whole table to find a few rows, the database will feel slow. The usual fix is to add an **index**. An index is a separate data structure that stores the values of one (or more) columns in a sorted order, together with pointers to the rows where those values appear. Think of it like the index at the back of a textbook: instead of flipping through every page to find the word "photosynthesis," you jump straight to the listed page numbers.

> **Note:** Adding an index speeds up look■ups but uses extra storage space and can slow down inserts, updates, and deletes because the index must be maintained.

Next, examine any **optional data** - information that some records have and others do not. In a school database, a student might have a middle name, but many do not. If you force every row to contain a value, you either waste space with empty placeholders or you have to invent a dummy value like "N/A," which can cause confusion later. Instead, declare those columns as **nullable**, meaning they are allowed to contain a special "no value" marker (NULL). This tells the database, "It's okay if this piece of information is missing."

> **Definition:** *NULL* is a special marker that means "unknown or not applicable." It is different from an empty string or the number zero; those are actual values.

After handling indexes and nullable columns, ask yourself whether any **relationships** could be simplified. Sometimes a many■to■many link table (for example, *StudentCourse*) ends up storing extra columns that never change, such as a duplicate copy of the student's name. Keeping only the foreign keys (the IDs that point to the related rows) reduces redundancy and prevents inconsistencies - if a student changes their name, you only need to update one place.

> **Common mistake:** Storing the same piece of information in several tables. This creates *redundancy* and makes it easy to forget to update one copy, leading to contradictory data.

Another refinement is to check the **data types** you chose. If a column stores a year, using a full 4■byte integer is wasteful; a small integer (2 bytes) is enough. Choosing the smallest appropriate type saves disk space and can improve performance because the database can read more rows into memory at once.

Now, run a few **stress tests**: insert a large number of rows, update many of them, and execute the most common queries. Observe how long each operation takes. If a query still feels slow, look at the **execution plan** (the database's step■by■step recipe for the query). The plan will show whether it is using an index or doing a full table scan. Adjust the indexes or rewrite the query until the plan shows an efficient path.

> **Remember:** The design process is iterative. You may need to go back to earlier steps - perhaps adding a new attribute or splitting a table - based on what you learn during testing.

Finally, document any changes you made and the reasons behind them. Good documentation is like a road map for future developers: it explains why an index exists, why a column can be NULL, and what assumptions were made. This makes future maintenance smoother and reduces the chance of accidental mistakes.

In summary, reviewing and refining a database design means:

1. Adding indexes for frequent look■ups. 2. Allowing NULL for truly optional data. 3. Removing redundant columns and keeping relationships simple. 4. Choosing the right data types. 5. Testing with realistic data volumes and checking execution plans. 6. Updating documentation.

By following these steps, you turn a working prototype into a robust, efficient system that can grow with the needs of its users.

Figure (described): Shows entities, attributes and relationships for a simple school database.

Figure (described): Example rows for the Student and Course tables after data is loaded.

Figure (described): Visual metaphor of an index speeding up searches in a large table.

Key terms

Entity: A thing or object about which we want to store information, such as Student or Book.

Attribute: A property or characteristic of an entity, like a student's name or a book's title.

Relationship: A connection between two entities, for example a Student borrows a Book.

Primary Key: A column (or set of columns) whose value uniquely identifies each row in a table.

Foreign Key: A column that references the primary key of another table to create a link between tables.

Normalization: The process of organizing tables to reduce redundancy and improve data integrity.

SQL SELECT: A query command that retrieves specific columns and rows from one or more tables.

Index: A data structure that speeds up searching on a column but uses extra storage.

We can make mistakes, and we'll be happy if you report any mistakes you find.